

Análises Ecológicas em Julia

Uma introdução à ciência de dados ecológicos com Julia

Eduardo Santos

03/10/2026

Índice

Prefácio	3
Escopo desta edição	3
Como usar este livro	3
Relação com <i>Análises Ecológicas no R</i>	4
Relação com o <i>ecoR</i>	4
Versão bilíngue	5
Como citar	5
Licença	5
I Fundamentos	6
1 Introdução	7
1.1 Por que Julia?	7
1.2 O custo honesto	7
1.3 A latência de compilação	8
1.4 Estrutura do livro	8
Exercícios	9
2 Da pergunta ao modelo	10
2.1 Por que um capítulo sobre perguntas em um livro de código	10
2.2 Hipóteses múltiplas de trabalho	10
2.3 Desenho amostral e estrutura de dependência	11
2.4 Perguntas limitadas por computação	11
Leitura recomendada	11
Exercícios	11
3 Pré-requisitos	12
3.1 Passo 1: abra um terminal	12
3.2 Windows	12
3.3 macOS	13
3.4 Linux	13
3.5 Passo 2: instale Julia	13
3.6 Windows	13
3.7 macOS e Linux	13
3.8 Passo 3: baixe a pasta do livro	14
3.9 Windows	14
3.10 macOS	14
3.11 Linux	14
3.12 Passo 4: instale o VS Code	15
3.13 Passo 5: abra a pasta do livro	15
3.14 Passo 6: instale os pacotes do livro	15
3.15 Passo 7: confira se tudo funciona	16
3.16 Passo 8: um primeiro script	16

3.17 Solução de problemas	18
3.18 Por dentro: ambientes de projeto	18
3.19 Por dentro: Quarto	20
Desafios	20
4 Introdução ao Julia	21
4.1 Vetores e indexação	21
4.2 Broadcasting: o ponto	23
4.3 Funções	24
4.4 Despacho múltiplo	25
4.5 Dados faltantes	26
4.6 Encadeamento	28
4.7 Estruturas de controle	28
4.8 Convenções de nomes	29
Desafios	30
II Manipulação e exploração de dados	31
5 Manipulação de dados com Tidier	32
5.1 Os dados	32
5.2 @chain: o pipe	33
5.3 Os cinco verbos	35
5.3.1 @filter — selecionar linhas	35
5.3.2 @select — selecionar colunas	36
5.3.3 @mutate — criar colunas	36
5.3.4 @group_by + @summarize — agregar	37
5.3.5 @arrange — ordenar	38
5.4 Dados faltantes	39
5.5 Formato longo e largo	40
5.6 Junções	41
5.7 DataFrames.jl por baixo	42
Desafios	42
6 Visualização de dados	44
6.1 TidierPlots.jl: a gramática do ggplot	45
6.2 AlgebraOfGraphics.jl: gramática algébrica	46
6.3 Facetas	48
6.4 Distribuições	50
6.5 Makie.jl puro: controle total	51
6.6 Salvando figuras	53
Desafios	54
7 Análise exploratória de dados	55
7.1 Primeiro contato: lendo o arquivo	56
7.2 Faltante, desconhecido e inconsistente	58
7.3 Estrutura: o que é uma réplica?	60
7.4 Distribuições	62
7.5 Erros e outliers	64
7.6 Relações e colinearidade	66
7.7 Por que os gráficos: o quarteto de Anscombe	70
Desafios	71

III Modelagem	72
8 Modelos lineares	73
8.1 Testes de hipótese	73
8.2 Regressão linear simples	75
8.3 Regressão múltipla	76
8.4 Preditores categóricos e interações	78
8.5 Seleção e comparação de modelos	79
8.6 Diagnóstico	80
8.7 Modelos mistos	82
Desafios	84
9 Modelos Lineares Generalizados	85
9.1 Por que GLMs em ecologia	85
9.2 Poisson: riqueza de espécies	86
9.3 Sobredispersão	88
9.4 Offsets	90
9.5 Regressão logística	91
9.6 GLMMs	94
9.7 Lacunas que você vai encontrar	95
9.7.1 A saída pragmática: RCall.jl	95
Desafios	96
IV Programação para ecologia	97
10 Escrevendo suas próprias funções	98
10.1 De um cálculo a uma função	99
10.2 Argumentos: valores padrão e palavras-chave	100
10.3 Recusando entradas ruins	101
10.4 Documentando uma função	103
10.5 A sua função, por grupo	104
10.6 Funções que recebem funções	106
10.7 Testando suas funções	108
Desafios	108
11 Reamostragem e simulação	109
11.1 Aleatoriedade que se reproduz	110
11.2 Testes de permutação	111
11.3 Permutando uma regressão	113
11.4 O bootstrap	115
11.5 Quantas lagoas eu preciso?	117
11.6 Um modelo nulo para coocorrência de espécies	120
Desafios	123
Referências	124
Histórico de versões	126
v1.0.0 (2026-10-03)	126

Prefácio

Este livro é uma introdução à análise de dados ecológicos usando a linguagem [Julia](#). Ele nasceu como uma resposta a uma pergunta simples: *o que da caixa de ferramentas do ecólogo já existe em Julia hoje, e o que ainda precisa ser construído?*

A ideia deste livro é inspirada no projeto super legal [Análises Ecológicas no R](#) (Da Silva et al. 2022), de Silva, Gonçalves-Souza, Paterno, Provete e Vancine — um livro aberto, em português, que se tornou porta de entrada para uma geração de ecólogos brasileiros. A estrutura deste volume segue deliberadamente a daquele livro, capítulo a capítulo, para que os dois possam ser lidos lado a lado.

Escopo desta edição

! O que este livro cobre — e o que ainda não cobre

Esta primeira edição vai **do básico da linguagem até modelos lineares generalizados** (capítulos 1-9), com um capítulo de análise exploratória de dados antes dos modelos, seguidos de dois capítulos de programação para ecologia: escrever as próprias funções e reamostragem e simulação (capítulos 10-11). Os capítulos de ecologia de comunidades do livro original (*Análises ecológicas no R*) — análises multidimensionais, rarefação, estimadores de riqueza, diversidade taxonômica, filogenética e funcional, e dados geoespaciais — **não estão incluídos**.

Isso não é um descuido. Em R, esses capítulos utilizam pacotes como `vegan`, `iNEXT`, `picante` e `FD`. Julia hoje não tem equivalentes maduros desses pacotes. Escrever aqueles capítulos honestamente exige escrever os pacotes primeiro, e esse é um projeto separado (que não é para mim). Veja o Seção [1.2](#) para uma avaliação detalhada.

Esse recorte não é pequeno. Ele cobre tudo o que a maior parte de uma análise ecológica realmente consome: importar e limpar dados, visualizá-los, ajustar modelos lineares e generalizados, e lidar com estrutura hierárquica via modelos mistos. Em todos esses pontos, o ecossistema Julia é sólido: os pacotes existem, são mantidos, e fazem muito bem o trabalho. Onde este livro faz comparações com R, elas se limitam ao que pode ser verificado — sintaxe, cobertura de funcionalidade, comportamento demonstrável no código — e não a afirmações de desempenho que eu não meça aqui.

Como usar este livro

Todo o código é executável e roda no ambiente Julia definido em `Project.toml`. Para reproduzir o livro na sua máquina:

```
git clone https://github.com/esantos2ua/EcologicalAnalysesInJulia.git
cd EcologicalAnalysesInJulia
julia --project=. scripts/setup.jl
quarto render pt
```

Ao longo do texto, caixas como esta traduzem o idioma do R para o de Julia:

Em R

`dplyr::filter(dados, altura > 10)` corresponde a `@filter(dados, altura > 10)` em `TidierData.jl`. A semântica é a mesma; a diferença é que `@filter` é um macro, não uma função.

Relação com *Análises Ecológicas no R*

Este projeto é **inspirado em** *Análises Ecológicas no R* (Da Silva et al. 2022), de Fernando Rodrigues da Silva, Thiago Gonçalves-Souza, Gustavo Brant Paterno, Diogo Borges Provete e Maurício Humberto Vancine, e tem com ele uma dívida estrutural explícita: a sequência de capítulos deste livro segue deliberadamente a daquele, para que os dois possam ser lidos lado a lado.

O que este livro **não** faz é reproduzir o conteúdo do original. O texto, os exemplos, o conjunto de dados e os exercícios aqui são escritos do zero para Julia. Os autores do livro original não têm envolvimento neste projeto, não o revisaram e não o endossam; qualquer erro aqui é meu.

Análises Ecológicas no R está disponível gratuitamente em <https://analises-ecologicas.com/> sob licença CC BY-NC 4.0, e em edição impressa pela Nupeea e Canal 6 (2022, 640 p., ISBN 978-85-7917-564-0). Se este livro lhe for útil, o original provavelmente lhe será mais ainda.

Relação com o *ecoR*

Os capítulos 7, 10 e 11 não vêm de *Análises Ecológicas no R*. Eles são **inspirados** nos módulos de análise exploratória, de programação e de reamostragem do **ecoR** (Oliveira 2026), o wiki do curso de R do Instituto de Biociências da Universidade de São Paulo, produzido e mantido por Alexandre Adalardo de Oliveira, e baseado em parte no material de João Luis Ferreira Batista e Paulo Inácio K. L. Prado. Como no caso do livro original, o texto, os exemplos e os exercícios daqui são escritos do zero; os autores do *ecoR* não revisaram este livro. Cada um dos três capítulos começa com links para o material correspondente do *ecoR*.

O *ecoR* está disponível gratuitamente em <https://ecor.ib.usp.br/> sob licença CC BY-SA 4.0, com exercícios corrigidos automaticamente no *notaR*.

Versão bilíngue

Este livro existe em português e inglês, a partir do mesmo repositório. A versão em inglês está em [/en](#). Os capítulos são traduções um do outro, e o código é idêntico nas duas versões. Eu estou escrevendo predominantemente em português e usando ferramentas de IA para traduzir o conteúdo para inglês (com revisões minhas).

Como citar

Cada versão do livro é arquivada no [Zenodo](#) com um DOI. Se você usar este livro, cite assim:

Santos, E. (2026). *Ecological Analyses in Julia* (versão 1.0.0). Zenodo. <https://doi.org/10.5281/zenodo.23126524>

O título do registro é o da edição em inglês. As duas edições, em português e em inglês, fazem parte do mesmo registro. O DOI acima aponta sempre para a versão mais recente. Para citar exatamente a versão que você leu, use o DOI dessa versão, listado na página do Zenodo, e veja o [Histórico de versões](#).

Em BibTeX:

```
@book{santos2026ecological,  
  author    = {Santos, Eduardo},  
  title     = {Ecological Analyses in Julia},  
  year      = {2026},  
  version   = {1.0.0},  
  publisher = {Zenodo},  
  doi       = {10.5281/zenodo.23126524},  
  url       = {https://doi.org/10.5281/zenodo.23126524}  
}
```

Licença

Texto sob [CC BY-NC-SA 4.0](#); código sob [licença MIT](#). Contribuições, correções e relatos de erro são bem-vindos no repositório.

Parte I

Fundamentos

1 Introdução

1.1 Por que Julia?

Julia foi criada para resolver o que seus autores chamaram de *problema das duas linguagens* (Bezanson et al. 2017): você prototipa em uma linguagem alta e produtiva (R, Python) e, quando o código fica lento demais, reescreve o gargalo em C, C++ ou Fortran. O resultado é que a biblioteca que você usa e o código que você escreve são escritos em linguagens diferentes — o que torna o interior dos pacotes opaco para o usuário.

Em Julia, `vegan` seria escrito em Julia. Isso importa para o ecólogo por três razões práticas:

1. **Você consegue ler o pacote que está usando.** Quando `metaMDS` faz algo inesperado, a resposta está em Fortran. Em Julia, `@edit` te leva ao código-fonte legível da função.
2. **Simulação deixa de ser proibitiva.** Modelos nulos, bootstrap, aleatorização — o pão-com-manteiga da ecologia de comunidades — rodam em loops. Loops em Julia são rápidos.
3. **Os pacotes compõem.** Tipos definidos em um pacote funcionam com funções de outro sem código de cola, por causa do despacho múltiplo (Seção 4.4).

1.2 O custo honesto

Esses benefícios não são de graça, e este livro não vai fingir o contrário. Deixa eu mostrar alguns benefícios e custos.

O que você ganha	O que você perde
Loops rápidos sem vetorizar	Ecossistema de ecologia de comunidades (<code>vegan</code> , <code>picante</code> , <code>FD</code>)
Código de pacote legível <code>MixedModels.jl</code> com a sintaxe de fórmula do <code>lme4</code>	Modelos aditivos generalizados (<code>mgcv</code>) Diagnóstico de resíduos ao estilo <code>DHARMa</code>
Composição via despacho múltiplo	Massa crítica de tutoriais e respostas no Stack Overflow
Um único ambiente reprodutível (<code>Project.toml</code> + <code>Manifest.toml</code>)	Latência de compilação na primeira chamada

i A validade desta tabela

A coluna da direita é uma avaliação do ecossistema Julia em **agosto de 2026**, feita consultando os registros de pacotes públicos. Ecossistemas mudam. Antes de concluir que algo não existe, verifique — e se encontrar um pacote que preencha uma destas lacunas, o capítulo está errado e deve ser corrigido. Vou tentar atualizar com certa frequência, mas certamente não conseguirei manter tudo em dia.

i Quando *não* usar Julia

Se sua análise é essencialmente `vegan::adonis2()` seguido de `metaMDS()`, use R. Este livro cobre até GLMs justamente porque é até aí que a recomendação “use Julia” se sustenta sem ressalvas.

1.3 A latência de compilação

Julia compila cada método na primeira vez que ele é chamado com um dado conjunto de tipos. Isso é o que torna a segunda (terceira, quarta, ...) chamada rápida, e a primeira, lenta.

```
soma_quadrados(v) = sum(x^2 for x in v)

v = randn(1_000_000)

# primeira chamada: inclui a compilação
@elapsed soma_quadrados(v)
```

0.010643709

```
# segunda chamada: só a execução
@elapsed soma_quadrados(v)
```

0.000765041

A diferença entre as duas chamadas é o tempo de compilação. Em uma sessão de análise real esse custo é pago uma vez. Em um script curto que roda uma única vez e termina, esse custo de compilação pode ter um impacto perceptível. Vale entender um pouco sobre o fenômeno antes de concluir que “Julia é uma linguagem lenta”.

1.4 Estrutura do livro

- **Fundamentos** (Capítulo 1 – Capítulo 4): por que Julia, como formular a pergunta, como instalar, e a linguagem em si.

- **Manipulação e visualização** (Capítulo 5 – Capítulo 6): `TidierData.jl` e a família `Makie`.
- **Modelagem** (Capítulo 8 – Capítulo 9): `GLM.jl`, `MixedModels.jl`, `HypothesisTests.jl`.

Exercícios

1. Instale Julia via `juliaup` e reproduza o exemplo de latência acima. Quanto tempo sua primeira chamada leva?
2. Liste três análises que você faz com frequência. Para cada uma, procure no [JuliaHub](#) se existe um pacote equivalente.

2 Da pergunta ao modelo

! Status deste capítulo

Este capítulo ainda não foi totalmente escrito. O que segue é um **plano**, com as referências. Ainda preciso elaborar muita coisa aqui.

O plano é meu, não um resumo de nenhuma outra obra. Ele não reproduz o conteúdo do capítulo correspondente de Da Silva et al. (2022), que não consultei ao escrevê-lo.

2.1 Por que um capítulo sobre perguntas em um livro de código

Nenhuma quantidade de sofisticação estatística resgata uma pergunta mal formulada. A ordem correta é: pergunta → desenho amostral → dados → modelo → inferência (poderia escrever um livro inteiro sobre esse processo todo). Ferramentas como Julia atuam nos dois últimos passos, e são pouco úteis se os dois primeiros (pergunta e desenho amostral) não forem elaborados com carinho.

Este capítulo existe para ligar as duas metades do livro: a pergunta que você faz determina a estrutura do modelo que aparece no Capítulo 8 e no Capítulo 9.

2.2 Hipóteses múltiplas de trabalho

A seção será construída sobre Chamberlin (1890), que propôs manter várias hipóteses rivais em jogo simultaneamente em vez de defender uma única — um argumento de 1890 que continua sendo a melhor descrição do que distingue uma pergunta produtiva de uma pergunta defensiva.

A desenvolver:

- Observação, padrão e processo: os três níveis, e por que confundi-los leva a modelos que respondem outra pergunta.
- Hipóteses mecanicistas versus fenomenológicas, e o que cada tipo permite concluir.

2.3 Desenho amostral e estrutura de dependência

A seção será construída sobre Hurlbert (1984), que nomeou a pseudorreplicação e catalogou suas formas, e sobre Zuur et al. (2009), cujo tratamento de estrutura de dependência conecta diretamente aos efeitos aleatórios usados neste livro.

A desenvolver:

- Unidade amostral, unidade experimental, e por que a distinção determina o denominador do seu teste.
- Como a estrutura do desenho se traduz em termos (1 | grupo) — o elo direto com o
 - 1.
- Tamanho de efeito e poder antes da coleta, não depois.

2.4 Perguntas limitadas por computação

Uma seção específica deste livro. Certas perguntas ecológicas são limitadas por computação, não por dados: modelos nulos com muitas permutações, simulação baseada em indivíduos, ajuste de modelos mecanicistas por otimização. São essas que mais se beneficiam da linguagem, pelas razões discutidas no Seção 1.1.

Esta seção precisa ser escrita com exemplos executáveis, como o resto do livro — não com afirmações de desempenho não verificadas.

Leitura recomendada

- Chamberlin (1890) sobre hipóteses múltiplas de trabalho.
- Hurlbert (1984) sobre pseudorreplicação e desenho experimental.
- Zuur et al. (2009), capítulo 1, sobre desenho e estrutura de dependência.
- Da Silva et al. (2022), para o tratamento do mesmo tema em R.

Exercícios

1. Escreva a pergunta central do seu projeto em uma frase, sem jargão estatístico.
2. Desenhe o modelo que ela implica *antes* de olhar para os dados: que resposta, que preditores, que estrutura de agrupamento?
3. Identifique a unidade experimental do seu desenho. Quantas réplicas verdadeiras você tem — não quantas medidas?

3 Pré-requisitos

Este capítulo prepara o seu computador para o resto do livro. Ao final, você terá instalado Julia e um editor, baixado os dados e os exercícios do livro e rodado um primeiro script que desenha um gráfico.

Siga os passos na ordem. Onde eles mudam de um sistema operacional para outro, escolha a aba do seu sistema. Reserve cerca de uma hora na primeira vez — a maior parte é espera enquanto os pacotes são instalados — e mantenha a conexão com a internet até concluir a Seção 3.14. Se algum passo não se comportar como descrito, veja a Seção 3.17.

i O que você vai instalar

O quê	Para quê
Julia (pelo juliaup) VS Code e a extensão de Julia A pasta do livro Os pacotes do livro	a linguagem em si onde você escreve e roda o código dados, exercícios e a lista de pacotes que o livro usa data frames, gráficos e modelos, nas versões exatas com que o livro foi gerado

Você não precisa de git, R nem Quarto.

3.1 Passo 1: abra um terminal

Um terminal é uma janela onde você digita comandos em vez de clicar. Você só precisa dele para instalar Julia. Depois disso, tudo o que vamos fazer aqui neste livro acontece dentro do VS Code.

3.2 Windows

Abra o menu Iniciar, digite **PowerShell** e abra o *Windows PowerShell* (ou o *Terminal*, no Windows 11).

3.3 macOS

Pressione Cmd+Espaço, digite **Terminal** e pressione Return.

3.4 Linux

Pressione Ctrl+Alt+T na maioria das distribuições, ou procure *Terminal* no menu de aplicativos.

Para rodar um comando, cole-o no terminal e pressione Enter. Colar é Ctrl+V no Windows, Cmd+V no macOS e Ctrl+Shift+V na maioria dos terminais Linux.

3.5 Passo 2: instale Julia

Use o **juliaup**, o instalador e gerenciador oficial de versões. Não instale Julia pelo gerenciador de pacotes do sistema (apt, brew), que tende a distribuir versões desatualizadas.

3.6 Windows

```
winget install --name Julia --id 9NJNWW8PVKMN -e -s msstore
```

Se o Windows pedir que você aceite os termos da Microsoft Store, digite Y e pressione Enter.

3.7 macOS e Linux

```
curl -fsSL https://install.julialang.org | sh
```

O instalador pergunta se deve seguir com as configurações padrão. Pressione Enter para aceitá-las.

Quando terminar, **feche o terminal e abra um novo** — o comando julia só fica disponível em terminais novos. Depois, confira a instalação:

```
julia --version
```

Deve aparecer algo como julia version 1.12.6. Qualquer versão a partir da 1.10 funciona com este livro. Agora digite julia e pressione Enter para abrir o Julia propriamente dito. O prompt muda para julia>; digite 1 + 1, pressione Enter, e Julia responde 2. Saia com exit().

Mais tarde, `juliaup update` atualiza o Julia e `juliaup status` lista as versões instaladas.

3.8 Passo 3: baixe a pasta do livro

O livro vem com uma pasta que contém os dados de exemplo, o verificador de exercícios e a lista de pacotes de que o livro precisa.

1. Acesse <https://github.com/esantos2ua/EcologicalAnalysesInJulia>.
2. Clique no botão verde **Code** e depois em **Download ZIP**.
3. Descompacte o arquivo:

3.9 Windows

Clique com o botão direito no arquivo baixado e escolha **Extrair Tudo...**. Trabalhe na pasta extraída, não dentro do ZIP: arquivos abertos de dentro de um ZIP são cópias somente leitura.

3.10 macOS

Dê um duplo clique no arquivo baixado.

3.11 Linux

Clique com o botão direito no arquivo e escolha *Extrair aqui*, ou rode `unzip` nele.

4. Mova a pasta extraída, `EcologicalAnalysesInJulia-main`, para um lugar permanente, como a pasta *Documentos*. Você pode renomeá-la, mas evite deixá-la em *Downloads*, que alguns sistemas limpam automaticamente.

Dentro dela você deve ver, entre outras coisas, as pastas `data` e `exercises` e os arquivos `Project.toml` e `Manifest.toml`.

💡 Se você já usa git

Clone o repositório em vez de baixar o ZIP; depois, `git pull` traz as atualizações do livro:

```
git clone  
↪ https://github.com/esantos2ua/EcologicalAnalysesInJulia.git
```

3.12 Passo 4: instale o VS Code

O **VS Code** com a extensão oficial de Julia é o editor padrão para Julia: oferece REPL integrado (o console onde o código roda), navegador de variáveis, painel de gráficos, depurador e avaliação linha a linha — o equivalente funcional do RStudio.

1. **Instale o VS Code** a partir de <https://code.visualstudio.com> e abra-o.
2. **Instale a extensão de Julia.** Abra o painel de extensões (Ctrl+Shift+X; Cmd+Shift+X no macOS), procure por *Julia* e instale a extensão publicada por *juliaup*. Ela encontra sozinha o Julia instalado pelo *juliaup*.

Se você prefere uma versão de código aberto, o **VSCodium** é o VS Code compilado a partir do mesmo código-fonte sob licença MIT, sem a telemetria da Microsoft, e instala a mesma extensão de Julia pelo registro **Open VSX**. O **Antigravity**, outro editor derivado do VS Code, também roda a mesma extensão. Tudo neste livro vale para os três (pelo menos até onde eu testei). Outras opções são o **Pluto.jl** (notebooks reativos, excelentes para ensino) e qualquer editor com suporte a LSP via `LanguageServer.jl`.

3.13 Passo 5: abra a pasta do livro

1. **Abra uma pasta, não um arquivo.** No VS Code, use *File → Open Folder...* e escolha a pasta do livro da Seção 3.8. Se o VS Code perguntar se você confia nos autores dos arquivos, responda que sim. A pasta aberta determina tanto onde Julia procura arquivos quanto quais pacotes ela usa.
2. **Confira o ambiente.** A barra de status, na parte de baixo da janela, deve mostrar Julia env: seguido do nome da pasta do livro. Se aparecer algo como Julia env: v1.12, clique nessa entrada e escolha a pasta do livro.
3. **Inicie o REPL.** Abra a paleta de comandos (Ctrl+Shift+P; Cmd+Shift+P no macOS) e rode *Julia: Start REPL*, ou pressione Alt+J e depois Alt+O. Um painel se abre embaixo com o prompt `julia>`, já dentro da pasta do livro e do seu ambiente.

Em R

Abrir uma pasta no VS Code cumpre o papel de abrir um `.Rproj` no RStudio: define o diretório de trabalho. Diferente do RStudio, também seleciona o ambiente de pacotes (Seção 3.18), de modo que as versões certas dos pacotes vêm junto automaticamente.

3.14 Passo 6: instale os pacotes do livro

Clique no painel do REPL e rode:

```
using Pkg
Pkg.instantiate()
```

Isso lê o `Manifest.toml`, baixa cada pacote que o livro usa na versão exata listada ali e os compila.

⚠ Isso demora

Na primeira vez, `Pkg.instantiate()` pode levar de vários minutos a meia hora, dependendo do seu computador e da sua conexão. Os pacotes de gráficos e de modelos são grandes, e Julia os compila antecipadamente para que rodem rápido depois. Linhas de progresso continuam aparecendo enquanto ele trabalha; deixe terminar e não feche o VS Code. Você só faz isso uma vez por computador.

i O que significa compilar?

Compilar significa traduzir o código legível por humanos diretamente para linguagem de máquina — as instruções que o processador do computador executa nativamente.

Enquanto linguagens como R e Python interpretam o código linha por linha durante a execução, Julia faz essa tradução com antecedência (na chamada *pré-compilação*). Isso exige paciência inicial ao instalar os pacotes, mas é o que garante que suas análises e gráficos rodem depois com velocidade máxima (veja mais sobre a latência de compilação na Seção 1.3).

3.15 Passo 7: confira se tudo funciona

A pasta do livro inclui um verificador, que você também vai usar nos exercícios a partir do Capítulo 4. Carregue-o e peça que ele confira a sua instalação:

```
include("exercises/checar.jl")
checar("setup")
```

Se estiver tudo certo, você verá:

- ✓ Julia 1.12.6
 - ✓ A pasta de trabalho é a pasta do livro
 - ✓ O ambiente do livro está ativo
 - ✓ Arquivo de dados encontrado: data/ponds.csv
 - ✓ Todos os 19 pacotes instalados
- Tudo pronto. Siga para o primeiro script.

Uma linha com `␣` diz o que está errado e como corrigir; a Seção 3.17 tem mais detalhes.

3.16 Passo 8: um primeiro script

Com a pasta do livro aberta e o REPL rodando, crie um arquivo chamado `primeiros-passos.jl` na raiz da pasta (*File* → *New File*, ou `Alt+J` e depois `Alt+N`) e cole:

```
## Carregar pacotes
using CSV, DataFrames, CairoMakie
```

```
## Onde estou?
pwd()          # a pasta que você abriu
readdir("data") # arquivos dentro de data/

## Ler os dados de exemplo
ponds = CSV.read(joinpath("data", "ponds.csv"), DataFrame)
first(ponds, 5)
describe(ponds)

## Gráfico de riqueza contra área da lagoa
fig = Figure()
ax = Axis(fig[1, 1]; xlabel = "Área da lagoa (m²)", ylabel = "Riqueza
↳ de espécies",
          xscale = log10)
scatter!(ax, ponds.area_m2, ponds.richness)
fig
```

Linhas que começam com `##` dividem o script em **células**. Coloque o cursor na primeira célula e pressione `Alt+Enter` para rodá-la; depois siga célula por célula (`Alt+Shift+Enter` roda a célula e pula para a próxima). Algumas coisas a observar:

- **Carregar pacotes é lento na primeira vez** em cada sessão — é a latência de compilação discutida na Seção 1.3. As células seguintes rodam na hora.
- **Os caminhos são relativos à pasta que você abriu.** `pwd()` confirma isso, e é por isso que `joinpath("data", "ponds.csv")` funciona sem digitar o caminho completo. `joinpath` insere o separador correto em qualquer sistema operacional.
- **`describe` é a sua primeira checagem dos dados.** Ele mostra que canopy tem 7 valores ausentes e que `area_m2` é fortemente assimétrica à direita (mediana perto de 149 m², máximo perto de 2000 m²) — por isso o gráfico usa escala logarítmica.

Os atalhos que você vai usar o tempo todo:

Atalho	Ação
Shift+Enter	roda a linha atual ou a seleção e passa para a próxima
Ctrl+Enter	roda a linha atual ou a seleção, sem sair do lugar
Alt+Enter	roda a célula atual (um bloco delimitado por linhas <code>##</code>)
Alt+J, Alt+R	reinicia o REPL
Ctrl+C no REPL	interrompe uma computação em andamento

No macOS, `Alt` é a tecla `Option`; `Ctrl` continua sendo `Ctrl`. Atalhos e nomes de menu foram conferidos na versão 1.249 da extensão de Julia em setembro de 2026.

Inspecionando o ambiente. Clique no ícone de Julia na barra de atividades (a faixa à esquerda). O painel **Workspace** lista todas as variáveis da sessão — `ponds`, `fig`, `ax` — com tipo e tamanho; expanda uma entrada para ver o conteúdo. Para um data frame, passe o mouse sobre o nome e clique no ícone *Open in VS Code*, ou rode

`vscode::display(ponds)` no REPL, para abri-lo em uma grade no estilo de planilha — o equivalente ao `View()` do RStudio.

Gráficos. Avaliar `fig` abre a figura no **painel de gráficos** (*plot pane*), uma aba do VS Code. O painel **Plot Navigator**, abaixo do Workspace, guarda o histórico de gráficos da sessão; com o painel de gráficos em foco, as setas do teclado navegam por ele. Para salvar uma figura em disco, use `save("riqueza-area.png", fig)`.

3.17 Solução de problemas

julia não é reconhecido / command not found. Feche todos os terminais e abra um novo; o instalador só atualiza terminais novos. Se ainda falhar, rode de novo o comando de instalação da Seção 3.5.

Windows: winget não é reconhecido. O winget vem com o aplicativo *Instalador de Aplicativo* (*App Installer*). Atualize-o pela Microsoft Store, ou procure **Julia** na Microsoft Store e instale por lá — é o mesmo instalador `juliaup`.

Linux: curl: command not found. Instale o curl pelo gerenciador de pacotes da sua distribuição (por exemplo, `sudo apt install curl` no Ubuntu) e repita a Seção 3.5.

O VS Code diz que não encontra o Julia. Em um terminal, rode `where.exe julia` (Windows) ou `which julia` (macOS, Linux) e copie o caminho exibido. No VS Code, abra as configurações (`Ctrl+,`; `Cmd+,` no macOS), procure por `julia.executablePath` e cole o caminho ali. Depois, reinicie o VS Code.

A barra de status mostra Julia env: v1.12 (ou outra versão) em vez da pasta do livro.

O VS Code está com a pasta errada aberta, ou com um único arquivo. Use *File* → *Open Folder...* e escolha a própria pasta do livro — a que contém o `Project.toml` —, não uma pasta acima ou dentro dela. Depois, reinicie o REPL (`Alt+J`, `Alt+R`).

`include("exercises/checar.jl")` falha com No such file or directory. O REPL não está rodando na pasta do livro. `pwd()` mostra onde ele está. Abra a pasta do livro como descrito acima e reinicie o REPL.

`Pkg.instantiate()` parece travado. Provavelmente está compilando; essa é a parte lenta. Enquanto novas linhas continuarem aparecendo a cada poucos minutos, espere. Se ele parou com um erro — por exemplo, porque a conexão caiu —, rode `Pkg.instantiate()` de novo; ele continua de onde parou.

Package CairoMakie not found in current path (ou outro pacote). Ou o ambiente do livro não está ativo (confira a barra de status), ou a Seção 3.14 não terminou. `checar("setup")` diz qual dos dois.

O primeiro `using CairoMakie` de cada sessão leva um minuto. É normal: Julia compila o código na primeira vez que ele roda em uma sessão (Seção 1.3). Os comandos seguintes são rápidos.

3.18 Por dentro: ambientes de projeto

Você já usou um ambiente sem precisar pensar nele: é o que a barra de status da Seção 3.13 mostra. Esta seção explica o que ele é — o conceito que mais difere da prática usual em R.

Em R, `install.packages()` instala em uma biblioteca global compartilhada por todos os seus projetos. Em Julia, cada projeto tem seu próprio ambiente, descrito por dois arquivos:

- **Project.toml** — as dependências diretas que você declarou.
- **Manifest.toml** — a árvore completa e resolvida de dependências, com as versões exatas de tudo.

Juntos eles tornam a análise reproduzível por construção: quem tiver a pasta do seu projeto e rodar `Pkg.instantiate()` obtém exatamente as suas versões — foi o que você fez na Seção 3.14.

Em R

`Project.toml` + `Manifest.toml` cumprem o papel de `renv.lock`, mas fazem parte da linguagem, não de um pacote adicional. Ativar um ambiente é a operação padrão, não a exceção.

`Pkg.status()` lista os pacotes do ambiente ativo:

```
using Pkg
Pkg.status()
```

```
Status `~/work/EcologicalAnalysesInJulia/EcologicalAnalysesInJulia/Project.toml`
^ [cbdf2221] AlgebraOfGraphics v0.13.1
⚡ [336ed68f] CSV v0.10.17
^ [13f3f980] CairoMakie v0.15.13
⚡ [324d7699] CategoricalArrays v0.10.9
[a93c6f00] DataFrames v1.8.2
[31c24e10] Distributions v0.25.131
[8f03c58b] Effects v1.7.1
[38e38edf] GLM v1.9.5
^ [09f84164] HypothesisTests v0.11.8
^ [ff71e718] MixedModels v5.8.2
⚡ [ce6b1742] RDatasets v0.7.7
^ [10745b16] Statistics v1.11.4
[2913bbd2] StatsBase v0.34.13
[3eaba693] StatsModels v0.7.10
[79ddc9fe] TidierCats v0.2.2
[fe2206b3] TidierData v0.17.2
[8ae5e7a9] TidierFiles v0.3.2
[337ecbd1] TidierPlots v1.0.0
[9a3f8284] Random v1.11.0
```

Info Packages marked with ^ and ⚡ have new versions available. Those with ^ may be upgraded
outdated`

Este livro foi gerado com:

```
versioninfo()
```

```
Julia Version 1.12.6
Commit 15346901f00 (2026-04-09 19:20 UTC)
Build Info:
  Official https://julialang.org release
Platform Info:
```

```
OS: Linux (x86_64-linux-gnu)
CPU: 4 × AMD EPYC 7763 64-Core Processor
WORD_SIZE: 64
LLVM: libLLVM-18.1.7 (ORCJIT, znver3)
GC: Built with stock GC
Threads: 1 default, 1 interactive, 1 GC (on 4 virtual cores)
Environment:
  JULIA_LOAD_PATH = @:stdlib
```

3.19 Por dentro: Quarto

Você não precisa desta seção para acompanhar o livro. Ela é para quem quer gerar o livro por conta própria ou escrever documentos parecidos.

Este livro é escrito em [Quarto](#), sucessor direto do bookdown. Desde a versão 1.5, Quarto executa blocos Julia nativamente, sem passar por Jupyter.

```
```${julia}
#| label: exemplo
1 + 1
```
```

O README.md do repositório explica como gerar o livro.

Desafios

1. No REPL, pressione] para entrar no modo de pacotes (o prompt fica azul; Backspace sai dele). Rode activate meu-teste e depois add DataFrames: isso cria um ambiente em uma pasta nova, meu-teste. Inspeção o Project.toml gerado e depois rode activate . para voltar ao ambiente do livro.
2. Apague meu-teste/Manifest.toml, ative meu-teste de novo e rode resolve. O que muda?
3. Rode o primeiros-passos.jl (Seção 3.16) célula por célula. No painel Workspace, que tipos o CSV.jl atribuiu às colunas region e canopy, e por que o tipo de canopy é diferente do de area_m2?

4 Introdução ao Julia

Este capítulo apresenta a linguagem para quem já programa em R. O foco está nas diferenças que importam na prática, não em uma varredura exaustiva da sintaxe.

```
using Statistics
using Random
Random.seed!(42)
```

TaskLocalRNG()

i Como funcionam os exercícios

A maioria das seções termina com um quadro *Sua vez*. Resolva no VS Code, com a pasta do livro aberta (Seção 3.16), e carregue o verificador uma vez por sessão:

```
include("exercises/checar.jl")
```

Depois passe sua resposta — um valor, ou uma função que você escreveu — junto com o número do exercício:

```
checar("4.3", simpson)
```

checar diz se a resposta está certa e, se não estiver, o que ela fez e o que era esperado. Cada quadro tem uma dica e uma solução recolhidas: tente primeiro, depois espie.

4.1 Vetores e indexação

```
alturas = [12.4, 8.1, 15.7, 9.3, 11.0]
length(alturas), first(alturas), last(alturas)
```

(5, 12.4, 11.0)

Duas diferenças imediatas em relação ao R:

```
alturas[1]          # indexação começa em 1, como em R
alturas[2:4]        # intervalos são inclusivos nas duas pontas
```

3-element Vector{Float64}:

```
8.1
15.7
9.3
```

```
# Julia NÃO recicla vetores silenciosamente:
try
    [1, 2, 3] + [1, 2]
catch e
    e
end
```

DimensionMismatch("a has size (3,), b has size (2,), mismatch at dim 1")

Em R

`c(1,2,3) + c(1,2)` recicla o vetor mais curto e emite apenas um aviso; Julia lança `DimensionMismatch`, como o bloco acima demonstra. A diferença é de política: R prioriza seguir adiante, Julia prioriza falhar cedo. Qual serve melhor depende do seu fluxo de trabalho — o que importa é saber qual das duas você está usando.

i Sua vez 4.1 — Tire um valor errado

Estas são as áreas (m²) de cinco lagoas. A última é um erro de digitação.

```
areas = [174.1, 161.6, 233.9, 174.5, 1980.0]
```

5-element Vector{Float64}:

```
174.1
161.6
233.9
174.5
1980.0
```

Crie `limpo`, com todas as lagoas menos a última. Não digite a posição 4: seu código deve continuar funcionando se mais lagoas forem adicionadas.

Dica

Dentro dos colchetes, `end` significa “a última posição”; logo, `end - 1` é a anterior.

Solução

```
limpo = areas[1:end-1]
checar("4.1", limpo)
```

✓ Correto!

4.2 Broadcasting: o ponto

Em R, quase toda função é vetorizada por padrão. Em Julia, você declara a vetorização com um ponto. Isso é explícito e vale para *qualquer* função, inclusive as que você escreve.

```
log.(alturas)           # aplica log elemento a elemento
alturas .^ 2            # potência elemento a elemento
alturas .> 10           # comparação elemento a elemento
```

5-element BitVector:

```
1
0
1
0
1
```

```
# funciona igual com uma função sua
padroniza(x, μ, σ) = (x - μ) / σ

padroniza.(alturas, mean(alturas), std(alturas))
```

5-element Vector{Float64}:

```
0.3724002529860428
-1.0833461905048525
1.4896010119441712
-0.6770913690655326
-0.10156370535983013
```

💡 A regra do ponto

Se você quer “aplicar isso a cada elemento”, ponha um ponto. $f.(x)$, $x .+ y$, $x .== y$. Uma cadeia inteira pode ser pontuada de uma vez com $@.$:

```
@. log(alturas) + 1
```

5-element Vector{Float64}:

```
3.517696472610991
3.0918640616783932
3.7536607123542622
3.2300144001592104
3.3978952727983707
```

i Sua vez 4.2 — Log das áreas

O gráfico de área em Seção 3.16 usou escala log, porque as áreas das lagoas são muito assimétricas à direita. Calcule o logaritmo na base 10 de cada valor de áreas e guarde em log_areas.

Dica

log10(areas) dá erro. Leia a mensagem e releia a regra do ponto logo acima.

Solução

```
log_areas = log10.(areas)
checar("4.2", log_areas)
```

✓ Correto!

log10 é definida para um número; o ponto a aplica a cada elemento.

4.3 Funções

```
# forma curta, para funções de uma linha
riqueza(abundancias) = count(>(0), abundancias)

# forma longa
function shannon(abundancias)
    total = sum(abundancias)
    p = abundancias ./ total
    p = filter(>(0), p)          # log(0) é indefinido
    return -sum(p .* log.(p))
end

comunidade = [45, 12, 0, 8, 23, 0, 3]
riqueza(comunidade), shannon(comunidade)
```

(5, 1.289250407583511)

i Sua vez 4.3 — Índice de Simpson

Escreva simpson(abundancias), que retorna o índice de Simpson $1 - \sum p_i^2$, onde p_i é a proporção de indivíduos da espécie i . As primeiras linhas de shannon já calculam as proporções.

Antes de chamar checar, teste simpson([3, 1]) você mesmo: deve dar 0.375.

Dica

`p .^ 2` eleva cada proporção ao quadrado e `sum` soma tudo. Zeros não precisam de cuidado especial aqui: $0^2 = 0$.

Solução

```
function simpson(abundancias)
    p = abundancias ./ sum(abundancias)
    return 1 - sum(p .^ 2)
end

checar("4.3", simpson)
```

✓ Correto!

4.4 Despacho múltiplo

Esta é a ideia central da linguagem, e a razão pela qual pacotes Julia compõem sem código de cola. Uma função pode ter vários **métodos**, e Julia escolhe qual rodar com base nos tipos de *todos* os argumentos.

```
# um método para contagens inteiras
diversidade(x::Vector{<:Integer}) = "índice para dados de contagem"

# outro para coberturas proporcionais
diversidade(x::Vector{<:AbstractFloat}) = "índice para dados de
    ↪ cobertura"

diversidade([45, 12, 8]), diversidade([0.45, 0.12, 0.08])
```

("índice para dados de contagem", "índice para dados de cobertura")

Na prática, isso significa que quando alguém define um novo tipo de dado ecológico, `mean`, `plot` e `fit` passam a funcionar com ele desde que os métodos apropriados sejam definidos — sem que os autores de `Statistics` ou `Makie` saibam da existência do novo tipo.

i Sua vez 4.4 — Um índice por local

`simpson` trata uma comunidade. Dados de campo costumam vir como uma matriz locais $\times$ espécies, uma linha por local. Adicione um segundo método, `simpson(M::Matrix)`, que retorne um índice por linha. O método para vetores tem que continuar funcionando.

Dica

`eachrow(M)` percorre as linhas uma a uma. Combine com o ponto, ou com uma compreensão: `[f(linha) for linha in eachrow(M)]`. Se `checar`

reclamar dos tipos dos argumentos, leia a mensagem com atenção — é uma armadilha comum.

Solução

```
simpson(M::Matrix) = simpson.(eachrow(M))

locais = [45 12 0 8;
          5  5  5 5;
          30 0  0 0]
simpson(locais)
```

```
3-element Vector{Float64}:
 0.47147928994082844
 0.75
 0.0
```

```
checar("4.4", simpson)
length(methods(simpson))
```

✓ Correto!

2

O método para matrizes reaproveita o método para vetores em cada linha. Uma função, dois métodos, e Julia escolhe pelo tipo do argumento; rode `methods(simpson)` no REPL para ver os dois.

4.5 Dados faltantes

```
medidas = [12.4, missing, 15.7, 9.3]

mean(medidas)           # propaga, como em R
```

missing

```
mean(skipmissing(medidas))  # equivalente a na.rm = TRUE
```

12.466666666666669

Em R

NA corresponde a missing. A diferença é que missing é um tipo próprio, e o compilador sabe se uma coluna pode ou não contê-lo — `Vector{Float64}` nunca contém missing, `Vector{Union{Missing,Float64}}` pode. Isso elimina uma classe inteira de bugs silenciosos.

i Sua vez 4.5 — Quanto está faltando?

A cobertura de dossel não foi medida em todas as lagoas:

```
dossel = [70.0, 42.6, missing, 49.4, missing, 81.2]
```

```
6-element Vector{Union{Missing, Float64}}:
```

```
70.0
42.6
missing
49.4
missing
81.2
```

Escreva `resumo_faltantes(x)`, que retorna uma tupla com dois números: quantos valores estão faltando e a média dos demais. Para `dossel`, o resultado é (2, 60.8).

Dica

`count(f, x)` conta os elementos para os quais `f` é `true`, e `ismissing` é uma função desse tipo. A média está no bloco logo acima. Retorne os dois com `return (n, m)`.

Solução

```
resumo_faltantes(x) = (count(ismissing, x),
    ↪ mean(skipmissing(x)))

resumo_faltantes(dossel)
```

```
(2, 60.8)
```

```
checar("4.5", resumo_faltantes)
```

✓ Correto!

4.6 Encadeamento

```
# o pipe nativo |> encadeia funções de um argumento
comunidade |> shannon |> exp      # número de Hill de ordem 1
```

3.63006446685595

Para pipelines com múltiplos argumentos, @chain (do Chain.jl, reexportado por TidierData.jl) cumpre o papel de |> do R. Ele é o eixo do Capítulo 5.

4.7 Estruturas de controle

```
resultado = String[]

for (i, a) in enumerate(comunidade)
    if a == 0
        push!(resultado, "espécie $i ausente")
    elseif a < 10
        push!(resultado, "espécie $i rara")
    else
        push!(resultado, "espécie $i comum")
    end
end

resultado
```

```
7-element Vector{String}:
"espécie 1 comum"
"espécie 2 comum"
"espécie 3 ausente"
"espécie 4 rara"
"espécie 5 comum"
"espécie 6 ausente"
"espécie 7 rara"
```

Loops são rápidos

Ao contrário do R, você não precisa vetorizar por performance. Escreva o loop que expressa a ideia. Isso muda como se escrevem modelos nulos e simulações — assunto que volta nos capítulos de ecologia de comunidades de uma futura edição.

i Sua vez 4.6 — Shannon, uma espécie por vez

Reescreva `shannon` como `shannon_laco`, usando um loop `for` em vez de broadcasting. O resultado tem que ser igual ao de `shannon`, inclusive para comunidades com contagens zero, como comunidade.

Dica

Comece com `H = 0.0`, percorra as contagens uma a uma, transforme cada uma em proporção e subtraia $p * \log(p)$ de `H`. Se aparecer `NaN`, digite `0 * log(0)` no REPL.

Solução

```
function shannon_laco(abundancias)
    total = sum(abundancias)
    H = 0.0
    for a in abundancias
        a == 0 && continue # espécies ausentes não
        ↪ contribuem
            p = a / total
            H -= p * log(p)
        end
    end
    return H
end

shannon_laco(comunidade), shannon(comunidade)
```

(1.289250407583511, 1.289250407583511)

```
checar("4.6", shannon_laco)
```

✓ Correto!

`a == 0 && continue` é a forma curta, em Julia, de `if a == 0 continue end`.

4.8 Convenções de nomes

| Convenção | Significado |
|--------------------------|--|
| <code>f!(x)</code> | modifica <code>x</code> no lugar (<code>sort!</code> , <code>push!</code>) |
| <code>f.(x)</code> | aplica <code>f</code> elemento a elemento |
| <code>@f x</code> | macro, não função — reescreve código antes de rodar |
| Tipo | tipos em CamelCase |
| <code>nome_funcao</code> | funções em minúsculas, geralmente sem separador |

Desafios

Estes não têm verificador nem uma única resposta certa. São sobre o que o seu código faz — e o que ele não diz.

1. `simpson([-3, 5])` retorna um número. Qual, e deveria? Altere o método de `simpson` para vetores de modo que contagens negativas gerem um erro informativo com `throw(ArgumentError("..."))`. O método para matrizes herda a correção?
2. As comunidades `[10, 4, 3, 3]` e `[8, 7, 4, 1]` têm a mesma riqueza e o mesmo número de indivíduos. Calcule Shannon e Simpson para as duas. Os dois índices concordam sobre qual comunidade é mais diversa? O que isso implica para um artigo que reporta “diversidade” como um único número?
3. Carregue `data/ponds.csv` como em Seção 3.16 e rode `resumo_faltantes` em `ponds.canopy`. A contagem bate com o que `describe(ponds)` mostra?

Parte II

Manipulação e exploração de dados

5 Manipulação de dados com Tidier

No livro original, este é o capítulo do tidyverse. Julia tem uma tradução notavelmente fiel: a família **Tidier**, uma reimplementação em Julia puro da mini-linguagem do tidyverse sobre DataFrames.jl.

```
using TidierData
using DataFrames
using Statistics
```

5.1 Os dados

Usaremos ao longo do livro um levantamento **simulado** de anuros em 180 lagoas distribuídas por três regiões. Os dados são simulados de propósito: como o processo gerador é conhecido, cada modelo ajustado nos capítulos Capítulo 8 e Capítulo 9 pode ser confrontado com a verdade. O script gerador está em scripts/make_data.jl.

```
using CSV

ponds = CSV.read(joinpath("../", "data", "ponds.csv"), DataFrame)
first(ponds, 5)
```

| | pond_id | region | hydro | area_m2 | canopy | temp_c | richness | focal | body_size |
|---|---------|----------|-----------|---------|----------|---------|----------|-------|-----------|
| | String7 | String15 | String15 | Float64 | Float64? | Float64 | Int64 | Int64 | Float64 |
| 1 | P001 | Atlantic | permanent | 174.1 | 70.0 | 22.4 | 8 | 0 | 34.55 |
| 2 | P002 | Amazon | permanent | 161.6 | 42.6 | 23.9 | 12 | 1 | 30.36 |
| 3 | P003 | Cerrado | temporary | 233.9 | 57.5 | 25.2 | 6 | 1 | 34.44 |
| 4 | P004 | Amazon | temporary | 174.5 | 49.4 | 25.6 | 13 | 1 | 39.3 |
| 5 | P005 | Cerrado | permanent | 314.4 | 30.0 | 27.6 | 6 | 0 | 43.65 |

```
describe(ponds, :eltype, :nmissing, :mean, :min, :max)
```

| | variable | eltype | nmissing | mean | min | max |
|---|-----------|-------------------------|----------|----------|-----------|-----------|
| | Symbol | Type | Int64 | Union... | Any | Any |
| 1 | pond_id | String7 | 0 | | P001 | P180 |
| 2 | region | String15 | 0 | | Amazon | Cerrado |
| 3 | hydro | String15 | 0 | | permanent | temporary |
| 4 | area_m2 | Float64 | 0 | 240.215 | 10.0 | 1981.8 |
| 5 | canopy | Union{Missing, Float64} | 7 | 49.1422 | 6.4 | 95.8 |
| 6 | temp_c | Float64 | 0 | 24.4406 | 17.0 | 33.9 |
| 7 | richness | Int64 | 0 | 7.13333 | 0 | 31 |
| 8 | focal | Int64 | 0 | 0.45 | 0 | 1 |
| 9 | body_size | Float64 | 0 | 35.6777 | 26.91 | 44.26 |

| Coluna | Significado |
|-----------|-------------------------------------|
| pond_id | identificador da lagoa |
| region | região (Atlantic, Cerrado, Amazon) |
| hydro | hidroperíodo (permanent, temporary) |
| area_m2 | área da lagoa (m²) |
| canopy | cobertura de dossel (%) |
| temp_c | temperatura da água (°C) |
| richness | riqueza de espécies (contagem) |
| focal | ocorrência da espécie-foco (0/1) |
| body_size | tamanho corporal médio (mm) |

i Exercícios deste capítulo

Os quadros *Sua vez* funcionam como no Capítulo 4. Aqui as respostas são tabelas e números calculados a partir dos dados das lagoas; por isso, comece cada sessão carregando o verificador, os pacotes e os dados, com a pasta do livro aberta no VS Code:

```
include("exercises/checar.jl")
using TidierData, DataFrames, Statistics, CSV
ponds = CSV.read(joinpath("data", "ponds.csv"), DataFrame)
```

i Nota

Os nomes das colunas estão em inglês nas duas versões do livro, de modo que o código seja literalmente idêntico em português e em inglês.

5.2 @chain: o pipe

@chain é o eixo de tudo o que vem a seguir. Ele passa o resultado de cada linha como primeiro argumento da próxima — exatamente o papel de |> (ou %>%) no R.

```
@chain ponds begin
  @filter(region == "Amazon")
  @select(pond_id, area_m2, canopy, richness)
  @arrange(desc(richness))
  first(5)
end
```

| | pond_id | area_m2 | canopy | richness |
|---|---------|---------|----------|----------|
| | String7 | Float64 | Float64? | Int64 |
| 1 | P065 | 1048.6 | 18.0 | 31 |
| 2 | P044 | 568.1 | 31.7 | 24 |
| 3 | P067 | 411.4 | 30.9 | 21 |
| 4 | P085 | 204.6 | 58.2 | 19 |
| 5 | P073 | 140.5 | 17.7 | 18 |

Em R

```
ponds %>%
  filter(region == "Amazon") %>%
  select(pond_id, area_m2, canopy, richness) %>%
  arrange(desc(richness)) %>%
  head(5)
```

A correspondência é linha a linha. A única diferença sintática é o @ e o begin ... end.

i Sua vez 5.1 — As maiores lagoas do Cerrado

Escreva um @chain que retorne as três maiores lagoas (por area_m2) do Cerrado, da maior para a menor. Guarde o resultado em top3 e verifique com checar("5.1", top3).

Dica

Três passos, um por linha: fique com as linhas do Cerrado, ordene pela área da maior para a menor e fique com as três primeiras. O bloco acima faz algo bem parecido.

Solução

```
top3 = @chain ponds begin
  @filter(region == "Cerrado")
  @arrange(desc(area_m2))
  first(3)
end
```

| | pond_id | region | hydro | area_m2 | canopy | temp_c | richness | focal | body_size |
|---|---------|----------|-----------|---------|----------|---------|----------|-------|-----------|
| | String7 | String15 | String15 | Float64 | Float64? | Float64 | Int64 | Int64 | Float64 |
| 1 | P130 | Cerrado | temporary | 1981.8 | 84.9 | 26.0 | 9 | 0 | 38.53 |
| 2 | P034 | Cerrado | temporary | 1126.3 | missing | 22.9 | 6 | 0 | 33.53 |
| 3 | P143 | Cerrado | temporary | 932.7 | 39.8 | 25.1 | 11 | 0 | 34.15 |

```
checar("5.1", top3)
```

✓ Correto!

5.3 Os cinco verbos

5.3.1 @filter — selecionar linhas

```
@chain ponds begin
  @filter(richness > 8 && hydro == "permanent")
  nrow
end
```

39

Atenção: dentro de @filter, use && e ||, não & e | como no dplyr.

i Sua vez 5.2 — Várias condições ao mesmo tempo

Quantas lagoas temporárias têm cobertura de dossel acima de 60% e riqueza abaixo da mediana geral de riqueza? Verifique o número com `checar("5.2", n)`.

Dica

Calcule a mediana antes, fora da cadeia: `med = median(ponds.richness)`. Dentro de uma macro do Tidier, um nome solto significa uma coluna; para usar a variável `med`, escreva `!!med`.

Se aparecer `TypeError: non-boolean (Missing) used in boolean context`, algumas lagoas não têm valor de dossel, e `missing > 60` não é nem verdadeiro nem falso. Remova essas linhas antes com `@drop_missing(canopy)` (Seção 5.4).

Solução

```
med = median(ponds.richness)

n = @chain ponds begin
  @drop_missing(canopy)
  @filter(hydro == "temporary" && canopy > 60 &&
    ↪ richness < !!med)
  nrow
end
```

18

```
checar("5.2", n)
```

✓ Correto!

Descartar as lagoas sem dossel é a decisão certa aqui: não dá para saber se o dossel delas passa de 60%. Já a mediana é calculada sobre as 180 lagoas, antes de descartar qualquer coisa.

5.3.2 @select — selecionar colunas

```
@chain ponds begin
  @select(pond_id, region, richness)
  first(3)
end
```

| | pond_id | region | richness |
|---|---------|----------|----------|
| | String7 | String15 | Int64 |
| 1 | P001 | Atlantic | 8 |
| 2 | P002 | Amazon | 12 |
| 3 | P003 | Cerrado | 6 |

```
# exclusão e intervalos funcionam como no dplyr
@chain ponds begin
  @select(-pond_id, -focal)
  names
end
```

9-element Vector{String}:

```
"region"
"hydro"
"area_m2"
"canopy"
"temp_c"
"richness"
"focal"
"body_size"
"pond_id"
```

5.3.3 @mutate — criar colunas

```
ponds_t = @chain ponds begin
  @mutate(
    log_area = log(area_m2),
    canopy_std = (canopy - mean(skipmissing(canopy))) /
    ↪ std(skipmissing(canopy)),
    open = canopy < 40,
  )
end

@chain ponds_t begin
  @select(pond_id, area_m2, log_area, canopy, canopy_std, open)
  first(5)
end
```

| | pond_id | area_m2 | log_area | canopy | canopy_std | open |
|---|---------|---------|----------|----------|------------|-------|
| | String7 | Float64 | Float64 | Float64? | Float64? | Bool? |
| 1 | P001 | 174.1 | 5.15963 | 70.0 | 0.946503 | 0 |
| 2 | P002 | 161.6 | 5.08512 | 42.6 | -0.296877 | 0 |
| 3 | P003 | 233.9 | 5.45489 | 57.5 | 0.379268 | 0 |
| 4 | P004 | 174.5 | 5.16192 | 49.4 | 0.0116988 | 0 |
| 5 | P005 | 314.4 | 5.75067 | 30.0 | -0.868651 | 1 |

i Sua vez 5.3 — Riqueza por 100 m²

Lagoas maiores tendem a ter mais espécies. Adicione uma coluna `rich_per_100m2` com o número de espécies por 100 m² de lagoa, mantendo todas as linhas e colunas de `ponds`. Guarde o resultado em `ponds_d` e verifique com `checar("5.3", ponds_d)`.

Dica

`richness / area_m2` é espécies por m²; multiplique por 100.

Solução

```
ponds_d = @chain ponds begin
    @mutate(rich_per_100m2 = richness / area_m2 * 100)
end

checar("5.3", ponds_d)
```

✓ Correto!

5.3.4 @group_by + @summarize — agregar

```
@chain ponds begin
    @group_by(region, hydro)
    @summarize(
        n          = n(),
        mean_rich  = mean(richness),
        sd_rich    = std(richness),
        prop_focal = mean(focal),
    )
    @arrange(region, hydro)
end
```

GroupedDataFrame with 3 groups based on key: region

First Group (2 rows): region = "Amazon"

| | region | hydro | n | mean_rich | sd_rich | prop_focal |
|---|----------|-----------|-------|-----------|---------|------------|
| | String15 | String15 | Int64 | Float64 | Float64 | Float64 |
| 1 | Amazon | permanent | 26 | 11.2692 | 5.99038 | 0.538462 |
| 2 | Amazon | temporary | 22 | 10.9091 | 5.95946 | 0.318182 |

...

Last Group (2 rows): region = "Cerrado"

| | region | hydro | n | mean_rich | sd_rich | prop_focal |
|---|----------|-----------|-------|-----------|---------|------------|
| | String15 | String15 | Int64 | Float64 | Float64 | Float64 |
| 1 | Cerrado | permanent | 31 | 4.77419 | 2.90643 | 0.580645 |
| 2 | Cerrado | temporary | 35 | 4.8 | 2.95854 | 0.4 |

i Sua vez 5.4 — Onde está a espécie-foco?

Calcule, para cada região, a proporção de lagoas onde a espécie-foco ocorre, em uma coluna chamada `prop_focal`, ordenada da maior para a menor. Verifique com `checar("5.4", por_regiao)`.

Dica

`focal` vale 0 ou 1, então a média é a proporção de lagoas onde ela vale 1. Ordenar da maior para a menor é `@arrange(desc(...))`.

Solução

```
por_regiao = @chain ponds begin
  @group_by(region)
  @summarize(prop_focal = mean(focal))
  @arrange(desc(prop_focal))
end

checar("5.4", por_regiao)
```

✓ Correto!

5.3.5 @arrange — ordenar

```
@chain ponds begin
  @arrange(region, desc(richness))
  @select(region, pond_id, richness)
  first(6)
end
```

| | region | pond_id | richness |
|---|----------|---------|----------|
| | String15 | String7 | Int64 |
| 1 | Amazon | P065 | 31 |
| 2 | Amazon | P044 | 24 |
| 3 | Amazon | P067 | 21 |
| 4 | Amazon | P085 | 19 |
| 5 | Amazon | P073 | 18 |
| 6 | Amazon | P164 | 18 |

5.4 Dados faltantes

A coluna canopy tem valores ausentes propositalmente.

```
sum(ismissing.(ponds.canopy))
```

7

```
@chain ponds begin
  @drop_missing(canopy)
  nrow
end
```

173

```
# agregação ignorando ausentes
@chain ponds begin
  @group_by(region)
  @summarize(mean_canopy = mean(skipmissing(canopy)))
end
```

| | region | mean_canopy |
|---|----------|-------------|
| | String15 | Float64 |
| 1 | Atlantic | 51.7125 |
| 2 | Amazon | 45.313 |
| 3 | Cerrado | 49.327 |

⚠ Aviso

`mean(canopy)` devolve `missing` se qualquer valor for ausente — o mesmo comportamento de `mean()` em R sem `na.rm = TRUE`. Seja explícito com `skipmissing`.

i Sua vez 5.5 — Onde estão as lacunas?

Antes de descartar valores faltantes, vale saber onde eles estão: se estiverem concentrados em uma região, descartá-los muda essa região mais do que as outras. Conte, por região, quantas lagoas não têm valor de dossel, em uma coluna chamada `n_missing`. Verifique com `checar("5.5", lacunas)`.

Dica

`ismissing.(canopy)` dá `true` para cada valor faltante, e `sum` conta os `true`. Os dois funcionam dentro de `@summarize`.

Solução

```
lacunas = @chain ponds begin
  @group_by(region)
  @summarize(n_missing = sum(ismissing.(canopy)))
end

checar("5.5", lacunas)
```

✓ Correto!

Sete valores faltantes espalhados pelas três regiões: nenhuma região é muito mais afetada que as outras, então descartá-los dificilmente distorce comparações entre regiões.

5.5 Formato longo e largo

```
longo = @chain ponds begin
  @select(pond_id, region, area_m2, canopy, temp_c)
  @pivot_longer(area_m2:temp_c, names_to = "variable", values_to =
    ↪ "value")
end

first(longo, 6)
```

| | pond_id | region | variable | value |
|---|---------|----------|----------|----------|
| | String7 | String15 | String | Float64? |
| 1 | P001 | Atlantic | area_m2 | 174.1 |
| 2 | P002 | Amazon | area_m2 | 161.6 |
| 3 | P003 | Cerrado | area_m2 | 233.9 |
| 4 | P004 | Amazon | area_m2 | 174.5 |
| 5 | P005 | Cerrado | area_m2 | 314.4 |
| 6 | P006 | Cerrado | area_m2 | 620.2 |

```
@chain longo begin
  @pivot_wider(names_from = variable, values_from = value)
  first(3)
end
```

| | pond_id | region | area_m2 | canopy | temp_c |
|---|---------|----------|----------|----------|----------|
| | String7 | String15 | Float64? | Float64? | Float64? |
| 1 | P001 | Atlantic | 174.1 | 70.0 | 22.4 |
| 2 | P002 | Amazon | 161.6 | 42.6 | 23.9 |
| 3 | P003 | Cerrado | 233.9 | 57.5 | 25.2 |

5.6 Junções

```
# uma tabela auxiliar com informação em nível de região
regioes = DataFrame(
    region      = ["Atlantic", "Cerrado", "Amazon"],
    biome_type  = ["forest", "savanna", "forest"],
    mean_rain   = [1400, 1500, 2300],
)

@chain ponds begin
    @left_join(regioes, region)
    @select(pond_id, region, biome_type, mean_rain, richness)
    first(5)
end
```

| | pond_id | region | biome_type | mean_rain | richness |
|---|---------|----------|------------|-----------|----------|
| | String7 | String15 | String? | Int64? | Int64 |
| 1 | P001 | Atlantic | forest | 1400 | 8 |
| 2 | P002 | Amazon | forest | 2300 | 12 |
| 3 | P003 | Cerrado | savanna | 1500 | 6 |
| 4 | P004 | Amazon | forest | 2300 | 13 |
| 5 | P005 | Cerrado | savanna | 1500 | 6 |

i Sua vez 5.6 — Floresta contra savana

Usando a tabela regioes acima, calcule a riqueza média das lagoas de floresta e de savana, em uma coluna chamada mean_rich. Verifique com `checar("5.6", por_bioma)`.

Dica

Primeiro faça a junção, para que cada lagoa ganhe seu biome_type; depois agrupe por biome_type e resuma.

Solução

```
por_bioma = @chain ponds begin
    @left_join(regioes, region)
    @group_by(biome_type)
    @summarize(mean_rich = mean(richness))
end

checar("5.6", por_bioma)
```

✓ Correto!

5.7 DataFrames.jl por baixo

Tidier é uma camada de conveniência. Vale saber a forma nativa, que é o que você vai encontrar na documentação da maior parte do ecossistema:

```
combine(  
  groupby(ponds, :region),  
  :richness => mean => :mean_rich,  
  :richness => length => :n,  
)
```

| | region | mean_rich | n |
|---|----------|-----------|-------|
| | String15 | Float64 | Int64 |
| 1 | Atlantic | 6.59091 | 66 |
| 2 | Amazon | 11.1042 | 48 |
| 3 | Cerrado | 4.78788 | 66 |

O mesmo resultado do `@group_by/@summarize` acima. Use Tidier para escrever, e reconheça a forma nativa para ler.

i Sua vez 5.7 — O mesmo, na forma nativa

Reescreva sua resposta ao 5.4 com `groupby`, `combine` e `sort` do `DataFrames.jl`, sem as macros do Tidier. Verifique com `checar("5.7", por_regiao_nativo)`.

Dica

`sort(df, :coluna, rev = true)` ordena da maior para a menor.

Solução

```
por_regiao_nativo = sort(  
  combine(groupby(ponds, :region), :focal => mean =>  
    ↪ :prop_focal),  
  :prop_focal, rev = true)  
  
checar("5.7", por_regiao_nativo)
```

✓ Correto!

Desafios

Estes não têm verificador. São sobre o que os números dizem — e o que não dizem.

1. No 5.4 o Cerrado fica em primeiro, mas as três proporções vão só de 0,42 a 0,48, cada uma calculada sobre 48 a 66 lagoas. Abra `scripts/make_data.jl`: a região afeta a espécie-foco na simulação? O que isso diz sobre ler um ranking em uma tabela resumo?

2. No 5.6 as lagoas de floresta são mais ricas que as de savana. Mas “floresta” aqui significa Atlantic mais Amazon. Calcule a riqueza média por região e decida se “floresta” é um agrupamento útil para essa pergunta.
3. Use `@pivot_longer` para colocar `area_m2`, `canopy` e `temp_c` em uma única coluna e depois calcule a média de cada variável por região em um único pipeline. Como você lida com os valores faltantes de dossel?

6 Visualização de dados

Julia tem três caminhos razoáveis para gráficos, todos assentados sobre [Makie.jl](#) (Danisch e Krumbiegel 2021):

| Caminho | Para quem |
|----------------------|---|
| TidierPlots.jl | quem vem do ggplot2 e quer a mesma gramática |
| AlgebraOfGraphics.jl | quem quer gramática de gráficos idiomática em Julia |
| Makie.jl puro | controle total sobre a figura |

Este capítulo mostra os três sobre o mesmo conjunto de dados, porque a escolha certa depende da tarefa — e porque saber traduzir entre eles evita ficar preso.

```
using TidierData
using CairoMakie
using AlgebraOfGraphics
using CSV, DataFrames

CairoMakie.activate!(type = "png")

ponds = CSV.read(joinpath("../", "data", "ponds.csv"), DataFrame)
ponds_c = @chain ponds begin
    @drop_missing(canopy)
    @mutate(log_area = log(area_m2))
end

nrow(ponds_c)
```

173

i Exercícios deste capítulo

Uma figura não pode ser verificada automaticamente; por isso, na maioria dos quadros *Sua vez*, você compara o seu gráfico com o da solução. Comece cada sessão carregando os pacotes e os dados, com a pasta do livro aberta no VS Code:

```
include("exercises/checar.jl")
using TidierData, TidierPlots, CairoMakie, AlgebraOfGraphics, CSV,
    DataFrames, Statistics
ponds = CSV.read(joinpath("data", "ponds.csv"), DataFrame)
ponds_c = @chain ponds begin
    @drop_missing(canopy)
    @mutate(log_area = log(area_m2))
end
```

6.1 TidierPlots.jl: a gramática do ggplot

```
using TidierPlots

ggplot(ponds_c, @aes(x = canopy, y = richness, color = region)) +
    geom_point(alpha = 0.7) +
    labs(x = "Cobertura de dossel (%)", y = "Riqueza de espécies",
         title = "Riqueza declina com o sombreamento")
```

Figura 6.1: Riqueza em função da cobertura de dossel, por região.

Em R

```
ggplot(ponds_c, aes(x = canopy, y = richness, colour = region)) +
    geom_point(alpha = 0.7) +
    labs(x = "Canopy cover (%)", y = "Species richness")
```

A tradução é quase caractere a caractere; a diferença é o @ em @aes.

i Sua vez 6.1 — Riqueza e temperatura

Refaça a Figura 6.1 com a temperatura da água (temp_c) no eixo x e os pontos coloridos pelo hidroperíodo (hydro). Dê nome aos dois eixos. A riqueza muda com a temperatura tão claramente quanto com o dossel?

Este não tem verificador: compare a sua figura com a da solução.

Dica

Só mudam o mapeamento em @aes e os rótulos.

Solução

```
ggplot(ponds_c, @aes(x = temp_c, y = richness, color =
  ↪ hydro)) +
  geom_point(alpha = 0.7) +
  labs(x = "Temperatura da água (°C)", y = "Riqueza de
    ↪ espécies")
```

A nuvem é bem mais achatada que a do dossel. A simulação inclui um efeito da temperatura (scripts/make_data.jl), mas pequeno: cerca de 3% a mais de espécies por grau, invisível diante da variação entre lagoas.

6.2 AlgebraOfGraphics.jl: gramática algébrica

AoG trata gráficos como uma álgebra: * combina camadas, + as sobrepõe.

```
plt = AlgebraOfGraphics.data(ponds_c) *
  mapping(:canopy => "Cobertura de dossel (%)",
    :richness => "Riqueza de espécies",
    color = :region => "Região") *
  (visual(Scatter, alpha = 0.6) + AlgebraOfGraphics.linear())

draw(plt; figure = (size = (700, 450),))
```

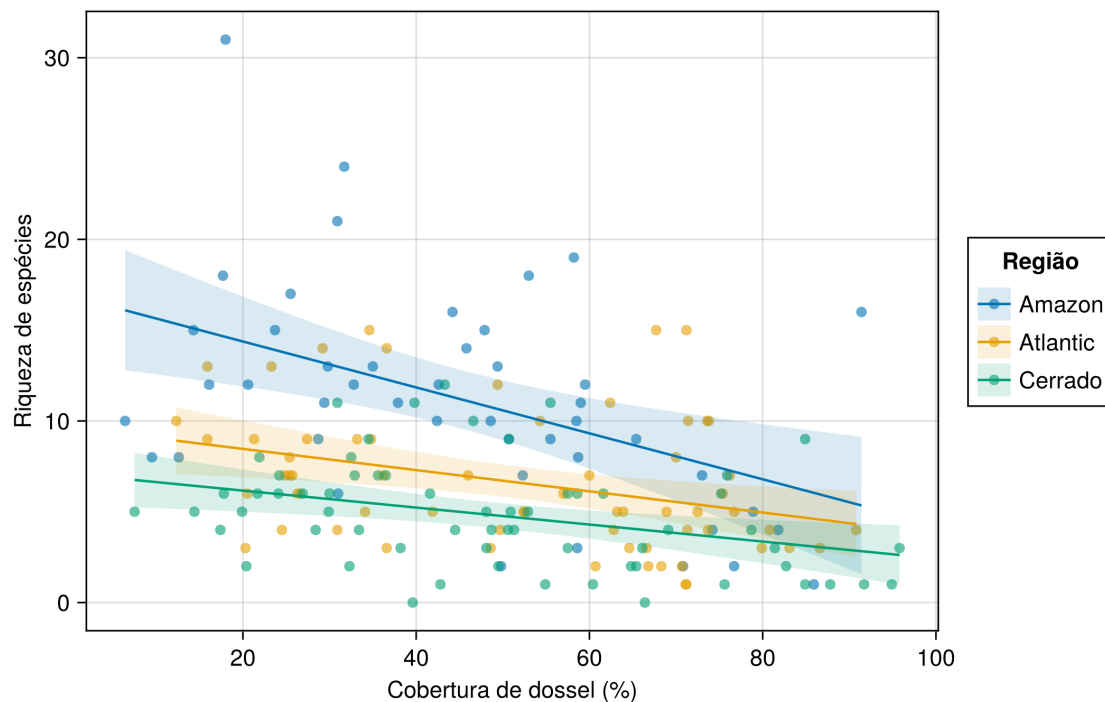


Figura 6.2: O mesmo gráfico em `AlgebraOfGraphics`, com ajuste linear por região.

i Nota

`AlgebraOfGraphics.data` e `AlgebraOfGraphics.linear` são qualificados porque os nomes `data` e `linear` colidem com outros pacotes carregados. Em um script que use só `AoG`, `data(...)` e `linear()` bastam.

i Sua vez 6.2 — Tamanho corporal, com retas ajustadas

Com `AlgebraOfGraphics`, faça o gráfico de `body_size` contra `temp_c`, colorido por `hydro`, com um ajuste linear para cada hidroperíodo. Lagoas permanentes e temporárias diferem?

Este não tem verificador: compare a sua figura com a da solução.

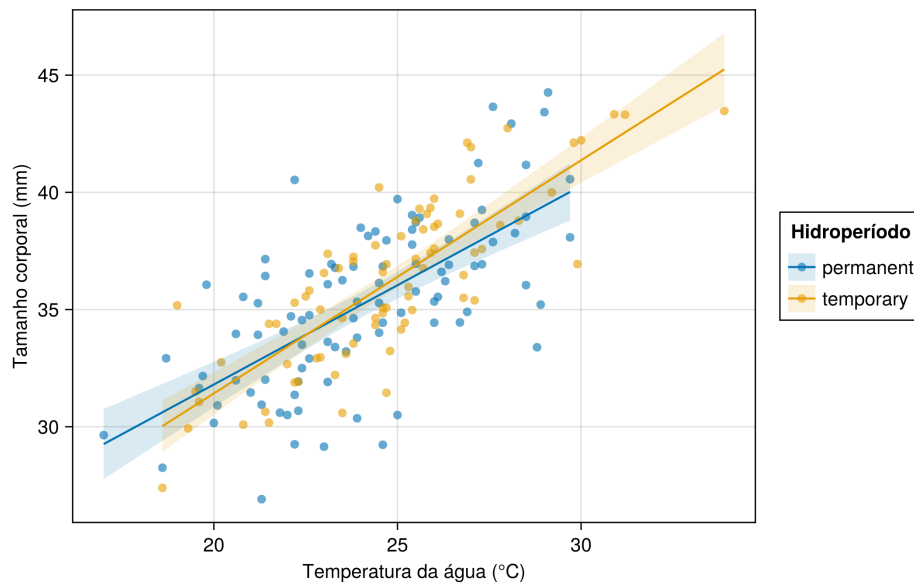
Dica

Parta do bloco acima e mude o mapping. As retas ajustadas seguem automaticamente o agrupamento por cor.

Solução

```
plt = AlgebraOfGraphics.data(ponds_c) *
  mapping(:temp_c => "Temperatura da água (°C)",
    :body_size => "Tamanho corporal (mm)",
    color = :hydro => "Hidroperíodo") *
  (visual(Scatter, alpha = 0.6) +
    ↪ AlgebraOfGraphics.linear())

draw(plt; figure = (size = (700, 450),))
```



As duas retas quase coincidem: na simulação, o tamanho corporal depende da temperatura e do dossel, mas não do hidroperíodo. Testamos isso formalmente no Capítulo 8.

6.3 Facetas

```
plt = AlgebraOfGraphics.data(ponds_c) *
  mapping(:log_area => "log(área)",
    :richness => "Riqueza",
    color = :region,
    layout = :hydro) *
  (visual(Scatter, alpha = 0.6) + AlgebraOfGraphics.linear())

draw(plt; figure = (size = (800, 400),))
```

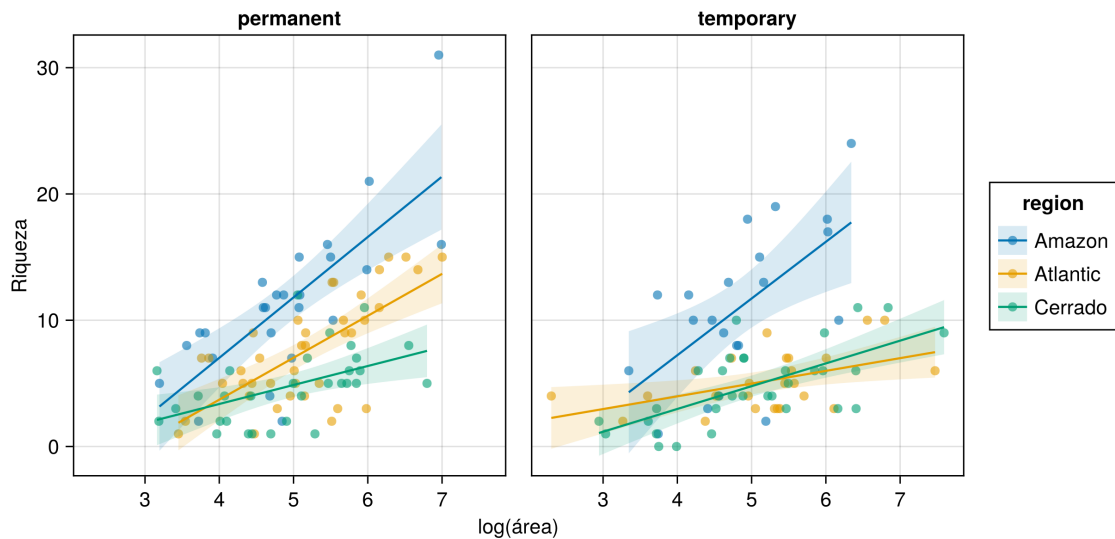


Figura 6.3: Relação riqueza-área facetada por hidroperíodo.

i Sua vez 6.3 — Facetas na gramática do ggplot

Traduza a Figura 6.3 para TidierPlots.jl, usando `facet_wrap` para separar os painéis por hidroperíodo. Deixe de fora as retas ajustadas (veja o aviso na solução). Este não tem verificador: compare a sua figura com a da solução.

Dica

`facet_wrap(facets = :hydro)` é adicionado com `+`, como qualquer outra camada.

Solução

```
ggplot(ponds_c, @aes(x = log_area, y = richness, color =
  ↪ region)) +
  geom_point(alpha = 0.6) +
  facet_wrap(facets = :hydro) +
  labs(x = "log(área)", y = "Riqueza")
```

Um limite do TidierPlots

No TidierPlots.jl 1.0.0 (conferido em setembro de 2026), adicionar `geom_smooth(method = "lm")` a um gráfico com facetas falha com column name `:facet` not found. `geom_smooth` funciona sem facetas, e facetas funcionam sem `geom_smooth`, mas não os dois juntos. Quando a gramática do ggplot chega a um limite como este, o AlgebraOfGraphics (Figura 6.3) é a saída.

6.4 Distribuições

```
plt = AlgebraOfGraphics.data(ponds_c) *  
  mapping(:region => "Região",  
    :richness => "Riqueza",  
    color = :hydro => "Hidroperíodo",  
    dodge = :hydro) *  
  visual(BoxPlot)  
  
draw(plt; figure = (size = (700, 420),))
```

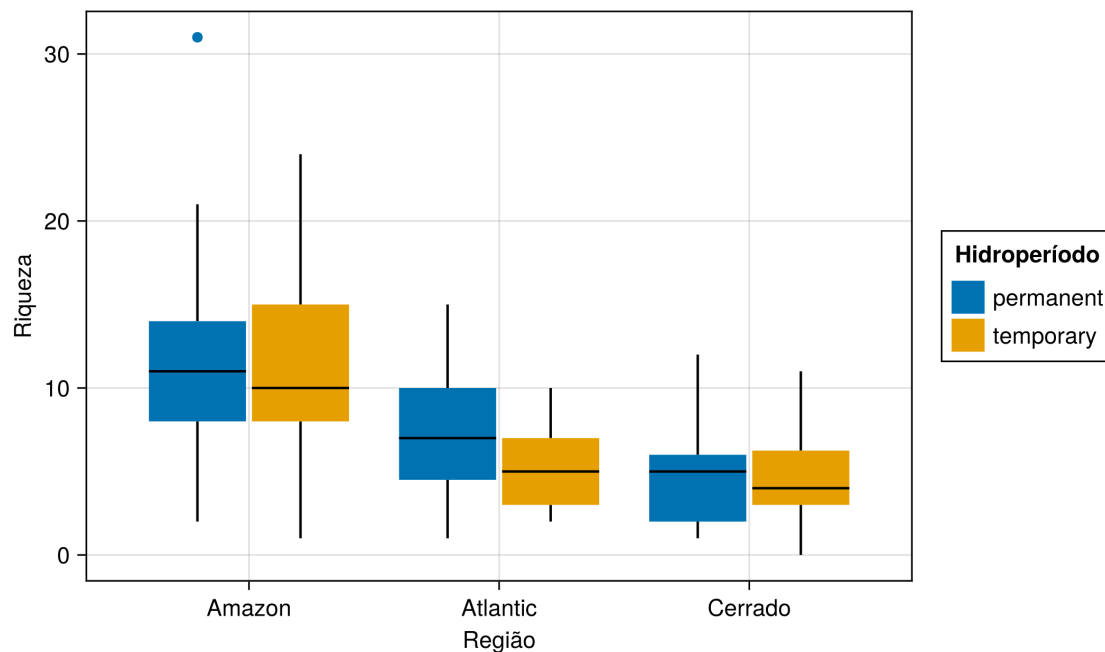


Figura 6.4: Distribuição da riqueza por região e hidroperíodo.

i Sua vez 6.4 — A forma de uma distribuição

Boxplots escondem a forma da distribuição. Mostre a distribuição de `body_size` em cada região como violinos (Violin), com uma cor por região. Este não tem verificador: compare a sua figura com a da solução.

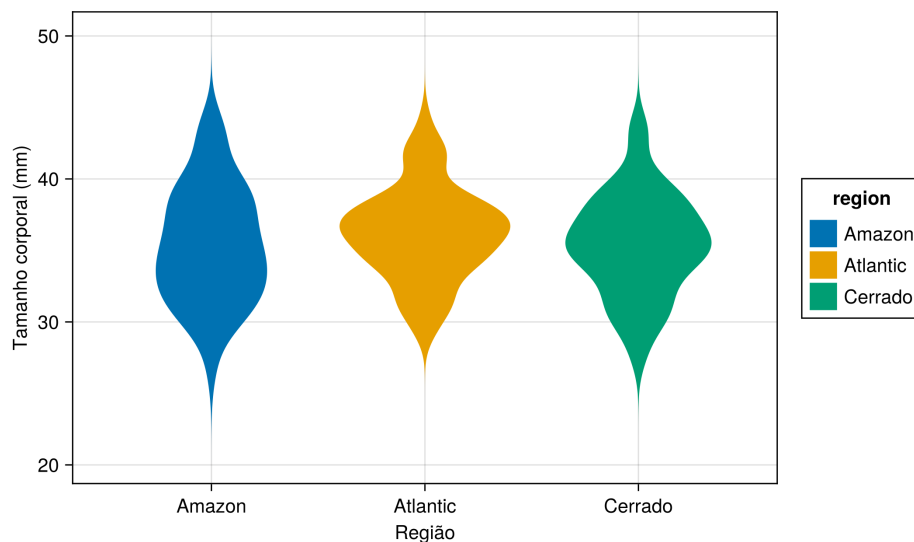
Dica

Troque `BoxPlot` por `Violin` e mapeie `region` tanto no eixo x quanto em `color`.

Solução

```
plt = AlgebraOfGraphics.data(ponds_c) *
  mapping(:region => "Região",
    :body_size => "Tamanho corporal (mm)",
    color = :region) *
  visual(Violin)

draw(plt; figure = (size = (700, 420),))
```



6.5 Makie.jl puro: controle total

Quando a figura precisa de um layout que a gramática não expressa, desça ao Makie.

```
fig = Figure(size = (860, 380))

ax1 = Axis(fig[1, 1], xlabel = "Cobertura de dossel (%)", ylabel =
  ↳ "Riqueza",
  title = "(a) Dossel")
scatter!(ax1, ponds_c.canopy, ponds_c.richness, color = (:steelblue,
  ↳ 0.6))

ax2 = Axis(fig[1, 2], xlabel = "log(área)", ylabel = "Riqueza",
  title = "(b) Área")
scatter!(ax2, ponds_c.log_area, ponds_c.richness, color = (:darkorange,
  ↳ 0.6))

ax3 = Axis(fig[1, 3], xlabel = "Riqueza", ylabel = "Frequência",
  title = "(c) Distribuição")
```

```
hist!(ax3, ponds_c.richness, bins = 12, color = (:seagreen, 0.7))

fig
```

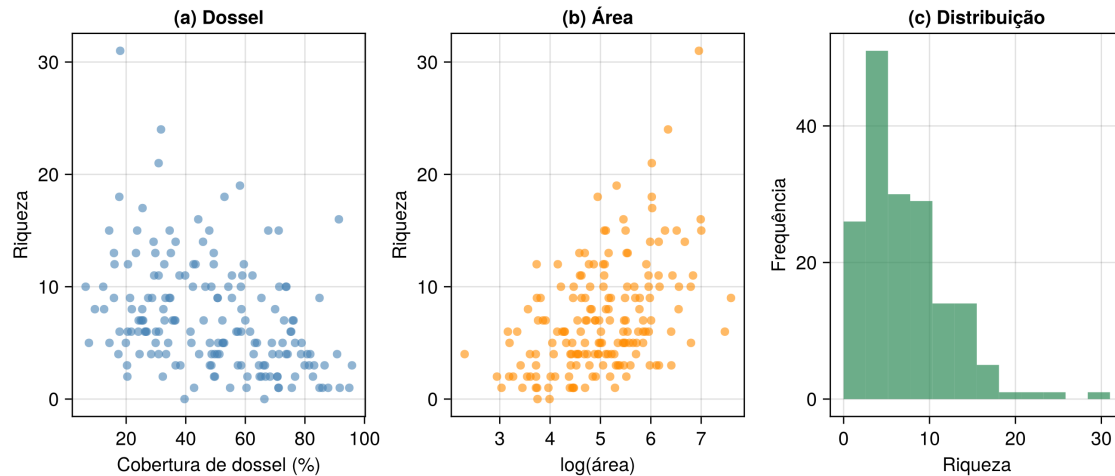


Figura 6.5: Figura composta montada manualmente com Makie.

O sistema de layout do Makie é baseado em grade (`fig[linha, coluna]`), cobrindo o papel de patchwork e cowplot no R sem pacote adicional.

i Sua vez 6.5 — Dois painéis à mão

Monte uma figura com `size = (800, 350)` e dois painéis: à esquerda, um boxplot da riqueza por região; à direita, um gráfico de barras com a proporção de lagoas com a espécie-foco em cada região (os números do 5.4). Guarde-a em `fig_regioes`. Este não tem verificador: compare a sua figura com a da solução.

Dica

`boxplot!` e `barplot!` do Makie querem números no eixo x. Dê um número a cada região com `[findfirst(==(r), regs) for r in ponds.region]`, onde `regs = ["Amazon", "Atlantic", "Cerrado"]`, e recoloque os nomes com `xticks = (1:3, regs)` em cada Axis.

Solução

```

regs = ["Amazon", "Atlantic", "Cerrado"]
idx = [findfirst(==(r), regs) for r in ponds.region]
props = [mean(ponds.focal[ponds.region .== r]) for r in
  ↪ regs]

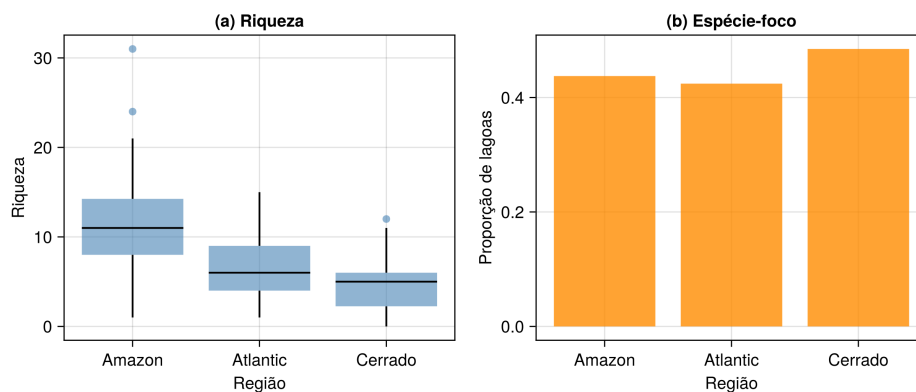
fig_regioes = Figure(size = (800, 350))

ax1 = Axis(fig_regioes[1, 1], xticks = (1:3, regs),
  xlabel = "Região", ylabel = "Riqueza", title =
  ↪ "(a) Riqueza")
boxplot!(ax1, idx, ponds.richness, color = (:steelblue,
  ↪ 0.6))

ax2 = Axis(fig_regioes[1, 2], xticks = (1:3, regs),
  xlabel = "Região", ylabel = "Proporção de
  ↪ lagoas", title = "(b) Espécie-foco")
barplot!(ax2, 1:3, props, color = (:darkorange, 0.8))

fig_regioes

```



Os dois painéis contam histórias diferentes: a riqueza varia muito entre regiões; a ocorrência da espécie-foco, quase nada.

6.6 Salvando figuras

```

save("figura.png", fig; px_per_unit = 3) # raster em alta resolução
save("figura.pdf", fig)                  # vetorial, para publicação
save("figura.svg", fig)

```

i Sua vez 6.6 — Uma figura para publicação

Salve a `fig_regioes` do 6.5 como `focal-by-region.png`, com três pixels por unidade, na pasta do livro. Depois verifique o arquivo: `checar("6.6", "focal-by-region.png")`.

Dica

O bloco acima mostra o argumento que define a resolução.

Solução

```
save("focal-by-region.png", fig_regioes; px_per_unit = 3)
checar("6.6", "focal-by-region.png")
```

Uma figura de 800×350 com três pixels por unidade gera uma imagem de 2400×1050 px — suficiente para uma figura de duas colunas em um periódico. Para saída vetorial, salve em `.pdf`.

💡 Escolhendo o backend

CairoMakie gera saída estática de qualidade de publicação (PNG, PDF, SVG) e é o que este livro usa. GLMakie abre janelas interativas com aceleração de GPU — útil para explorar dados, inadequado para gerar um livro.

Desafios

1. No 6.1 você julgou o efeito da temperatura a olho. Adicione um ajuste linear por hidroperíodo com `geom_smooth(method = "lm")`. A reta muda a sua leitura do gráfico? O que você precisaria saber antes de confiar nela?
2. O painel (b) do 6.5 mostra proporções sem incerteza. Adicione a cada barra uma barra de erro de ± 2 erros-padrão, com $EP = \sqrt{p(1-p)/n}$ e `errorbars!`. As regiões ainda parecem diferentes?
3. Refaça a Figura 6.5 com uma cor por região em todos os painéis e uma única legenda compartilhada à direita (`Legend(fig[1, 4], ...)`).

7 Análise exploratória de dados

Antes de qualquer modelo, olhe os dados. A exploração tem quatro tarefas (Zuur et al. 2010): encontrar **erros e outliers**, entender a **estrutura** dos dados — o que conta como réplica —, ver as **distribuições** que os modelos terão de descrever e detectar **colinearidade** entre variáveis que um modelo terá dificuldade em separar. Pular essa etapa não economiza tempo: os problemas continuam lá, e aparecem depois como resultados estranhos de modelo.

As lagoas simuladas dos capítulos anteriores são limpas demais para isso. Este capítulo usa dados reais, com todos os problemas que dados reais têm.

i De onde vem este capítulo

Como o Capítulo 10 e o Capítulo 11, este capítulo é **inspirado no ecoR** (Oliveira 2026), o wiki do curso de R do Instituto de Biociências da Universidade de São Paulo, produzido e mantido por Alexandre Adalardo de Oliveira. A sequência — conferir o data frame, estatísticas descritivas, gráficos univariados, relações entre variáveis e o quarteto de Anscombe — segue o módulo de análise exploratória do ecoR. O texto, os dados, os exemplos e os exercícios daqui foram escritos do zero para Julia; os autores do ecoR não revisaram este livro. Se quiser ir além, o material original está disponível gratuitamente (CC BY-SA 4.0):

- [4. Análise Exploratória de Dados](#) — a apostila;
- [4. Tutoriais de Análise Exploratória de Dados](#) — o tutorial passo a passo;
- [exercícios de análise exploratória](#), corrigidos automaticamente no [notaR](#), em R.

i Os dados: ATLANTIC BIRD TRAITS

O **ATLANTIC BIRD TRAITS** (Rodrigues et al. 2019) reúne medidas morfológicas de aves das florestas atlânticas da América do Sul: 72 483 registros, compilados de coleções de museu, projetos de anilhamento e da literatura entre 1820 e 2018, por 179 autores. O artigo de dados afirma que não há restrições de direitos autorais nem de propriedade sobre o seu uso, e pede que seja citado quando usado em publicações ou no ensino — como aqui.

O `data/atlantic_bird_traits.csv` mantém **todos** os registros e 21 das 83 colunas originais, com os nomes e códigos originais. Nada foi limpo: encontrar os problemas é o objetivo. O `scripts/prepare_birds.jl` documenta como o subconjunto foi feito a partir do arquivo original, que está no material suplementar do artigo.

```
using TidierData
using DataFrames, CSV
using Statistics, StatsBase
using GLM, RDatasets, Distributions
using CairoMakie

CairoMakie.activate!(type = "png")
arquivo_aves = joinpath("../", "data", "atlantic_bird_traits.csv")
```

"../data/atlantic_bird_traits.csv"

i Exercícios deste capítulo

As respostas são números calculados a partir dos dados das aves. Comece cada sessão com:

```
include("exercises/checar.jl")
using TidierData, DataFrames, CSV, Statistics, StatsBase, GLM,
    ↪ Distributions
arquivo_aves = joinpath("data", "atlantic_bird_traits.csv")
```

A primeira seção mostra como ler o arquivo; os exercícios seguintes supõem que você o leu desse jeito, em `birds`, com as colunas renomeadas.

7.1 Primeiro contato: lendo o arquivo

Leia o arquivo como ele vem e olhe o tipo de cada coluna:

```
bruto = CSV.read(arquivo_aves, DataFrame)
size(bruto)
```

(72483, 21)

```
first(describe(bruto, :eltype, :nmissing), 8)
```

| | variable | eltype | nmissing |
|---|-------------------|----------|----------|
| | Symbol | DataType | Int64 |
| 1 | ID_ABT | String7 | 0 |
| 2 | Order | String31 | 0 |
| 3 | Family | String31 | 0 |
| 4 | Binomial | String31 | 0 |
| 5 | Body_mass.g. | String7 | 0 |
| 6 | Wing_length.mm. | String7 | 0 |
| 7 | Tail_length.mm. | String7 | 0 |
| 8 | Tarsus_length.mm. | String7 | 0 |

Massa corporal, comprimento da asa, comprimento da cauda: tudo lido como **texto** (String), e nenhum valor faltante. O motivo está no próprio arquivo — os valores faltantes estão escritos como as duas letras NA, que o CSV.jl não trata como faltante por padrão. Um único não número em uma coluna faz a coluna inteira virar texto. Diga ao CSV.read qual código significa faltante:

```
birds = CSV.read(arquivo_aves, DataFrame; missingstring = "NA")
first(describe(birds, :eltype, :nmissing), 8)
```

| | variable | eltype | nmissing |
|---|-------------------|-------------------------|----------|
| | Symbol | Type | Int64 |
| 1 | ID_ABT | String7 | 0 |
| 2 | Order | String31 | 0 |
| 3 | Family | String31 | 0 |
| 4 | Binomial | String31 | 0 |
| 5 | Body_mass.g. | Union{Missing, Float64} | 11664 |
| 6 | Wing_length.mm. | Union{Missing, Float64} | 51134 |
| 7 | Tail_length.mm. | Union{Missing, Float64} | 26183 |
| 8 | Tarsus_length.mm. | Union{Missing, Float64} | 50742 |

Em R

O read.csv trata "NA" como faltante por padrão, então em R esse problema passaria despercebido. Aqui ele fica visível: uma coluna numérica que chega como texto é o primeiro sinal de que o arquivo usa um código de faltante que você não declarou. Outros códigos comuns são "", "-", ".", -999 e 99999; missingstring também aceita um vetor deles.

Os nomes das colunas misturam unidades, pontos e maiúsculas, como Body_mass.g.. São incômodos de digitar, e as macros do Tidier não conseguem usá-los diretamente. Renomeie-os uma vez, no início:

```
rename!(birds,
  "ID_ABT" => :id, "Order" => :order, "Family" => :family, "Binomial"
  ↪ => :species,
  "Body_mass.g." => :mass_g, "Wing_length.mm." => :wing_mm,
  "Tail_length.mm." => :tail_mm, "Tarsus_length.mm." => :tarsus_mm,
  "Bill_length.mm." => :bill_mm, "Age" => :age, "Sex" => :sex,
  ↪ "Status" => :status,
  "Recapture" => :recapture, "Ring" => :ring,
  "Latitude_decimal_degrees" => :lat, "Longitude_decimal_degrees" =>
  ↪ :lon,
  "Altitude" => :altitude, "Year" => :year, "Main_researcher" =>
  ↪ :source,
  "Outside.range" => :outside_range, "Obs.spp" => :species_note)

names(birds)
```

21-element Vector{String}:

```
"id"
"order"
"family"
```

```
"species"  
"mass_g"  
"wing_mm"  
"tail_mm"  
"tarsus_mm"  
"bill_mm"  
"age"  
"  
"recapture"  
"ring"  
"lat"  
"lon"  
"altitude"  
"year"  
"source"  
"outside_range"  
"species_note"
```

Os nomes novos ficam em inglês, como as colunas dos dados das lagoas, para que o código seja o mesmo nas duas versões do livro.

i Sua vez 7.1 — Quantas aves foram pesadas?

Quantos registros têm massa corporal? Passe o número para checar("7.1", ...).

Dica

`count(!ismissing, v)` conta os valores não faltantes de um vetor. Isso só faz sentido depois que o arquivo foi lido com `missingstring = "NA"`.

Solução

```
n_massa = count(!ismissing, birds.mass_g)
```

60819

```
checar("7.1", n_massa)
```

✓ Correto!

60 819 de 72 483: a massa corporal é o atributo mais registrado, e ainda assim falta em um de cada seis registros.

7.2 Faltante, desconhecido e inconsistente

Colunas categóricas merecem, antes de tudo, uma tabela dos seus níveis:

```
(idade = countmap(birds.age), sexo = countmap(birds.sex))
```

```
(idade = Dict{Union{Missing, String15}, Int64}(missing => 4935, String15("Juvenile") => 87
```

Há **dois tipos de faltante** aqui. `missing` quer dizer que a informação nunca foi anotada; `"Unknown"` quer dizer que alguém olhou a ave e não conseguiu determinar. Não é a mesma coisa: um estudo de razão sexual deve contar os desconhecidos, mas não os não anotados.

Outras colunas discordam de si mesmas na grafia:

```
(recaptura = countmap(birds.recapture), fora_distribuicao =  
  ↪ countmap(birds.outside_range))
```

```
(recaptura = Dict{Union{Missing, String3}, Int64}(missing => 33682, String3("Yes") => 8556
```

"no" e "Yes": a capitalização muda entre os níveis. Aqui isso é inofensivo, porque cada nível tem uma única grafia, mas um único "yes" em algum lugar criaria um terceiro nível, que o agrupamento trataria como diferente. Padronizar custa pouco:

```
birds.recapture      = passmissing(lowercase).(birds.recapture)  
birds.outside_range = lowercase.(birds.outside_range)  
unique(birds.recapture)
```

```
3-element Vector{Union{Missing, String}}:  
 "no"  
 missing  
 "yes"
```

`passmissing(f)` aplica `f` aos valores e deixa `missing` em paz.

i Sua vez 7.2 — Com que frequência dá para saber o sexo?

Entre os registros em que o sexo foi **anotado** (não `missing`), que proporção foi sexada como "Male" ou "Female"? Passe-a para checar("7.2", ...).

Dica

Fique com os valores não faltantes com `collect(skipmissing(birds.sex))` e tire a média de `in(("Male", "Female"))`.

Solução

```
sexo_anotado = collect(skipmissing(birds.sex))  
p_sexado = mean(in(("Male", "Female")).(sexo_anotado))
```

```
0.47551540822059907
```

```
checar("7.2", p_sexado)
```

✓ Correto!

Menos da metade: muitos passeriformes da Mata Atlântica não podem ser sexados na mão. Qualquer análise por sexo é uma análise de uma metade não aleatória dos dados.

7.3 Estrutura: o que é uma réplica?

Uma linha é um registro, mas registros não são observações independentes. Eles estão agrupados de várias formas ao mesmo tempo.

Por espécie. Os registros estão muito desigualmente distribuídos:

```
por_especie = sort(combine(groupby(birds, :species), nrow => :n), :n,  
  ↪ rev = true)  
  
(nomes = nrow(por_especie),  
  cinco_mais = first(por_especie, 5),  
  com_um_registro = count(==1), por_especie.n))
```

(nomes = 780, cinco_mais = 5×2 DataFrame

| Row | species
String31 | n
Int64 |
|-----|--------------------------|-----------------------------|
| 1 | Chiroxiphia caudata | 1857 |
| 2 | Turdus albicollis | 1686 |
| 3 | Trichothraupis melanops | 1493 |
| 4 | Turdus leucomelas | 1475 |
| 5 | Basileuterus culicivorus | 1419, com_um_registro = 68) |

780 nomes — alguns identificados só até gênero, como *Elaenia* sp. —, enquanto o artigo de dados relata 711 espécies. Antes de contar espécies, decida o que conta como uma. As dez espécies mais registradas concentram um quinto de todos os registros; 68 nomes aparecem uma única vez.

Por indivíduo. Uma ave anilhada capturada de novo aparece de novo:

```
anilhados = dropmissing(birds[:, [:ring, :species]], :ring)  
por_anilha = combine(groupby(anilhados, :ring),  
  :species => (s -> length(unique(s))) =>  
  ↪ :n_species, nrow => :n)  
first(sort(por_anilha[por_anilha.n_species .> 1, :], :n, rev = true),  
  ↪ 6)
```

| | ring | n_species | n |
|---|----------|-----------|-------|
| | String31 | Int64 | Int64 |
| 1 | BRD | 25 | 31 |
| 2 | VRD | 24 | 27 |
| 3 | AZD | 21 | 26 |
| 4 | AZE | 22 | 24 |
| 5 | IND1 | 4 | 15 |
| 6 | BRE | 14 | 14 |

Algumas “anilhas” são compartilhadas por mais de vinte espécies. Não são números de anilha, e sim **códigos de anilhas coloridas**, como BRD ou BLUE RIGHT, que muitas aves carregam. Uma contagem ingênua de recapturas tomaria todas essas aves por um único indivíduo.

Por fonte e método. Cada registro vem de um de 65 pesquisadores principais, e de aves vivas ou peles de museu, ao longo de dois séculos:

```
combine(groupby(dropmissing(birds[:, [:status, :year]]), :status),
        nrow => :n, :year => minimum => :primeiro, :year => median =>
  ↪ :mediana,
        :year => maximum => :ultimo)
```

| | status | n | primeiro | mediana | ultimo |
|---|---------|-------|----------|---------|--------|
| | String7 | Int64 | Int64 | Float64 | Int64 |
| 1 | live | 57862 | 1990 | 2010.0 | 2018 |
| 2 | museum | 11152 | 1820 | 2007.0 | 2017 |

Cada um desses agrupamentos pode tornar os registros de um mesmo grupo mais parecidos entre si do que registros de grupos diferentes — a dependência que os modelos mistos da 1 levam em conta. Explorar a estrutura diz quais termos de agrupamento um modelo precisa.

i Sua vez 7.3 — Quantas aves foram capturadas duas vezes?

Conte os indivíduos registrados mais de uma vez: anilhas que aparecem em mais de um registro **da mesma espécie**, ignorando os códigos feitos só de letras (anilhas coloridas). Passe o número para checar("7.3", ...).

Dica

Descarte os códigos só de letras com `occursin(r"^[A-Za-z]+$"`, `ring`), que dá `true` para eles. Depois agrupe por `ring` e `species`, conte as linhas e conte os grupos com mais de uma.

Solução

```
anilhas_reais = anilhados[.!occursin(r"^[A-Za-z]+$",
  ↪ anilhados.ring), :]
por_ave = combine(groupby(anilhas_reais, [:ring,
  ↪ :species]), nrow => :n)
n_recapturadas = count(>(1), por_ave.n)
```

4503

```
checar("7.3", n_recapturadas)
```

✓ Correto!

Cerca de 4 500 aves contribuem com dois registros ou mais, uma delas com até 43. Tratar esses registros como independentes superestimaria o tamanho amostral — e os códigos de cor teriam acrescentado 27 “indivíduos” falsos à contagem.

7.4 Distribuições

A massa corporal de todas as aves cobre três ordens de grandeza, de beija-flores a grandes aves marinhas. Em escala linear, quase tudo se acumula à esquerda; em escala log, a forma aparece:

```
massas = collect(skipmissing(birds.mass_g))

fig = Figure(size = (860, 340))
ax1 = Axis(fig[1, 1], xlabel = "Massa corporal (g)", ylabel =
    ↳ "Registros", title = "(a) Linear")
hist!(ax1, massas, bins = 60, color = (:steelblue, 0.7))
ax2 = Axis(fig[1, 2], xlabel = "Massa corporal (g, escala log)", title =
    ↳ "(b) Logarítmica",
    xscale = log10)
hist!(ax2, massas, bins = 10 .^ range(0, 3.6, length = 60), color =
    ↳ (:steelblue, 0.7))
fig
```

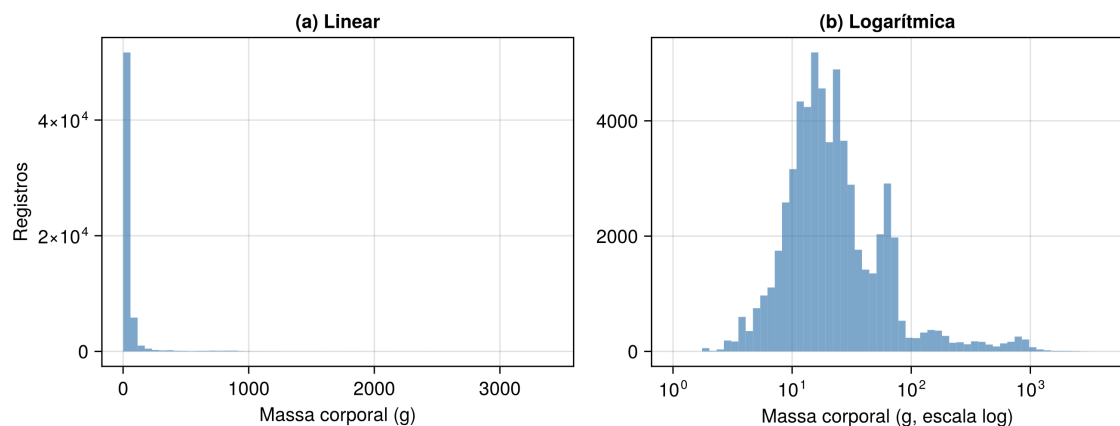


Figura 7.1: Massa corporal de todas as aves pesadas, em escala linear (a) e logarítmica (b).

Tamanhos, áreas e concentrações são limitados em zero e crescem de forma multiplicativa; por isso costumam ser assimétricos à direita e mais simétricos em escala log. É por isso que modelos dessas variáveis usam tão frequentemente o seu logaritmo.

Dentro de uma espécie, o quadro é outro. Para o sabiá-coleira (*Turdus albicollis*), uma das espécies mais registradas, compare a distribuição com uma curva normal de mesma média e desvio-padrão:

```
sabia = collect(skipmissing(birds.mass_g[birds.species .== "Turdus
↪ albicollis"]))
μ, σ = mean(sabia), std(sabia)

fig = Figure(size = (860, 340))
ax1 = Axis(fig[1, 1], xlabel = "Massa corporal (g)", ylabel =
↪ "Densidade",
           title = "(a) Histograma")
hist!(ax1, sabia, bins = 40, normalization = :pdf, color = (:seagreen,
↪ 0.6))
xs = range(minimum(sabia), maximum(sabia), length = 200)
lines!(ax1, xs, @.(exp(-(xs - μ)^2 / (2σ^2)) / (σ * sqrt(2π))), color =
↪ :black)

ax2 = Axis(fig[1, 2], xlabel = "Quantis da normal", ylabel = "Massa
↪ corporal (g)",
           title = "(b) QQ-plot")
n = length(sabia)
z = quantile.(Normal(), ((1:n) .- 0.5) ./ n)
scatter!(ax2, z, sort(sabia), color = (:seagreen, 0.5), markersize = 5)
ablines!(ax2, μ, σ, color = :black, linestyle = :dash)
fig
```

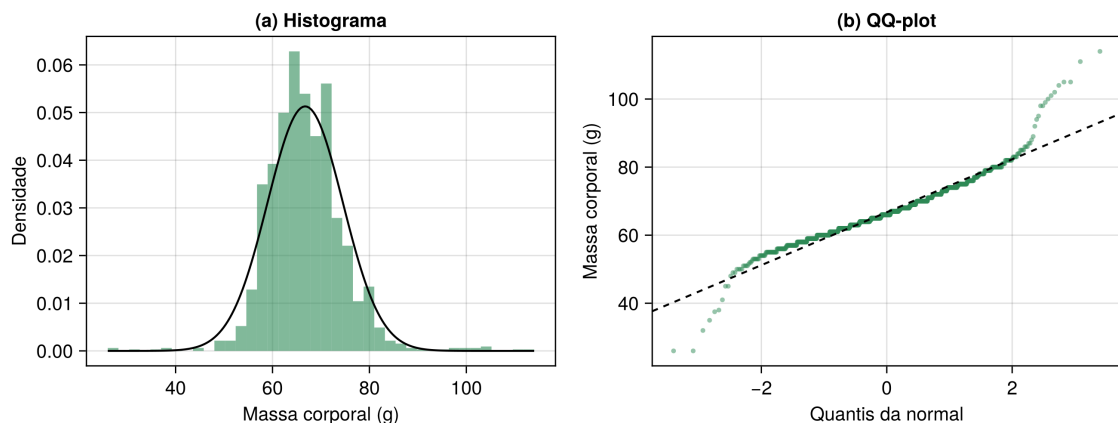


Figura 7.2: Massa corporal de *Turdus albicollis*: histograma com uma curva normal de mesma média e desvio-padrão (a), e QQ-plot contra a normal (b).

Perto da normal no meio, com algumas aves bem mais leves ou mais pesadas que as outras — visíveis como os pontos que se afastam da reta nas duas pontas do QQ-plot.

Essas caudas são o assunto da próxima seção.

i Sua vez 7.4 — Média ou mediana?

Calcule a mediana e a média da massa corporal de *Turdus albicollis* e passe-as, nessa ordem, como um vetor para `checar("7.4", ...)`. Qual das duas é menos afetada pelas aves extremas?

Dica

sabia, acima, já tem as massas. `[median(x), mean(x)]` monta o vetor.

Solução

```
centro_sabia = [median(sabia), mean(sabia)]
```

```
2-element Vector{Float64}:
```

```
66.0
```

```
66.74542818610924
```

```
checar("7.4", centro_sabia)
```

✓ Correto!

66 e 66,7 g: quase iguais, como se espera de uma distribuição aproximadamente simétrica. A mediana ignora o quão extremas são as caudas; a média é puxada por elas. Com alguns registros de 26 g ou 114 g, a média se move pouco; com os erros grosseiros da próxima seção, se moveria muito mais.

7.5 Erros e outliers

Um histograma geral não mostra o erro de uma ave de 14 g registrada com 1 240 g: 1 240 g é uma massa perfeitamente comum para uma ave grande. Outliers em dados de atributos são outliers **em relação à espécie**. Um sinal simples e robusto compara cada massa com a mediana da sua espécie:

```
pesadas = dropmissing(birds[:, [:species, :mass_g, :age, :status,
↪ :source]], :mass_g)
pesadas = transform(groupby(pesadas, :species),
                    :mass_g => median => :species_median, nrow =>
↪ :n_species)
pesadas.ratio = pesadas.mass_g ./ pesadas.species_median

suspeitas = sort(pesadas[(pesadas.n_species .>= 10) .& (pesadas.ratio
↪ .> 3), :],
```

```

      :ratio, rev = true)
first(suspeitas[, [:species, :mass_g, :species_median, :ratio, :age,
↪   :status]], 8)

```

| | species | mass_g | species_median | ratio | age | status |
|---|------------------------|---------|----------------|---------|-----------|----------|
| | String31 | Float64 | Float64 | Float64 | String15? | String7? |
| 1 | Thlypopsis sordida | 1240.0 | 14.0 | 88.5714 | Unknown | museum |
| 2 | Thlypopsis sordida | 1070.0 | 14.0 | 76.4286 | Unknown | museum |
| 3 | Corythopsis delalandi | 214.0 | 15.0 | 14.2667 | Adult | live |
| 4 | Tityra cayana | 940.0 | 73.0 | 12.8767 | Juvenile | museum |
| 5 | Tityra cayana | 840.0 | 73.0 | 11.5068 | Juvenile | museum |
| 6 | Platyrinchus mystaceus | 102.0 | 9.0 | 11.3333 | missing | missing |
| 7 | Thalurania glaucopis | 50.0 | 4.5 | 11.1111 | Adult | live |
| 8 | Phaethornis eurynome | 55.0 | 5.0 | 11.0 | Adult | live |

O topo da lista mostra vários tipos de problema:

- *Thlypopsis sordida*, uma saíra de 14 g, registrada com mais de 1 kg: quase certamente um erro de digitação, talvez um separador decimal fora do lugar ou uma unidade errada.
- Várias massas cerca de **dez vezes** a mediana da espécie — 102 g para um papamoscas de 9 g, 50 g para um beija-flor de 4,5 g: a assinatura de uma vírgula deslocada.
- Jovens pesados de *Tityra cayana* em uma coleção de museu: talvez erros, talvez um espécime com etiqueta trocada. Só a fonte pode dizer.

Agora a outra cauda:

```

first(sort(pesadas[(pesadas.n_species .>= 10) .& (pesadas.ratio .<
↪   1/3), :], :ratio)[:,
      [:species, :mass_g, :species_median, :ratio, :age, :status]], 6)

```

| | species | mass_g | species_median | ratio | age | status |
|---|---------------------------|---------|----------------|-----------|-----------|----------|
| | String31 | Float64 | Float64 | Float64 | String15? | String7? |
| 1 | Crypturellus parvirostris | 8.5 | 185.0 | 0.0459459 | Unknown | museum |
| 2 | Aramides saracura | 22.5 | 346.0 | 0.0650289 | Unknown | museum |
| 3 | Aramides saracura | 22.5 | 346.0 | 0.0650289 | Juvenile | museum |
| 4 | Aramides saracura | 24.0 | 346.0 | 0.0693642 | Unknown | museum |
| 5 | Aramides saracura | 24.0 | 346.0 | 0.0693642 | Juvenile | museum |
| 6 | Asio clamator | 34.5 | 470.0 | 0.0734043 | Unknown | museum |

Aqui alguns dos “outliers” são reais. Uma saracura-do-mato (*Aramides saracura*, mediana de 346 g) de 24 g marcada como jovem é plausivelmente um filhote. Apagá-la removeria uma ave real; mantê-la em uma análise de tamanho adulto a distorceria.

! Sinalize, não apague

Uma regra estatística só pode dizer que um valor é incomum, nunca que ele está errado. Sinalize os valores suspeitos, olhe-os um a um, volte à fonte quando puder e decida — e registre o que decidiu e por quê, em código, para que a decisão possa ser revisada. Em muitas análises, a escolha certa é restringir os dados ao que a pergunta trata (adultos, digamos) em vez de remover pontos individuais.

Os dados também carregam um problema mais discreto, a **precisão**. A maioria das massas é um número inteiro:

```
mean(pesadas.mass_g == round.(pesadas.mass_g))
```

0.7217974646080995

72% das massas são inteiras, o que quer dizer que muitas foram pesadas com precisão de 1 grama. Para um beija-flor de 3 g, arredondar ao grama é um erro de até 17%; para um sabiá, menos de 1%. Uma precisão que varia com o tamanho do animal é fácil de não notar, e vale saber disso antes de comparar espécies pequenas.

i Sua vez 7.5 — Quantas massas suspeitas?

Conte os registros cuja massa é mais de 3 vezes, ou menos de um terço, da mediana da sua espécie, entre as espécies com pelo menos 10 registros pesados. Passe o número para checar("7.5", ...).

Dica

pesadas já tem ratio e n_species. Combine as condições com .& e .| e conte os true com count.

Solução

```
n_suspeitas = count((pesadas.n_species .>= 10) .&  
                    ((pesadas.ratio .> 3) .|  
                     ↪ (pesadas.ratio .< 1/3)))
```

291

```
checar("7.5", n_suspeitas)
```

✓ Correto!

291 de cerca de 60 000 registros pesados, 0,5%: poucos, mas suficientes para arruinar a média de uma espécie. O limiar de 3 é uma escolha, não uma regra; com 2, você sinalizaria mais que o dobro.

7.6 Relações e colinearidade

Atributos morfológicos andam juntos: aves grandes têm asas longas, caudas longas, pernas longas e bicos longos. Para ver com que força, resuma cada espécie pela mediana de cada atributo — a mediana, por causa dos outliers acima — e compare as espécies:

```

atributos = [:mass_g, :wing_mm, :tail_mm, :tarsus_mm, :bill_mm]
mediana_ou_missing(x) = (v = collect(skipmissing(x)); isempty(v) ?
    ↪ missing : median(v))

por_especie_med = dropmissing(combine(groupby(birds, :species),
    ↪ [t => mediana_ou_missing => t for
    ↪ t in atributos]...))
log_atributos = DataFrame(log.(Matrix(por_especie_med[:, atributos])),
    ↪ atributos)

nrow(por_especie_med)

```

538

```

round.(cor(Matrix(log_atributos)); digits = 2)

```

5×5 Matrix{Float64}:

```

1.0  0.93  0.71  0.81  0.72
0.93  1.0  0.77  0.72  0.64
0.71  0.77  1.0  0.56  0.5
0.81  0.72  0.56  1.0  0.64
0.72  0.64  0.5  0.64  1.0

```

```

rotulos = ["log massa", "log asa", "log cauda", "log tarso", "log
    ↪ bico"]
fig = Figure(size = (820, 820))
for i in 1:5, j in 1:5
    ax = Axis(fig[i, j], xlabel = i == 5 ? rotulos[j] : "", ylabel = j
    ↪ == 1 ? rotulos[i] : "",
    ↪ xticklabelsvisible = i == 5, yticklabelsvisible = j == 1
    ↪ && i != 1)
    if i == j                # diagonal: a distribuição de cada
    ↪ atributo (contagens)
        hist!(ax, log_atributos[:, i], bins = 25, color = (:grey60,
    ↪ 0.8))
    else
        scatter!(ax, log_atributos[:, j], log_atributos[:, i],
    ↪ markersize = 3,
    ↪ color = (:steelblue, 0.5))
    end
end
end
fig

```

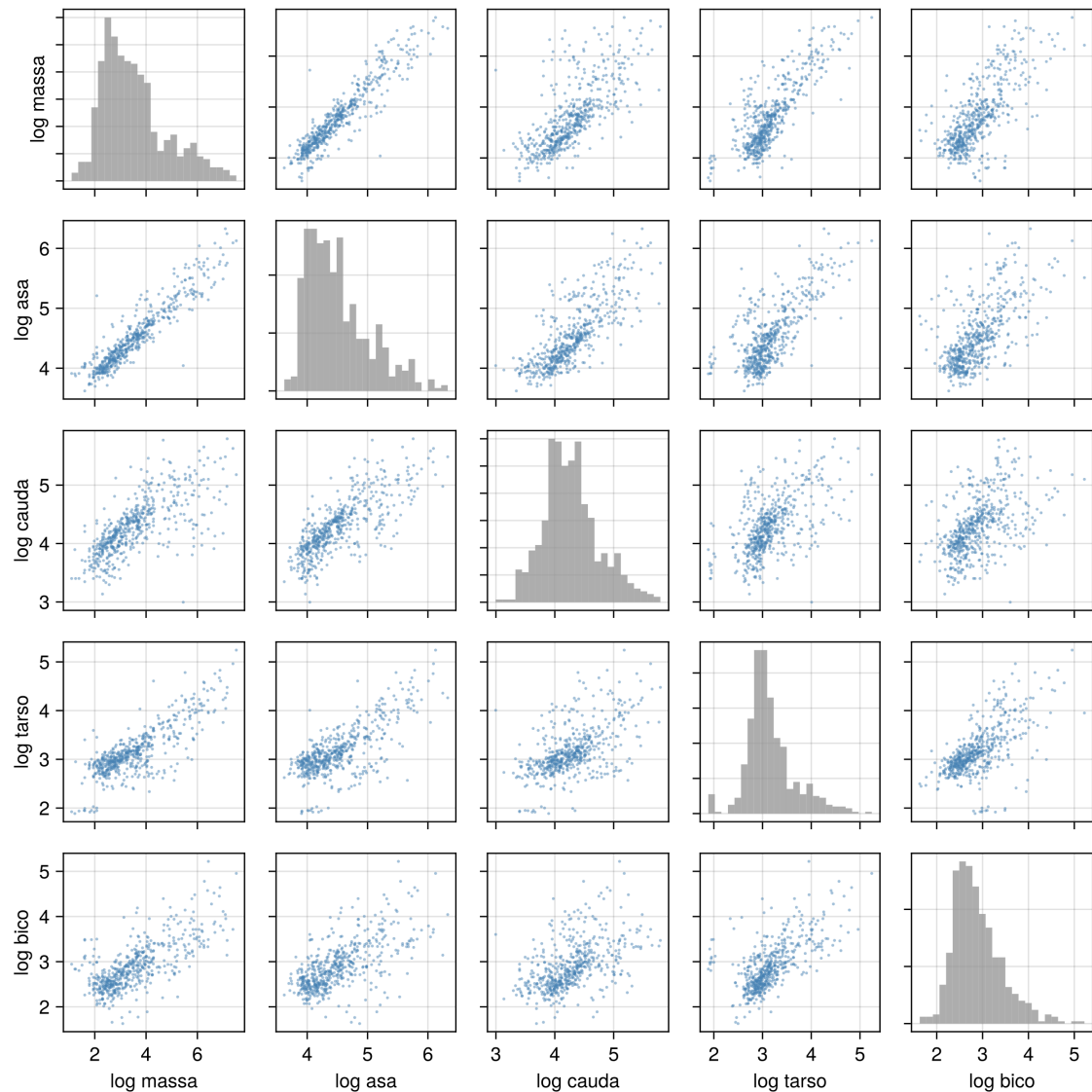


Figura 7.3: Medianas de cinco atributos por espécie, em escala log, um ponto por espécie. Fora da diagonal, cada painel mostra um par de atributos; na diagonal, a distribuição de cada atributo.

Massa e comprimento da asa são quase uma única variável entre espécies. Se os dois entrassem em um modelo como preditores, o modelo não conseguiria separar os seus efeitos: as estimativas seriam instáveis e os erros-padrão, grandes. O **fator de inflação da variância** (VIF) mede isso. Para cada preditor, faça a regressão dele em todos os outros; com o R^2 dessa regressão, $VIF = 1/(1 - R^2)$:

```
function vif(df, col)
  outros = setdiff(propertynames(df), [col])
  f = Term(col) ~ sum(Term.(outros))
```

```

    return 1 / (1 - r2(lm(f, df)))
end

DataFrame(atributo = atributos,
           vif = round.([vif(log_atributos, t) for t in atributos];
           ↪ digits = 1))

```

| | atributo | vif |
|---|-----------|---------|
| | Symbol | Float64 |
| 1 | mass_g | 11.9 |
| 2 | wing_mm | 9.7 |
| 3 | tail_mm | 2.5 |
| 4 | tarsus_mm | 3.0 |
| 5 | bill_mm | 2.1 |

Zuur et al. (2010) usam um limite de 3; outros autores usam 5 ou 10. Massa e asa ficam acima de todos eles. As respostas usuais são manter só um de cada grupo colinear, combiná-los (um índice de tamanho) ou fazer uma pergunta que não precise deles separadamente.

i Sua vez 7.6 — Massa e asa

Qual é a correlação entre log da massa corporal e log do comprimento da asa entre espécies, usando as medianas por espécie acima? Passe-a para checar("7.6", ...).

Dica

cor(x, y) em duas colunas de log_atributos.

Solução

```

r_massa_asa = cor(log_atributos.mass_g,
↪ log_atributos.wing_mm)

```

0.9326447165263585

```

checar("7.6", r_massa_asa)

```

✓ Correto!

Cerca de 0,93. Entre espécies, esses dois atributos carregam quase a mesma informação. Dentro de uma espécie, a correlação é muito mais fraca — cerca de 0,25 entre indivíduos de *Turdus albicollis* —, porque indivíduos de uma espécie variam pouco em tamanho: a colinearidade depende da escala da pergunta.

7.7 Por que os gráficos: o quarteto de Anscombe

Resumos não substituem olhar. Anscombe (1973) construiu quatro pequenos conjuntos de dados com as mesmas médias, variâncias, correlações e retas de regressão:

```
anscombe = dataset("datasets", "anscombe")
DataFrame(conjunto = 1:4,
          media_x = [mean(anscombe[:, "X$i"]) for i in 1:4],
          media_y = [round(mean(anscombe[:, "Y$i"]); digits = 2) for i
↪ in 1:4],
          cor_xy = [round(cor(anscombe[:, "X$i"], anscombe[:, "Y$i"]);
↪ digits = 3) for i in 1:4])
```

| | conjunto | media_x | media_y | cor_xy |
|---|----------|---------|---------|---------|
| | Int64 | Float64 | Float64 | Float64 |
| 1 | 1 | 9.0 | 7.5 | 0.816 |
| 2 | 2 | 9.0 | 7.5 | 0.816 |
| 3 | 3 | 9.0 | 7.5 | 0.816 |
| 4 | 4 | 9.0 | 7.5 | 0.817 |

```
fig = Figure(size = (760, 560))
for i in 1:4
    x, y = anscombe[:, "X$i"], anscombe[:, "Y$i"]
    ax = Axis(fig[(i - 1) ÷ 2 + 1, (i - 1) % 2 + 1], title = "Conjunto
↪ $i",
              limits = (3, 20, 3, 14))
    scatter!(ax, x, y, color = :steelblue)
    b = coef(lm(@formula(y ~ x), DataFrame(; x, y)))
    ablines!(ax, b[1], b[2], color = :darkorange)
end
fig
```

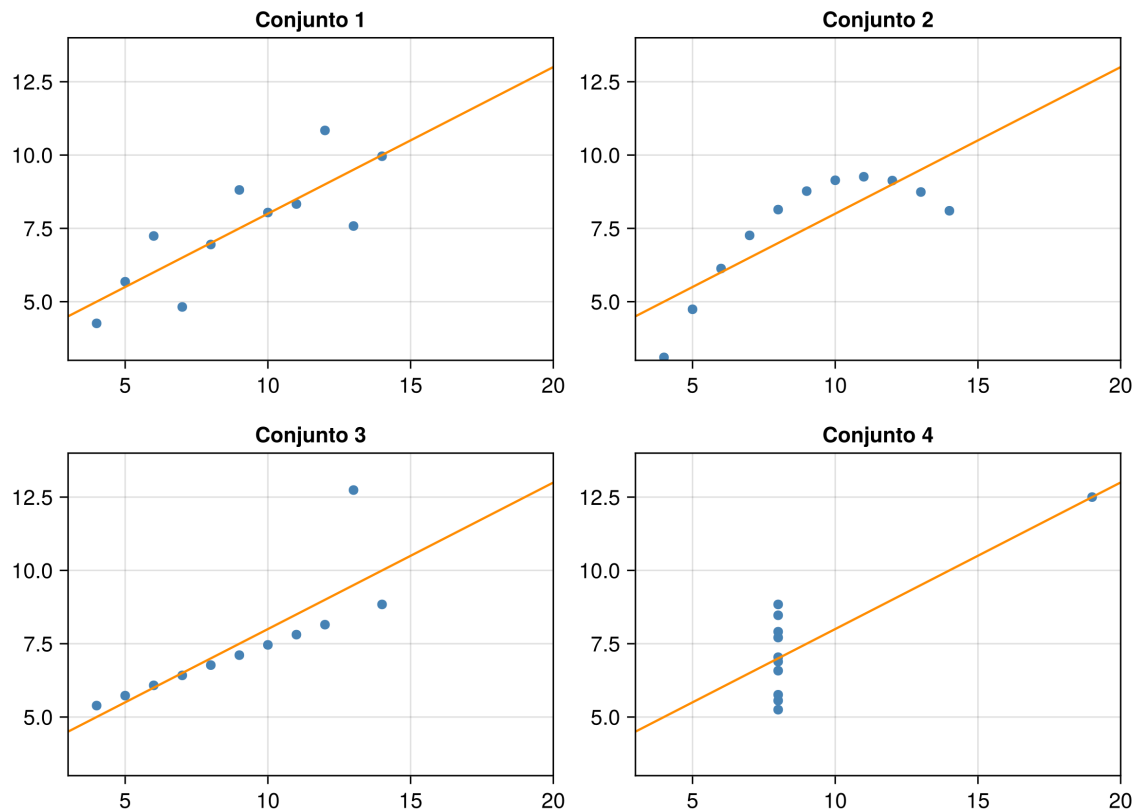


Figura 7.4: O quarteto de Anscombe: quatro conjuntos de dados com as mesmas estatísticas resumo e a mesma reta de mínimos quadrados.

O primeiro parece o que um modelo linear supõe. O segundo é uma curva. No terceiro, um ponto afasta a reta de uma relação de resto perfeita. No quarto, toda a inclinação depende de uma única observação. Só os gráficos mostram isso.

Desafios

1. Faça o gráfico das coordenadas de todos os registros (lon, lat) e pinte-os por `outside_range`. Os registros marcados como fora da distribuição da espécie estão onde você esperaria? Há pontos fora da região da Mata Atlântica?
2. Machos e fêmeas de *Chiroxiphia caudata* (tangará) diferem muito na plumagem. Diferem na massa? Compare as distribuições, não só as medianas, e decida o que fazer com os registros de sexo "Unknown".
3. Repita a regra de outliers da Seção 7.5 para o comprimento da asa. As espécies com asas suspeitas são as mesmas com massas suspeitas? O que isso diria sobre a origem dos erros?

Parte III

Modelagem

8 Modelos lineares

```
using TidierData
using DataFrames, CSV
using GLM, StatsModels, StatsBase, Statistics, Distributions
using HypothesisTests
using CairoMakie
using MixedModels

CairoMakie.activate!(type = "png")

ponds = @chain CSV.read(joinpath("../", "data", "ponds.csv"), DataFrame)
↳ begin
    @drop_missing(canopy)
    @mutate(log_area = log(area_m2))
end

nrow(ponds)
```

173

i Exercícios deste capítulo

Nos quadros *Sua vez*, você ajusta um modelo e passa para checar um número extraído dele — um valor-p, coeficientes, um intervalo. Todos usam as lagoas com dossel medido, como este capítulo. Comece cada sessão com:

```
include("exercises/checar.jl")
using TidierData, DataFrames, CSV, GLM, StatsModels, Statistics
using HypothesisTests, MixedModels
ponds = @chain CSV.read(joinpath("data", "ponds.csv"), DataFrame)
↳ begin
    @drop_missing(canopy)
    @mutate(log_area = log(area_m2))
end
```

8.1 Testes de hipótese

Antes dos modelos, o básico. `HypothesisTests.jl` cobre o repertório usual.

```
perm = @chain ponds begin @filter(hydro == "permanent") end
temp_ = @chain ponds begin @filter(hydro == "temporary") end

UnequalVarianceTTest(perm.richness, temp_.richness)
```

Two sample t-test (unequal variance)

Population details:

```
parameter of interest: Mean difference
value under h_0:      0
point estimate:       1.01849
95% confidence interval: (-0.452, 2.489)
```

Test summary:

```
outcome with 95% confidence: fail to reject h_0
two-sided p-value:          0.1734
```

Details:

```
number of observations: [95,78]
t-statistic:            1.3673234271337709
degrees of freedom:     168.03764735213443
empirical standard error: 0.7448775533304762
```

Em R

`t.test(richness ~ hydro, data = ponds)` usa Welch por padrão — o mesmo que `UnequalVarianceTTest`. Para o teste de variâncias iguais, use `EqualVarianceTTest`.

i Sua vez 8.1 — O hidroperíodo afeta o tamanho corporal?

Compare `body_size` entre lagoas permanentes e temporárias com um teste t de Welch. Passe o valor-p para `checar("8.1", ...)`.

Dica

Reaproveite `perm` e `temp_` do bloco acima. `pvalue(teste)` extrai o valor-p.

Solução

```
t_corpo = UnequalVarianceTTest(perm.body_size,
  ↪ temp_.body_size)
pvalue(t_corpo)
```

```
0.10269215805061108
```

```
checar("8.1", pvalue(t_corpo))
```

✓ Correto!

$p \approx 0,10$: nenhuma evidência de diferença. Aqui podemos ir além do usual, porque na simulação o hidroperíodo realmente não afeta o tamanho corporal. Com dados reais, um teste não significativo não mostra que o efeito não existe — só que os dados não permitem dizer.

```
# normalidade e correlação
(
  shapiro_like = ExactOneSampleKSTest(
    (ponds.body_size .- mean(ponds.body_size)) ./
    ↪ std(ponds.body_size),
    ↪ Normal(0, 1)) |> pvalue,
  correlacao = CorrelationTest(ponds.canopy, ponds.body_size) |>
    ↪ pvalue,
)
```

```
(shapiro_like = 0.9802464726099859, correlacao = 4.331805547836094e-5)
```

8.2 Regressão linear simples

A resposta contínua do conjunto é `body_size`, o tamanho corporal médio.

```
m1 = lm(@formula(body_size ~ temp_c), ponds)
```

```
StatsModels.TableRegressionModel{LinearModel{GLM.LmResp{Vector{Float64}}, GLM.DensePredChol{
```

```
body_size ~ 1 + temp_c
```

Coefficients:

| | Coef. | Std. Error | t | Pr(> t) | Lower 95% | Upper 95% |
|-------------|----------|------------|-------|----------|-----------|-----------|
| (Intercept) | 13.1837 | 1.5861 | 8.31 | <1e-13 | 10.0528 | 16.3145 |
| temp_c | 0.921736 | 0.0645072 | 14.29 | <1e-30 | 0.794403 | 1.04907 |

A interface de fórmulas de `StatsModels.jl` é a mesma do R: `~`, `+`, `*`, `:`, `-1` e transformações inline como `log(x)` funcionam como esperado.

```
(
  r2      = r2(m1),
  adj_r2  = adjr2(m1),
  aic     = aic(m1),
  coefs   = coef(m1),
  intervalos = confint(m1),
)
```

```
(r2 = 0.544209635943369, adj_r2 = 0.5415441952178917, aic = 801.3969591368274, coefs = [13
```

i Sua vez 8.2 — Tamanho corporal e dossel

Ajuste uma regressão simples de `body_size` em `canopy` e passe os dois coeficientes para `checar("8.2", ...)`.

Dica

`coef(m)` retorna o intercepto e a inclinação, nessa ordem.

Solução

```
m_dossel = lm(@formula(body_size ~ canopy), ponds)
coef(m_dossel)
```

```
2-element Vector{Float64}:
 38.134435320100685
 -0.04966562886720355
```

```
checar("8.2", coef(m_dossel))
```

✓ Correto!

A inclinação, cerca de $-0,050$, fica mais longe do verdadeiro $-0,04$ do que em `m2` (próxima seção), porque este modelo deixa a temperatura de fora.

8.3 Regressão múltipla

```
m2 = lm(@formula(body_size ~ temp_c + canopy), ponds)
```

```
StatsModels.TableRegressionModel{LinearModel{GLM.LmResp{Vector{Float64}}, GLM.DensePredChol{
```

```
body_size ~ 1 + temp_c + canopy
```

Coefficients:

| | Coef. | Std. Error | t | Pr(> t) | Lower 95% | Upper 95% |
|-------------|------------|------------|-------|----------|-----------|------------|
| (Intercept) | 15.761 | 1.54229 | 10.22 | <1e-18 | 12.7164 | 18.8055 |
| temp_c | 0.901451 | 0.059815 | 15.07 | <1e-32 | 0.783375 | 1.01953 |
| canopy | -0.0423641 | 0.00777948 | -5.45 | <1e-06 | -0.057721 | -0.0270073 |

O processo gerador (ver `scripts/make_data.jl`) usou $38.0 + 0.9(\text{temp} - 24.5) - 0.04\text{canopy}$. Compare com as estimativas: essa é a vantagem de ensinar com dados simulados.

```
DataFrame(
  termo      = coefnames(m2),
  estimado   = round.(coef(m2), digits = 3),
  verdadeiro = [38.0 - 0.9 * 24.5, 0.9, -0.04],
)
```

| | termo | estimado | verdadeiro |
|---|-------------|----------|------------|
| | String | Float64 | Float64 |
| 1 | (Intercept) | 15.761 | 15.95 |
| 2 | temp_c | 0.901 | 0.9 |
| 3 | canopy | -0.042 | -0.04 |

i Sua vez 8.3 — O intervalo captura a verdade?

Ajuste `body_size ~ temp_c + canopy + hydro` e calcule o intervalo de confiança de 95% para o coeficiente de `temp_c`. Ele contém o valor verdadeiro, 0,9? Passe o intervalo (dois números) para `checar("8.3", ...)`.

Dica

`confint(m)` retorna uma linha por coeficiente; `temp_c` é a segunda, então `confint(m)[2, :]`.

Solução

```
m_hidro = lm(@formula(body_size ~ temp_c + canopy +
  ↪ hydro), ponds)
ic_temp = confint(m_hidro)[2, :]
```

```
2-element Vector{Float64}:
 0.7785892869884607
 1.0167713699307568
```

```
checar("8.3", ic_temp)
```

✓ Correto!

O intervalo, cerca de $[0,78; 1,02]$, contém 0,9. Em cerca de um a cada 20 conjuntos de dados simulados, não conteria: é isso que “95%” significa.

8.4 Preditores categóricos e interações

```
m3 = lm(@formula(body_size ~ temp_c + canopy + region), ponds)
```

```
StatsModels.TableRegressionModel{LinearModel{GLM.LmResp{Vector{Float64}}, GLM.DensePredCho
```

```
body_size ~ 1 + temp_c + canopy + region
```

Coefficients:

| | Coef. | Std. Error | t | Pr(> t) | Lower 95% | Upper 95% |
|------------------|------------|------------|-------|----------|-----------|-----------|
| (Intercept) | 15.769 | 1.54634 | 10.20 | <1e-18 | 12.7162 | 18.8218 |
| temp_c | 0.893539 | 0.0597492 | 14.95 | <1e-31 | 0.775583 | 1.0115 |
| canopy | -0.0437397 | 0.00780336 | -5.61 | <1e-07 | -0.059145 | - |
| 0.0283344 | | | | | | |
| region: Atlantic | 0.664579 | 0.436092 | 1.52 | 0.1294 | -0.196347 | 1.52551 |
| region: Cerrado | 0.018937 | 0.43474 | 0.04 | 0.9653 | -0.83932 | 0.877194 |

Julia trata String e CategoricalArray como fatores automaticamente, com codificação de tratamento (dummy) e o primeiro nível como referência — igual ao R. Para mudar a referência ou o esquema de contrastes:

```
m3b = lm(@formula(body_size ~ temp_c + region), ponds;  
         contrasts = Dict{:region} => DummyCoding(base = "Cerrado"))  
  
coefnames(m3b)
```

```
4-element Vector{String}:
```

```
"(Intercept)"
```

```
"temp_c"
```

```
"region: Amazon"
```

```
"region: Atlantic"
```

i Sua vez 8.4 — Escolhendo a referência

Ajuste `body_size ~ temp_c + region` com a região Atlantic como nível de referência e passe os nomes dos coeficientes para checar ("8.4", ...).

Dica

Copie o argumento `contrasts` do bloco acima e troque o nível.

Solução

```
m_atl = lm(@formula(body_size ~ temp_c + region), ponds;
           contrasts = Dict(:region => DummyCoding(base =
             ↪ "Atlantic"))))
coefnames(m_atl)
```

```
4-element Vector{String}:
 "(Intercept)"
 "temp_c"
 "region: Amazon"
 "region: Cerrado"
```

```
checar("8.4", coefnames(m_atl))
```

✓ Correto!

O nível de referência não tem coeficiente próprio: ele fica absorvido no intercepto, e as outras regiões são comparadas com ele. Sem contrasts, a referência é o primeiro nível em ordem alfabética, Amazon.

```
m4 = lm(@formula(body_size ~ temp_c * hydro), ponds)
```

```
StatsModels.TableRegressionModel{LinearModel{GLM.LmResp{Vector{Float64}}, GLM.DensePredChol{Vector{Float64}}}}
```

```
body_size ~ 1 + temp_c + hydro + temp_c & hydro
```

Coefficients:

| | Coef. | Std. Error | t | Pr(> t) | Lower 95% | Upper 95% |
|---------------------------|----------|------------|-------|----------|-----------|-----------|
| (Intercept) | 14.8606 | 2.16225 | 6.87 | <1e-09 | 10.5921 | 19.1291 |
| temp_c | 0.847037 | 0.0890498 | 9.51 | <1e-16 | 0.671244 | 1.02283 |
| hydro: temporary | -3.33192 | 3.20923 | -1.04 | 0.3006 | -9.66726 | 3.00341 |
| temp_c & hydro: temporary | 0.147595 | 0.130228 | 1.13 | 0.2587 | -0.109489 | 0.404679 |

8.5 Seleção e comparação de modelos

```
modelos = ["temp"      => m1,
           "temp+canopy" => m2,
           "temp+can+reg" => m3,
           "temp*hydro"  => m4]
```

```
DataFrame(
  modelo = first(modelos),
  k       = [length(coef(m)) for (_, m) in modelos],
  aic     = round([aic(m) for (_, m) in modelos], digits = 1),
  bic     = round([bic(m) for (_, m) in modelos], digits = 1),
  adj_r2  = round([adjr2(m) for (_, m) in modelos], digits = 3),
)
```

| | modelo | k | aic | bic | adj_r2 |
|---|--------------|-------|---------|---------|---------|
| | String | Int64 | Float64 | Float64 | Float64 |
| 1 | temp | 2 | 801.4 | 810.9 | 0.542 |
| 2 | temp+canopy | 3 | 775.6 | 788.2 | 0.607 |
| 3 | temp+can+reg | 5 | 776.1 | 795.0 | 0.611 |
| 4 | temp*hydro | 4 | 803.5 | 819.3 | 0.541 |

```
# teste F entre modelos aninhados, equivalente a anova() no R
fctest(m1.model, m2.model, m3.model)
```

F-test: 3 models fitted on 173 observations

| | DOF | ΔDOF | SSR | ΔSSR | R ² | ΔR ² | F* | p(>F) |
|-----|-----|------|-----------|-----------|----------------|-----------------|---------|--------|
| [1] | 3 | | 1005.3438 | | 0.5442 | | | |
| [2] | 4 | 1 | 856.0194 | -149.3243 | 0.6119 | 0.0677 | 29.6549 | <1e-06 |
| [3] | 6 | 2 | 839.0138 | -17.0056 | 0.6196 | 0.0077 | 1.7026 | 0.1853 |

8.6 Diagnóstico

```
res = residuals(m3)
fit_ = predict(m3)

fig = Figure(size = (860, 350))

ax1 = Axis(fig[1, 1], xlabel = "Valores ajustados", ylabel =
  ↳ "Resíduos",
            title = "(a) Resíduos vs ajustados")
scatter!(ax1, fit_, res, color = (:steelblue, 0.6))
hlines!(ax1, 0, color = :grey40, linestyle = :dash)

ax2 = Axis(fig[1, 2], xlabel = "Quantis teóricos", ylabel = "Quantis
  ↳ amostrais",
            title = "(b) QQ-plot")
# NOTA: Makie exporta `qqnorm` como receita de gráfico, então os quantis
  ↳ são
# calculados explicitamente aqui em vez de virem de uma função
  ↳ auxiliar.
```

```

n = length(res)
teoricos = quantile.(Normal(0, 1), ((1:n) .- 0.5) ./ n)
scatter!(ax2, teoricos, sort(res), color = (:darkorange, 0.6))
ablines!(ax2, 0, std(res), color = :grey40, linestyle = :dash)

ax3 = Axis(fig[1, 3], xlabel = "Resíduos", ylabel = "Frequência",
           title = "(c) Distribuição")
hist!(ax3, res, bins = 20, color = (:seagreen, 0.7))

fig

```

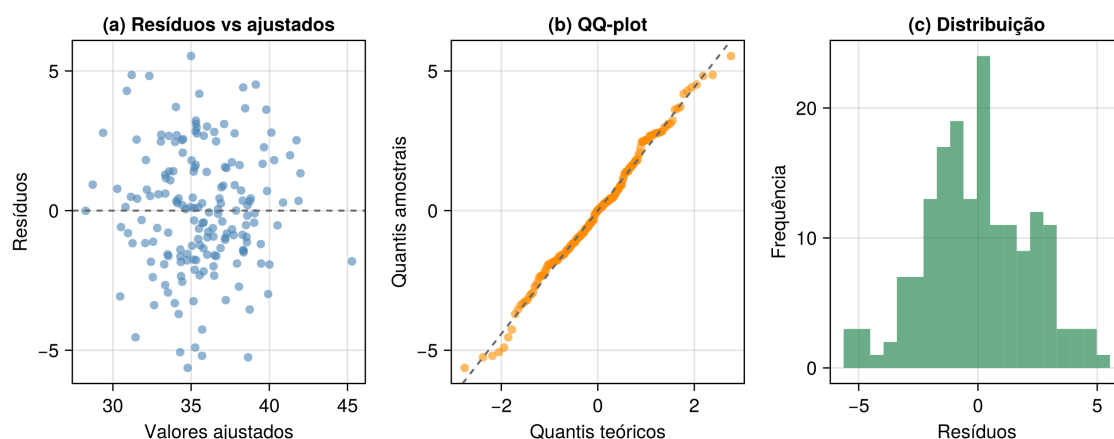


Figura 8.1: Diagnóstico de resíduos do modelo `m3`.

⚠ Uma lacuna real

R tem `plot(modelo)`, que devolve os quatro gráficos de diagnóstico de uma vez, e `performance::check_model()`, que vai muito além. Julia não tem equivalente consolidado; os diagnósticos acima são montados à mão. É uma lacuna genuína, e o tipo de coisa que um pacote da comunidade resolveria bem.

i Sua vez 8.5 — Quantos resíduos grandes?

Se os resíduos de `m3` forem aproximadamente normais, cerca de 5% deles devem estar a mais de dois desvios-padrão de zero. Calcule a proporção que está e passe-a para `checar("8.5", ...)`.

Dica

Divida `res` (do bloco de diagnóstico) por `std(res)` e tire a média de `abs.(z) .> 2` — a média de valores `true/false` é uma proporção.

Solução

```
z = res ./ std(res)
prop_grandes = mean(abs.(z) .> 2)
```

0.057803468208092484

```
checar("8.5", prop_grandes)
```

✓ Correto!

Cerca de 5,8% — 10 de 173 resíduos, perto dos 5% esperados. É a versão numérica do QQ-plot acima.

8.7 Modelos mistos

MixedModels.jl (Bates et al. 2026) é mantido por Douglas Bates, primeiro autor do lme4 (Bates et al. 2015), e usa a mesma notação de fórmula — inclusive $(1 \mid \text{grupo})$.

Sobre desempenho: comparações entre os dois dependem fortemente do modelo, do tamanho dos dados e da estrutura dos efeitos aleatórios, e não conheço um benchmark publicado que eu possa citar aqui. Se o tempo de ajuste importa para o seu problema, meça no seu próprio caso.

```
mm1 = fit(MixedModel,
          @formula(body_size ~ temp_c + canopy + (1 | region)),
          ponds)
```

Minimizing 2 Time: 0:00:00 (0.16 s/it)
objective: 780.4261079240766

Minimizing 18 Time: 0:00:00 (32.44 ms/it)

| | Est. | SE | z | p | σ_{region} |
|-------------|---------|--------|-------|--------|--------------------------|
| (Intercept) | 15.7973 | 1.5290 | 10.33 | <1e-24 | 0.1246 |
| temp_c | 0.9002 | 0.0592 | 15.20 | <1e-51 | |
| canopy | -0.0426 | 0.0077 | -5.52 | <1e-07 | |
| Residual | 2.2209 | | | | |

```
(
  efeitos_fixos = fixef(mm1),
```

```
desvios_random = VarCorr(mm1),
icc             = first(mm1.σs.region)^2 /
                  (first(mm1.σs.region)^2 + mm1.σ^2),
)
```

```
(efeitos_fixos = [15.79726348949173, 0.9002362213796093, -0.042569241640770514], desvios_r
      Column   Variance Std.Dev.
region  (Intercept)  0.015537 0.124649
Residual                4.932481 2.220919
, icc = 0.0031401442761034163)
```

Em R

```
lmer(body_size ~ temp_c + canopy + (1 | region), data = ponds)
```

Idêntico, inclusive a notação (1 | grupo). MixedModels.jl também suporta (1 + x | grupo), (1 | a) + (1 | b) e termos aninhados.

i Sua vez 8.6 — Um intercepto aleatório para região

Ajuste um modelo misto de `body_size` em `temp_c` com intercepto aleatório para `region` e passe os efeitos fixos para checar("8.6", ...). Depois olhe o `VarCorr` do seu modelo: quanto da variação a região explica?

Dica

Mesma estrutura de `mm1`, com uma fórmula menor. `fixef(mm)` retorna os efeitos fixos.

Solução

```
mm_temp = fit(MixedModel, @formula(body_size ~ temp_c +
  ↪ (1 | region)), ponds;
              progress = false)
fixef(mm_temp)
```

```
2-element Vector{Float64}:
 13.183682687070387
  0.9217361109462527
```

```
checar("8.6", fixef(mm_temp))
VarCorr(mm_temp)
```

✓ Correto!

| | Column | Variance | Std.Dev |
|----------|-------------|----------|---------|
| region | (Intercept) | 0.00000 | 0.00000 |
| Residual | | 5.81124 | 2.41065 |

A variância de região é estimada como exatamente zero — um ajuste *singular*. A simulação não tem efeito de região sobre o tamanho corporal, e três regiões trazem muito pouca informação sobre uma variância de qualquer forma. Com variância zero, o modelo misto se reduz a `lm(@formula(body_size ~ temp_c), ponds)`: compare os coeficientes.

```
# intervalos por bootstrap paramétrico – rápido o bastante para ser
  ↳ rotina
using Random
boot = parametricbootstrap(MersenneTwister(42), 1000, mm1)
DataFrame(shortestcovint(boot))
```

Desafios

1. Ajuste `body_size ~ canopy` separadamente para cada região e compare as inclinações com as de um único modelo com interação `canopy * region`. Por que elas coincidem?
2. O modelo `m4` tem interação significativa? A simulação não tem interação `temp_c × hydro`: se a sua fosse significativa, o que isso significaria?
3. No 8.6 a variância de região saiu zero. Compare `mm1` com um modelo que trata `region` como efeito fixo. Com três regiões, qual você reportaria, e por quê?

9 Modelos Lineares Generalizados

```
using TidierData
using DataFrames, CSV
using GLM, StatsModels, StatsBase, Statistics, Distributions
using MixedModels
using CairoMakie

CairoMakie.activate!(type = "png")

ponds = @chain CSV.read(joinpath("../", "data", "ponds.csv"), DataFrame)
↳ begin
    @drop_missing(canopy)
    @mutate(log_area = log(area_m2))
end

nrow(ponds)
```

173

i Exercícios deste capítulo

Como no Capítulo 8, em cada quadro *Sua vez* você ajusta um modelo e passa para checar um número extraído dele. Comece cada sessão com:

```
include("exercises/checar.jl")
using TidierData, DataFrames, CSV, GLM, StatsModels, Statistics,
↳ MixedModels
ponds = @chain CSV.read(joinpath("data", "ponds.csv"), DataFrame)
↳ begin
    @drop_missing(canopy)
    @mutate(log_area = log(area_m2))
end
```

9.1 Por que GLMs em ecologia

Contagens não são normais, proporções vivem em $[0, 1]$, presenças são binárias. O modelo linear gaussiano trata todas como se fossem contínuas e ilimitadas, e falha de formas previsíveis: intervalos de confiança que incluem riquezas negativas, variância mal especificada, resíduos em leque.

O GLM resolve isso com duas escolhas: a **distribuição** da resposta e a **função de ligação** que conecta a média ao preditor linear.

| Resposta | Distribuição | Ligação | Em Julia |
|----------------------------------|--------------------|---------|----------------------------|
| Contagem | Poisson | log | Poisson(),
LogLink() |
| Contagem
sobredispersa | Binomial negativa | log | negbin(...) |
| Presença/ausência | Bernoulli/Binomial | logit | Binomial(),
LogitLink() |
| Proporção com
denominador | Binomial | logit | Binomial(),
LogitLink() |
| Contínua positiva
assimétrica | Gama | log | Gamma(),
LogLink() |

9.2 Poisson: riqueza de espécies

```
g1 = glm(@formula(richness ~ log_area + canopy + temp_c),
         ponds, Poisson(), LogLink())
```

StatsModels.TableRegressionModel{GeneralizedLinearModel{GLM.GlmResp{Vector{Float64}}, Poisson{1}}

richness ~ 1 + log_area + canopy + temp_c

Coefficients:

| | Coef. | Std. Error | z | Pr(> z) | Lower 95% | Upper 95% |
|-------------|-----------|------------|-------|----------|------------|-------------|
| (Intercept) | -0.30001 | 0.289756 | -1.04 | 0.3005 | -0.867921 | 0.267902 |
| log_area | 0.331763 | 0.0300907 | 11.03 | <1e-27 | 0.272786 | 0.390739 |
| canopy | -0.011555 | 0.0013362 | -8.65 | <1e-17 | -0.0141739 | -0.00893614 |
| temp_c | 0.0441958 | 0.00959497 | 4.61 | <1e-05 | 0.02539 | 0.0630016 |

O processo gerador usou $\log \mu = 0.9 + 0.28 \log(\text{área}) - 0.011 \text{ canopy} + 0.03 (\text{temp} - 24.5) + \dots$

```
DataFrame(
  termo      = coefnames(g1),
  estimado   = round.(coef(g1), digits = 4),
  verdadeiro = [missing, 0.28, -0.011, 0.03],
)
```

| | termo | estimado | verdadeiro |
|---|-------------|----------|----------------|
| | String | Float64 | Float64? |
| 1 | (Intercept) | -0.3 | <i>missing</i> |
| 2 | log_area | 0.3318 | 0.28 |
| 3 | canopy | -0.0116 | -0.011 |
| 4 | temp_c | 0.0442 | 0.03 |

i Sua vez 9.1 — Incluindo o hidroperíodo

Ajuste um GLM Poisson como o g1, incluindo hydro no fim da fórmula. Passe os coeficientes para checar("9.1", ...) e depois compare o AIC dele com o de g1.

Dica

Copie o g1 e acrescente + hydro. aic(g) dá o AIC; quanto menor, melhor.

Solução

```
g_hidro = glm(@formula(richness ~ log_area + canopy +
  ↪ temp_c + hydro),
  ponds, Poisson(), LogLink())
coef(g_hidro)
```

5-element Vector{Float64}:

```
-0.2955819444424689
 0.3308987450570613
-0.011543093000739657
 0.04752717672780732
-0.1895805246826413
```

```
checar("9.1", coef(g_hidro))
(aic_g1 = aic(g1), aic_hidro = aic(g_hidro))
```

✓ Correto!

```
(aic_g1 = 952.9021516391434, aic_hidro = 944.1070557908267)
```

O AIC cai cerca de 9, então hydro merece o lugar. A simulação somou 0,22 à média log das lagoas permanentes; o modelo estima −0,19 para as temporárias, o mesmo efeito visto do outro lado.

💡 Interpretando coeficientes na escala log

Um coeficiente β na ligação log significa que um aumento de uma unidade no preditor multiplica a média por e^β . Para canopy:

```
exp(coef(g1)[3]), (exp(coef(g1)[3]) - 1) * 100
```

(0.9885114593513831, -1.148854064861693)

Ou seja, cada ponto percentual de dossel a mais reduz a riqueza esperada em cerca de 1%.

i Sua vez 9.2 — Lagoas temporárias, em números simples

Usando o modelo do 9.1, por qual fator a riqueza esperada muda em uma lagoa temporária em comparação com uma permanente, mantido o resto igual? Passe esse fator para checar("9.2", ...).

Dica

O coeficiente de hydro: temporary é o quinto. Na ligação log, um coeficiente β multiplica a média por e^β .

Solução

```
fator_temp = exp(coef(g_hidro)[5])
```

0.8273060956543652

```
checar("9.2", fator_temp)
```

✓ Correto!

Cerca de 0,83: espera-se que lagoas temporárias tenham cerca de 17% menos espécies que as permanentes.

9.3 Sobredispersão

A Poisson assume $\text{Var}(Y) = E(Y)$. Dados ecológicos raramente cooperam.

```
# Resíduo de Pearson: (y - mu) / sqrt(Var(mu)). Na Poisson, Var(mu) =  
#   mu.  
mu = predict(g1)  
res_pearson = (ponds.richness .- mu) ./ sqrt.(mu)  
gl = dof_residual(g1)  
phi = sum(res_pearson .^ 2) / gl  
  
(estatistica_dispersao = phi,
```

```
diagnostico = phi > 1.5 ? "sobredisperso" : "aceitável")
```

```
(estatistica_dispersao = 1.952703419456855, diagnostico = "sobredisperso")
```

Se $\hat{\phi} \gg 1$, as alternativas são a binomial negativa ou um modelo misto com efeito aleatório em nível de observação.

! Sobredispersão é um sintoma, não um diagnóstico

ponds.csv foi gerado como um processo Poisson *puro* — não há variância extra-Poisson no código gerador. Ainda assim, $\hat{\phi} \approx 1.95$. Por quê?

Porque o modelo gerador incluía um efeito de região que gl omite. Heterogeneidade não modelada infla a estatística de dispersão exatamente como a sobredispersão genuína, e a estatística não distingue os dois casos.

Isso importa na prática: o reflexo de recorrer à binomial negativa sempre que $\hat{\phi} > 1.5$ pode encobrir um preditor ausente. Verifique a estrutura do modelo antes de trocar a distribuição. O GLMM da Seção 9.6 trata da causa real aqui.

i Sua vez 9.3 — Testando o aviso

O quadro acima afirma que a sobredispersão de gl vem de um preditor ausente. Teste isso: ajuste um GLM Poisson com log_area, canopy, temp_c, hydro e region, calcule a estatística de dispersão como no bloco de dispersão e passe-a para checar("9.3", ...).

Dica

Copie o bloco de dispersão, trocando gl pelo seu modelo. predict(g) dá as médias ajustadas.

Solução

```
g_completo = glm(@formula(richness ~ log_area + canopy +
  ↪ temp_c + hydro + region),
  ponds, Poisson(), LogLink())

mu_completo = predict(g_completo)
phi_completo = sum(((ponds.richness .- mu_completo) ./
  ↪ sqrt(mu_completo)) .^ 2) /
  dof_residual(g_completo)
```

0.9410041742922786

```
checar("9.3", phi_completo)
```

✓ Correto!

Cerca de 0,94: com os preditores do modelo gerador no lugar, a “sobredispersão” desaparece. Incluir só hydro (9.1) quase não a mudou; region era a peça que faltava.

```
g2 = negbin(@formula(richness ~ log_area + canopy + temp_c), ponds,
  ↪ LogLink())
```

StatsModels.TableRegressionModel{GeneralizedLinearModel{GLM.GlmResp{Vector{Float64}}, Negat.

richness ~ 1 + log_area + canopy + temp_c

Coefficients:

| | Coef. | Std. Error | z | Pr(> z) | Lower 95% | Upper 95% |
|-------------|------------|------------|-------|----------|------------|-------------|
| (Intercept) | -0.165087 | 0.424739 | -0.39 | 0.6975 | -0.997561 | 0.667387 |
| log_area | 0.334165 | 0.0432899 | 7.72 | <1e-13 | 0.249319 | 0.419012 |
| canopy | -0.0120529 | 0.0019084 | -6.32 | <1e-09 | -0.0157933 | -0.00831254 |
| temp_c | 0.0391606 | 0.0141787 | 2.76 | 0.0057 | 0.0113708 | 0.0669503 |

```
DataFrame(
  modelo = ["Poisson", "Binomial negativa"],
  aic = round.([aic(g1), aic(g2)], digits = 1),
  bic = round.([bic(g1), bic(g2)], digits = 1),
  loglik = round.([loglikelihood(g1), loglikelihood(g2)], digits =
  ↪ 1),
)
```

| | modelo | aic | bic | loglik |
|---|-------------------|---------|---------|---------|
| | String | Float64 | Float64 | Float64 |
| 1 | Poisson | 952.9 | 965.5 | -472.5 |
| 2 | Binomial negativa | 911.3 | 927.0 | -450.6 |

9.4 Offsets

Quando o esforço amostral varia entre unidades, modele a **taxa**, não a contagem bruta, com um offset.

```
g3 = glm(@formula(richness ~ canopy + temp_c), ponds, Poisson(),
  ↪ LogLink();
  offset = log.(ponds.area_m2))

coefstable(g3)
```

| | Coef. | Std. Error | z | Pr(> z) | Lower 95% | Upper 95% |
|-------------|-----------|------------|--------|----------|-----------|-----------|
| (Intercept) | -3.56359 | 0.244593 | -14.57 | <1e-47 | -4.04299 | -3.0842 |
| canopy | - | 0.001304 | -11.60 | <1e-30 | - | - |
| | 0.0151224 | | | | 0.0176782 | 0.0125666 |
| temp_c | 0.0310744 | 0.00949171 | 3.27 | 0.0011 | 0.012471 | 0.0496778 |

Isso ajusta riqueza *por unidade de área*, fixando o coeficiente de log(área) em 1.

9.5 Regressão logística

```
g4 = glm(@formula(focal ~ canopy + temp_c + hydro),
         ponds, Binomial(), LogitLink())
```

StatsModels.TableRegressionModel{GeneralizedLinearModel{GLM.GlmResp{Vector{Float64}}, Binomial{Float64, Vector{Float64}}, StatsModels.LogitLink{Float64}}

focal ~ 1 + canopy + temp_c + hydro

Coefficients:

| | Coef. | Std. Error | z | Pr(> z) | Lower 95% | Upper 95% |
|------------------|-----------|------------|-------|----------|-----------|-----------|
| (Intercept) | -5.79176 | 1.6009 | -3.62 | 0.0003 | -8.92947 | - |
| 2.65406 | | | | | | |
| canopy | 0.0172429 | 0.00759302 | 2.27 | 0.0232 | 0.0023609 | 0.032125 |
| temp_c | 0.209118 | 0.0616577 | 3.39 | 0.0007 | 0.088271 | 0.329965 |
| hydro: temporary | -0.926093 | 0.338929 | -2.73 | 0.0063 | -1.59038 | - |
| 0.261805 | | | | | | |

```
DataFrame(
  termo      = coefnames(g4),
  coef       = round.(coef(g4), digits = 4),
  odds_ratio = round.(exp.(coef(g4)), digits = 3),
  ic_inf     = round.(exp.(confint(g4)[:, 1]), digits = 3),
  ic_sup     = round.(exp.(confint(g4)[:, 2]), digits = 3),
)
```

| | termo | coef | odds_ratio | ic_inf | ic_sup |
|---|------------------|---------|------------|---------|---------|
| | String | Float64 | Float64 | Float64 | Float64 |
| 1 | (Intercept) | -5.7918 | 0.003 | 0.0 | 0.07 |
| 2 | canopy | 0.0172 | 1.017 | 1.002 | 1.033 |
| 3 | temp_c | 0.2091 | 1.233 | 1.092 | 1.391 |
| 4 | hydro: temporary | -0.9261 | 0.396 | 0.204 | 0.77 |

i Sua vez 9.4 — Chances para uma mudança que faz sentido

A razão de chances para um ponto percentual de dossel fica perto de 1, o que é difícil de interpretar. Calcule, a partir de `g4`, a razão de chances para um aumento de **10** pontos percentuais e passe-a para `checar("9.4", ...)`.

Dica

Dez pontos multiplicam as chances por $e^{10\beta}$, não por $10e^\beta$.

Solução

```
rc_10 = exp(10 * coef(g4)[2])
```

```
1.1881878414800107
```

```
checar("9.4", rc_10)
```

✓ Correto!

Cerca de 1,19: cada 10 pontos a mais de dossel multiplicam as chances de encontrar a espécie-foco por 1,19, um aumento de 19%.

```
grade = DataFrame(
    canopy = repeat(range(0, 100, length = 100), 2),
    temp_c = fill(mean(ponds.temp_c), 200),
    hydro = repeat(["permanent", "temporary"], inner = 100),
)
grade.p = predict(g4, grade)

fig = Figure(size = (700, 430))
ax = Axis(fig[1, 1], xlabel = "Cobertura de dossel (%)",
          ylabel = "P(ocorrência)", title = "Espécie-foco")

for (h, cor) in zip(["permanent", "temporary"], [:steelblue,
    ↪ :darkorange])
    sub = grade[grade.hydro .== h, :]
    lines!(ax, sub.canopy, sub.p, color = cor, linewidth = 2.5, label
    ↪ = h)
end
scatter!(ax, ponds.canopy, Float64.(ponds.focal), color = (:grey40,
    ↪ 0.25))
axislegend(ax, position = :lt)

fig
```

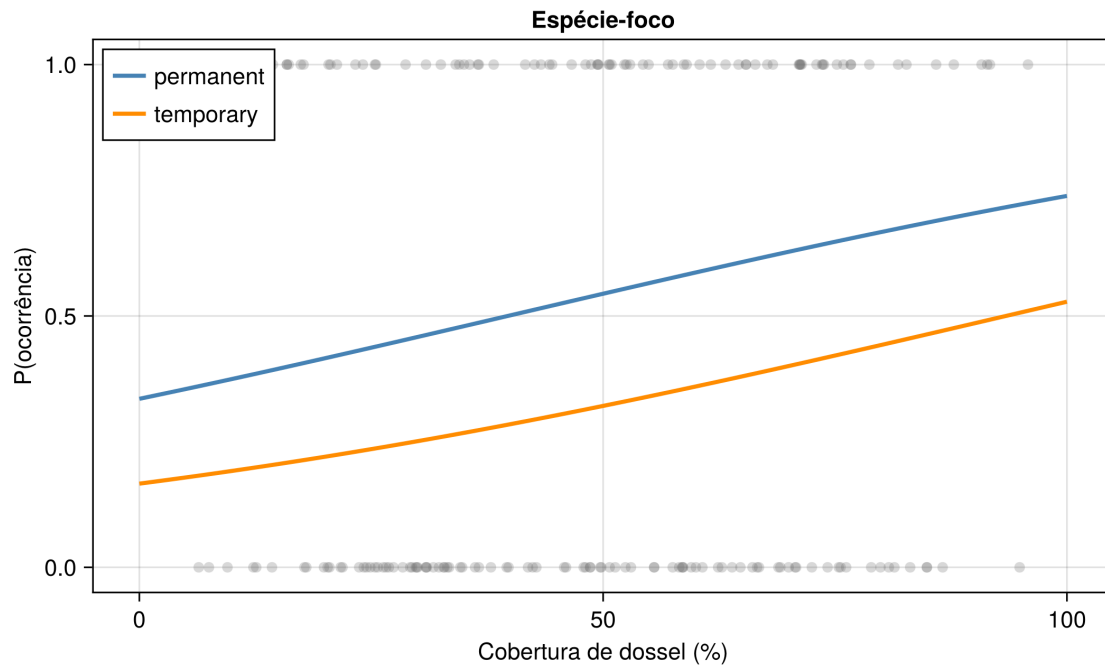


Figura 9.1: Probabilidade prevista de ocorrência da espécie-foco ao longo do gradiente de dossel.

i Sua vez 9.5 — Prevendo uma lagoa nova

Uma lagoa temporária tem 80% de dossel e água a 26 °C. Segundo o g4, qual a probabilidade de encontrar a espécie-foco nela? Passe-a para `checar("9.5", ...)`.

Dica

Monte um `DataFrame` de uma linha com as colunas `canopy`, `temp_c` e `hydro` e passe-o para `predict(g4, ...)`, como com grade acima.

Solução

```
lagoa_nova = DataFrame(canopy = [80.0], temp_c = [26.0],
  ↪ hydro = ["temporary"])
p_nova = predict(g4, lagoa_nova)
```

```
1-element Vector{Union{Missing, Float64}}:
 0.5246412509540388
```

```
checar("9.5", p_nova)
```

✓ Correto!

Cerca de 0,52. Em um GLM logístico, `predict` retorna probabilidades por padrão: a função de ligação é desfeita para você.

9.6 GLMMs

```
gml = fit(MixedModel,  
          @formula(richness ~ log_area + canopy + (1 | region)),  
          ponds, Poisson())
```

Minimizing 2 Time: 0:00:00 (0.23 s/it)
objective: 217.73449006691334

Minimizing 281 Time: 0:00:00 (2.87 ms/it)

| | Est. | SE | z | p | σ_{region} |
|-------------|---------|--------|-------|--------|--------------------------|
| (Intercept) | 0.5193 | 0.2688 | 1.93 | 0.0533 | 0.3534 |
| log_area | 0.3702 | 0.0307 | 12.05 | <1e-32 | |
| canopy | -0.0103 | 0.0013 | -7.72 | <1e-13 | |

Em R

```
glmer(richness ~ log_area + canopy + (1 | region),  
       data = ponds, family = poisson)
```

A notação de fórmula é a mesma, `(1 | grupo)` incluído. Para o método de estimação usado e as opções disponíveis, consulte a documentação de `MixedModels.jl` (Bates et al. 2026): são detalhes que mudam entre versões, e não vale reproduzi-los aqui de memória.

i Sua vez 9.6 — Um GLMM com hidroperíodo

Inclua `hydro` no `gml` e passe os efeitos fixos do novo modelo para `checar("9.6", ...)`. Compare a estimativa de `hydro` com a do 9.1.

Dica

Acrescente `+ hydro` antes do termo de efeito aleatório. `fixef(gm)` retorna os efeitos fixos.

Solução

```
gm_hidro = fit(MixedModel,
               @formula(richness ~ log_area + canopy +
                        ↪ hydro + (1 | region)),
               ponds, Poisson(); progress = false)
fixef(gm_hidro)
```

```
4-element Vector{Float64}:
 0.6004949810030696
 0.3694782152614212
-0.010471183105793884
-0.15990712087588418
```

```
checar("9.6", fixef(gm_hidro))
```

✓ Correto!

9.7 Lacunas que você vai encontrar

Este é o ponto em que ser honesto vale mais que ser entusiasta. A tabela abaixo é uma avaliação de **agosto de 2026**, feita consultando os registros públicos de pacotes; verifique antes de concluir que algo não existe.

| Precisa de | Em R | Em Julia hoje |
|--------------------------------|---------------------|--------------------------------------|
| Zero-inflação, hurdle | glmmTMB, pscl | sem equivalente maduro |
| Regressão beta | betareg, glmmTMB | sem equivalente maduro |
| Modelos aditivos (GAM) | mgcv | sem equivalente maduro |
| Resíduos quantílicos simulados | DHARMA | sem equivalente; implementável à mão |
| Médias marginais, contrastes | emmeans | Effects.jl, cobertura parcial |
| Regressão ordinal | ordinal, MASS::polr | cobertura parcial |

9.7.1 A saída pragmática: RCall.jl

Não há vergonha em chamar R de dentro de Julia quando o pacote certo só existe lá. O fluxo continua reprodutível, e os dados atravessam a fronteira automaticamente.

```
using RCall

@rput ponds          # envia o DataFrame para o R

R"""
```

```
library(mgcv)
m <- gam(richness ~ s(canopy) + s(log_area) + region,
        family = poisson, data = ponds)
summary(m)
"""
```

i Onde isso deixa você

Para o repertório coberto por este capítulo — Poisson, binomial negativa, binomial, gama, offsets, GLMMs — Julia cobre o necessário, e este capítulo o demonstrou com código executado. Fora dele, `RCall.jl` é a ponte. Reconhecer a fronteira com clareza é mais útil ao leitor do que escondê-la.

Desafios

1. Calcule a estatística de dispersão para g2. A binomial negativa resolveu a sobre-dispersão? Depois do 9.3, você ainda a escolheria?
2. Construa a curva de probabilidade prevista da Figura 9.1 para cada hidroperíodo em três temperaturas (22, 24,5 e 27 °C). Quanto a temperatura desloca a curva?
3. Implemente resíduos quantílicos ao estilo DHARMA para o g1: simule 250 conjuntos de dados a partir do modelo ajustado e, para cada lagoa, calcule a proporção de valores simulados abaixo do observado. Se o modelo estiver certo, essas proporções são uniformes. Elas são? Repita para o modelo do 9.3.

Parte IV

Programação para ecologia

10 Escrevendo suas próprias funções

Cedo ou tarde toda análise precisa de um cálculo que nenhum pacote oferece — um índice de um artigo, uma checagem de dados própria do seu protocolo de campo, um resumo que o seu laboratório sempre reporta. Este capítulo é sobre transformar esse cálculo em uma função fácil de chamar, difícil de usar errado, documentada e testada.

A Seção 4.3 apresentou a sintaxe. Aqui o foco é no que torna uma função boa: argumentos sensatos, erros claros, uma docstring e testes que avisam quando você a quebra.

i De onde vem este capítulo

Este capítulo e o Capítulo 11 são **inspirados** nos módulos de programação e de reamostragem do **ecoR** (Oliveira 2026), o wiki do curso de R do Instituto de Biociências da Universidade de São Paulo, produzido e mantido por Alexandre Adalardo de Oliveira. A ideia de construir índices ecológicos como funções, passo a passo, vem de lá. O texto, os exemplos e os exercícios daqui foram escritos do zero para Julia; os autores do ecoR não revisaram este livro.

Se quiser ir além, o material original está disponível gratuitamente (CC BY-SA 4.0):

- [9. Noções de Programação](#) — a apostila de programação;
- [9. Criando Funções Básicas](#) — o tutorial passo a passo;
- [exercícios de funções](#), corrigidos automaticamente no [notaR](#), em R.

```
using TidierData
using DataFrames, CSV
using Statistics
using Test

ponds = CSV.read(joinpath("../", "data", "ponds.csv"), DataFrame)
nrow(ponds)
```

180

i Exercícios deste capítulo

Como no Capítulo 4, a maioria das respostas aqui são funções que você escreve, e que checar chama com as próprias entradas de teste. Comece cada sessão com:

```
include("exercises/checar.jl")
using TidierData, DataFrames, CSV, Statistics, Test
ponds = CSV.read(joinpath("data", "ponds.csv"), DataFrame)
```

10.1 De um cálculo a uma função

Suponha que você queira saber quão variável é a riqueza entre as lagoas, em relação à média. O coeficiente de variação responde isso, e cabe em uma linha:

```
std(ponds.richness) / mean(ponds.richness) * 100
```

68.12190985250986

No momento em que você o quer para uma segunda variável, ou por região, a linha vira uma função. Dê a ela um nome que diga o que calcula, e ao argumento um nome que diga o que ela espera:

```
cv(x) = std(x) / mean(x) * 100

cv(ponds.richness), cv(ponds.body_size)
```

(68.12190985250986, 9.971208205402203)

A riqueza varia muito mais, em relação à média, do que o tamanho corporal.

Uma função pode retornar várias coisas ao mesmo tempo. A forma idiomática é uma **tupla nomeada**, cujos campos você lê pelo nome:

```
resumo_contagem(x) = (n = length(x), media = mean(x), var = var(x))

r = resume_contagem(ponds.richness)
r.media, r.var
```

(7.133333333333334, 23.613407821229057)

i Sua vez 10.1 — Razão variância/média

Para contagens, a razão variância/média diz como os indivíduos — ou as espécies — se distribuem entre as unidades amostrais. Se as contagens seguem uma Poisson, a variância é igual à média e a razão fica perto de 1; acima de 1 as contagens são mais agregadas, abaixo de 1 mais regulares. Escreva `vmr(x)` e verifique com `checar("10.1", vmr)`.

Dica

Mesma forma que cv: uma linha, var sobre mean. var é a variância amostral, que divide por $n - 1$.

Solução

```
vmr(x) = var(x) / mean(x)

vmr(ponds.richness)
```

3.310290816060148

```
checar("10.1", vmr)
```

✓ Correto!

A riqueza das 180 lagoas tem razão de cerca de 3,3: muito mais variável que uma Poisson com média única. Isso ainda não quer dizer que o processo de contagem seja estranho — as lagoas diferem em área, dossel e região. A Seção 10.5 volta a isso.

10.2 Argumentos: valores padrão e palavras-chave

Uma função muitas vezes tem um padrão sensato que o usuário pode querer mudar. cv retorna uma porcentagem; alguém pode preferir uma proporção. Há duas formas de oferecer isso.

Um **argumento posicional com valor padrão** vem depois dos outros e é definido pela posição:

```
cv_pos(x, percentual = true) = std(x) / mean(x) * (percentual ? 100 :
  ↪ 1)

cv_pos(ponds.richness), cv_pos(ponds.richness, false)
```

(68.12190985250986, 0.6812190985250985)

Um **argumento nomeado** (*keyword*) vem depois de um ponto e vírgula e é sempre definido pelo nome:

```
cv_kw(x; percentual = true) = std(x) / mean(x) * (percentual ? 100 : 1)

cv_kw(ponds.richness; percentual = false)
```

0.6812190985250985

Para opções como esta, argumentos nomeados são a melhor escolha: `cv_kw(x; percentual = false)` diz o que faz, enquanto `cv_pos(x, false)` obriga o leitor a procurar o que `false` significa.

Uma função também pode recolher argumentos nomeados que não conhece e repassá-los com `kwargs....`. Aqui `cv_arredondado` trata `digits` e entrega todo o resto para `cv_kw`:

```
cv_arredondado(x; digits = 1, kwargs...) = round(cv_kw(x; kwargs...);
  ↪ digits)

cv_arredondado(ponds.richness), cv_arredondado(ponds.richness;
  ↪ percentual = false, digits = 3)
```

(68.1, 0.681)

Em R

R tem um único tipo de argumento, que pode ser passado por posição ou por nome. Julia separa os dois: argumentos antes do `;` são posicionais, os depois dele são nomeados e precisam do nome. `kwargs...` cumpre o papel do `...` do R ao repassar opções para outra função.

i Sua vez 10.2 — Padronizando, com opções

Escreva `standardize(x; center = true, scale = true)`. Por padrão ela retorna $(x - \bar{x})/s$; `center = false` deixa de subtrair a média e `scale = false` deixa de dividir pelo desvio-padrão. Verifique com `checar("10.2", standardize)`.

Dica

Monte o resultado em dois passos, cada um dependendo da sua opção: `centrado = center ? x .- mean(x) : x`, depois divida `centrado` por `std(x)` só se `scale` for verdadeiro.

Solução

```
function standardize(x; center = true, scale = true)
  centrado = center ? x .- mean(x) : x
  return scale ? centrado ./ std(x) : centrado
end

checar("10.2", standardize)
```

✓ Correto!

`std(x)` é calculado sobre o `x` original, então é o mesmo com ou sem centralizar.

10.3 Recusando entradas ruins

Uma função que retorna bobagem em silêncio é pior que uma que para. `cv` tem dois pontos cegos:

```
cv([0, 0, 0]), cv([-5, 5])
```

(NaN, Inf)

NaN para um levantamento vazio, e um coeficiente infinito para dados com um valor negativo, que contagens não podem ter. Checagens no início da função pegam os dois casos, e `throw(ArgumentError(...))` diz a quem chamou que o problema está na entrada, não na função:

```
function cv_checked(x)
    any(<(0), x) && throw(ArgumentError("os valores devem ser não
    ↪ negativos"))
    mean(x) > 0 || throw(ArgumentError("a média é zero; o CV não está
    ↪ definido"))
    return std(x) / mean(x) * 100
end

try
    cv_checked([-5, 5])
catch e
    e
end
```

`ArgumentError("os valores devem ser não negativos")`

A forma condição `&&` ação se lê “se a condição, então a ação”; condição `||` ação, “a menos que a condição, a ação”. As duas são idiomáticas para checagens como estas.

💡 @assert é para os seus próprios erros

`@assert` condição para quando algo que nunca deveria acontecer acontece. Use-o para conferir o seu próprio raciocínio dentro de uma função; use `ArgumentError` para entradas que um usuário poderia plausivelmente errar.

i Sua vez 10.3 — Um `vmr` mais seguro

Reescreva `vmr` para que lance um `ArgumentError` quando receber menos de dois valores, algum valor negativo ou média zero, e continue retornando a razão nos outros casos. Verifique com `checar("10.3", vmr)`.

Dica

Três checagens, uma por linha, antes do cálculo. As condições são `length(x) < 2`, `any(<(0), x)` e `mean(x) == 0`.

Solução

```
function vmr(x)
  length(x) < 2 && throw(ArgumentError("são necessários
  ↪ pelo menos dois valores"))
  any(<(0), x) && throw(ArgumentError("contagens não
  ↪ podem ser negativas"))
  mean(x) == 0 && throw(ArgumentError("a média é zero;
  ↪ a razão não está definida"))
  return var(x) / mean(x)
end

checar("10.3", vmr)
```

✓ Correto!

Isso substitui o `vmr` de uma linha do 10.1: um método com os mesmos tipos de argumento sobrescreve o antigo.

10.4 Documentando uma função

Um texto colocado imediatamente antes de uma função vira a sua **docstring**, que é o que `?cv_documentado` mostra no REPL e o que o VS Code mostra quando você passa o mouse sobre o nome. A convenção é a assinatura com recuo de quatro espaços, depois uma ou duas frases sobre o que a função faz e retorna:

```
"""
    cv_documentado(x; percentual = true)

Coeficiente de variação de `x`: o desvio-padrão dividido pela média,
↪ vezes 100 a menos
que `percentual = false`. Lança um `ArgumentError` se a média for zero.
"""
function cv_documentado(x; percentual = true)
  mean(x) > 0 || throw(ArgumentError("a média é zero; o CV não está
  ↪ definido"))
  return std(x) / mean(x) * (percentual ? 100 : 1)
end
```

Main.Notebook.cv_documentado

Em R

Docstrings cumprem o papel dos comentários do roxygen2, com uma diferença: fazem parte da linguagem e ficam disponíveis assim que a função é definida, sem etapa de compilação da documentação.

i Sua vez 10.4 — Documente o vmr

Adicione uma docstring ao seu vmr do 10.3: a assinatura, o que a razão significa e quais entradas ela recusa. Rode a definição de novo e experimente ?vmr no REPL. Depois verifique com `checar("10.4", vmr)`.

Dica

A docstring vai logo acima de `function vmr(x)`, sem nada entre as duas — nem mesmo uma linha em branco.

Solução

```
"""
    vmr(x)

    Razão variância/média das contagens `x`. Perto de 1 para
    ↪ contagens Poisson, acima de 1
    quando as contagens são agregadas, abaixo de 1 quando são
    ↪ mais regulares. Lança um
    `ArgumentError` para menos de dois valores, valores
    ↪ negativos ou média zero.
"""
function vmr(x)
    length(x) < 2 && throw(ArgumentError("são necessários
    ↪ pelo menos dois valores"))
    any(<(0), x) && throw(ArgumentError("contagens não
    ↪ podem ser negativas"))
    mean(x) == 0 && throw(ArgumentError("a média é zero;
    ↪ a razão não está definida"))
    return var(x) / mean(x)
end

checar("10.4", vmr)
```

✓ Correto!

10.5 A sua função, por grupo

Uma função sua funciona dentro de um pipeline do Tidier — com um porém. O Tidier sabe que `mean`, `std` ou `sum` resumem uma coluna inteira, mas não sabe disso sobre o seu `cv`. Por padrão, ele aplica funções desconhecidas **a cada valor separadamente**, o que aqui calcularia um coeficiente de variação para cada lagoa:

```
errado = @chain ponds begin
    @group_by(region)
    @summarize(cv_richness = cv(richness))
end
```

```
nrow(errado), first(errado.cv_richness, 3)
```

(180, [NaN, NaN, NaN])

180 linhas de NaN em vez de três números, e nenhum erro. Um ~ na frente da função diz ao Tidier para passar a coluna inteira:

```
@chain ponds begin
  @group_by(region)
  @summarize(cv_richness = ~cv(richness), cv_body = ~cv(body_size))
end
```

| | region | cv_richness | cv_body |
|---|----------|-------------|---------|
| | String15 | Float64 | Float64 |
| 1 | Atlantic | 56.2043 | 8.88778 |
| 2 | Amazon | 53.2696 | 11.3731 |
| 3 | Cerrado | 60.8118 | 9.92402 |

O mesmo com o `DataFrames.jl` diretamente (Seção 5.7), onde a questão não aparece:

```
combine(groupby(ponds, :region), :richness => cv => :cv_richness)
```

| | region | cv_richness |
|---|----------|-------------|
| | String15 | Float64 |
| 1 | Atlantic | 56.2043 |
| 2 | Amazon | 53.2696 |
| 3 | Cerrado | 60.8118 |

⚠ Confira o número de linhas

Depois do `@summarize`, você deve ter uma linha por grupo. Se tiver uma linha por observação, o Tidier aplicou a sua função a cada valor: acrescente o ~.

i Sua vez 10.5 — A riqueza é agregada dentro das regiões?

Calcule a razão variância/média de richness para cada região, em uma coluna chamada `vmr`, usando a sua função. Verifique com `checar("10.5", por_regiao)`. As razões ficam mais perto de 1 do que o valor de todas as lagoas juntas?

Dica

O bloco com `~cv` desta seção, com `~vmr` no lugar de `~cv`. Sem o ~, o seu `vmr` do 10.3 recebe uma lagoa por vez e, com razão, a recusa: “são necessários pelo menos dois valores”.

Solução

```

por_regiao = @chain ponds begin
  @group_by(region)
  @summarize(vmr = ~vmr(richness))
end

checar("10.5", por_regiao)

```

✓ Correto!

As razões, de cerca de 1,8 a 3,2, são todas menores que a das lagoas juntas (3,3): parte da variação vinha das diferenças entre regiões. Elas continuam acima de 1, porque lagoas da mesma região diferem em área e dossel. Na Seção 9.3, um modelo Poisson que levava esses preditores em conta não tinha mais sobredispersão. Uma razão acima de 1 em contagens brutas é motivo para procurar preditores, não prova de que as contagens não sejam Poisson.

10.6 Funções que recebem funções

Em Julia, funções são valores: podem ser guardadas em um vetor, passadas para outra função ou escritas sem nome. Isso facilita aplicar vários resumos de uma vez:

```
[f(ponds.richness) for f in (mean, median, cv, vmr)]
```

```

4-element Vector{Float64}:
 7.1333333333333334
 6.0
68.12190985250986
 3.310290816060148

```

Uma **função anônima** se escreve `x -> expressão`. `map` aplica uma função a cada elemento de uma coleção — aqui, a três colunas dos dados:

```

map(col -> round(cv(col); digits = 1), eachcol(ponds[:, [:area_m2,
↪ :temp_c, :body_size]]))

```

```

3-element Vector{Float64}:
117.6
 11.6
 10.0

```

A área varia enormemente entre lagoas; a temperatura, muito pouco. O coeficiente de variação põe variáveis com unidades diferentes na mesma escala.

i Sua vez 10.6 — O índice de Morisita

O índice de dispersão de Morisita compara a frequência com que dois indivíduos sorteados caem na mesma unidade amostral com o que uma distribuição aleatória daria:

$$I_d = n \frac{\sum_i x_i(x_i - 1)}{N(N - 1)}$$

onde x_i é a contagem na unidade i , n o número de unidades e $N = \sum_i x_i$. Ele fica perto de 1 para uma distribuição aleatória e acima de 1 para uma agregada. Escreva `morisita(x)`, verifique com `checar("10.6", morisita)` e aplique-o a estas contagens de agrupamentos de girinos em dez quadrats ao longo da margem de uma lagoa:

```
quadrats = [0, 3, 0, 0, 7, 1, 0, 0, 5, 0]
```

```
10-element Vector{Int64}:
```

```
0
3
0
0
7
1
0
0
5
0
```

Dica

`x .* (x .- 1)` dá $x_i(x_i - 1)$ para todas as unidades de uma vez; `sum` soma tudo.

Solução

```
function morisita(x)
    N = sum(x)
    return length(x) * sum(x .* (x .- 1)) / (N * (N - 1))
end

morisita(quadrats)
```

```
2.8333333333333335
```

```
checar("10.6", morisita)
```

✓ Correto!

Cerca de 2,8: os agrupamentos estão fortemente agregados, em três dos dez quadrats. Uma versão para uso real recusaria $N < 2$, como no 10.3.

10.7 Testando suas funções

checar vem testando as suas funções com entradas de resposta conhecida. Você pode fazer o mesmo com as suas próprias funções usando a biblioteca padrão Test, e rodar os testes de novo toda vez que mudar o código:

```
@testset "cv_checked" begin
  @test cv_checked([10, 10, 10]) == 0          # sem variação
  @test cv_checked([1, 2, 3]) ≈ 50             # desvio 1, média 2
  @test_throws ArgumentError cv_checked([0, 0])
  @test_throws ArgumentError cv_checked([-1, 1])
end;
```

```
Test Summary: | Pass  Total  Time
cv_checked    |    4      4  1.0s
```

Bons testes usam casos que você consegue calcular à mão e incluem as entradas que a função deve recusar. Digite $\approx$ como `\approx` seguido de Tab; ele compara números de ponto flutuante com uma tolerância, onde `==` falharia por arredondamento.

💡 Onde suas funções devem morar

Quando uma função passa a ser útil além de um script, mova-a para um arquivo próprio — digamos `funcoes.jl` na pasta do projeto — e carregue-a com `include("funcoes.jl")` no início de cada análise. Os testes podem morar em um segundo arquivo ao lado. Quando a coleção crescer, o próximo passo é um pacote, que o Pkg cria para você com `Pkg.generate`.

Desafios

1. Escreva um `@testset` para o seu `morisita` com casos que você consegue calcular à mão: todos os indivíduos em uma unidade, e exatamente um indivíduo por unidade. O que cada um deve dar?
2. Adicione a `vmr` um método para `Matrix` (Seção 4.4) que retorne uma razão por coluna, tratando cada coluna como uma espécie contada ao longo dos quadrats (linhas).
3. `cv` falha em uma coluna com valores faltantes, como `canopy`. Adicione um argumento nomeado `skipmissing = false` que, quando verdadeiro, os ignore. Qual padrão é mais seguro, e por quê?

11 Reamostragem e simulação

Todos os testes do Capítulo 8 e do Capítulo 9 se apoiam em uma distribuição teórica: a t , a normal, a χ^2 . Este capítulo constrói essas distribuições a partir dos próprios dados, embaralhando e reamostrando, e depois inverte a ideia: simula dados de um processo conhecido para perguntar com que frequência um estudo de certo tamanho encontraria um efeito.

Julia combina bem com isso. Reamostrar significa rodar o mesmo cálculo milhares de vezes, e em Julia o loop simples que expressa a ideia também é rápido (Seção 4.7); não é preciso contorcer o código para evitá-lo.

i De onde vem este capítulo

Como o Capítulo 10, este capítulo é **inspirado** no **ecoR** (Oliveira 2026), o wiki do curso de R do Instituto de Biociências da Universidade de São Paulo, produzido e mantido por Alexandre Adalardo de Oliveira. A sequência — permutação, depois bootstrap, depois simulação a partir de distribuições conhecidas, com um modelo nulo de coocorrência de espécies — segue o seu módulo de reamostragem. O texto, os exemplos e os exercícios daqui foram escritos do zero para Julia; os autores do ecoR não revisaram este livro.

Se quiser ir além, o material original está disponível gratuitamente (CC BY-SA 4.0):

- [8. Reamostragem e Simulação](#) — a apostila;
- [8. Reamostragem e Simulação](#) — o tutorial passo a passo;
- [exercícios de reamostragem](#), corrigidos automaticamente no [notaR](#), em R;
- [Simulando Dados](#), um curso curto sobre simular dados para modelos lineares, mistos e lineares generalizados.

```
using TidierData
using DataFrames, CSV
using Statistics, StatsBase, Random, Distributions
using GLM, HypothesisTests
using CairoMakie

CairoMakie.activate!(type = "png")
Random.seed!(2026)

ponds = @chain CSV.read(joinpath("../", "data", "ponds.csv"), DataFrame)
  ↪ begin
    @drop_missing(canopy)
    @mutate(log_area = log(area_m2))
  end
```

```
nrow(ponds)
```

173

i Exercícios deste capítulo

A maioria das respostas aqui vem de sorteios aleatórios; por isso a sua não vai bater com a da solução até a última casa, e checar aceita qualquer valor dentro da variação aleatória esperada. Use pelo menos tantas permutações ou simulações quanto o exercício pede: com poucas, a resposta fica ruidosa demais para passar. Comece cada sessão com:

```
include("exercises/checar.jl")
using TidierData, DataFrames, CSV, Statistics, StatsBase, Random,
    ↪ Distributions
using GLM, HypothesisTests
ponds = @chain CSV.read(joinpath("data", "ponds.csv"), DataFrame)
    ↪ begin
    @drop_missing(canopy)
    @mutate(log_area = log(area_m2))
end
```

11.1 Aleatoriedade que se reproduz

Três funções fazem quase todo o trabalho deste capítulo. `shuffle` reordena um vetor; `sample`, do `StatsBase.jl`, sorteia elementos dele com ou sem reposição:

```
x = [1, 2, 3, 4, 5]

(embaralhado = shuffle(x),
 sem_reposicao = sample(x, 3; replace = false),
 com_reposicao = sample(x, 5))           # replace = true é o padrão
```

(`embaralhado` = [1, 3, 4, 2, 5], `sem_reposicao` = [2, 4, 5], `com_reposicao` = [2, 5, 2, 5, 4])

Os números aleatórios em Julia vêm de um **gerador de números aleatórios** (RNG). Fixar a semente, como o bloco inicial faz com `Random.seed!(2026)`, faz um script produzir os mesmos sorteios toda vez que roda. Para ter mais controle, crie o seu próprio gerador e passe-o como primeiro argumento:

```
rng = Xoshiro(42)
sample(rng, x, 3)
```

```
3-element Vector{Int64}:  
 4  
 3  
 3
```

⚠ Mesma semente, outro Julia, outros números

O manual do Julia avisa que uma versão *minor* nova (da 1.11 para a 1.12, por exemplo) *pode mudar* a sequência de números aleatórios gerada a partir de uma semente. A semente torna a sua análise reprodutível na sua instalação, não em todas as versões. Para reprodutibilidade exata, salve os dados aleatórios, ou fixe a versão do Julia e o `Manifest.toml`. É também por isso que os exercícios daqui aceitam uma faixa de respostas.

11.2 Testes de permutação

Na Seção 8.1, um teste *t* de Welch perguntou se o tamanho corporal difere entre lagoas permanentes e temporárias. Um teste de permutação faz a mesma pergunta sem supor nenhuma distribuição, em cinco passos:

1. Escolha uma estatística que meça o efeito: aqui, a diferença no tamanho corporal médio entre lagoas permanentes e temporárias.
2. Calcule-a para os dados reais.
3. Escreva a hipótese nula como um embaralhamento: se o hidroperíodo não tem efeito, os rótulos “permanent” e “temporary” são intercambiáveis.
4. Embaralhe os rótulos muitas vezes, recalculando a estatística a cada vez. Isso constrói a distribuição da estatística *sob a hipótese nula*.
5. O valor-p é a proporção de embaralhamentos com estatística pelo menos tão extrema quanto a observada.

```
eh_perm = ponds.hydro .== "permanent"  
dif_medias(y, g) = mean(y[g]) - mean(y[.!g])  
  
observado = dif_medias(ponds.body_size, eh_perm)  
  
B = 9_999  
difs_nulas = zeros(B)  
rotulos = copy(eh_perm)  
for b in 1:B  
    shuffle!(rotulos)                # passos 3–4: embaralha,  
    ↪ recalcula  
    difs_nulas[b] = dif_medias(ponds.body_size, rotulos)  
end  
  
extremos = count(abs.(difs_nulas) .>= abs(observado))  
p_perm = (extremos + 1) / (B + 1)  
  
(observado = observado, p_perm = p_perm)
```

```
(observado = -0.8929446693657184, p_perm = 0.1073)
```

Dois detalhes. `abs` torna o teste bilateral: uma diferença de $-0,9$ conta como tão extrema quanto uma de $+0,9$. E o $+1$ no numerador e no denominador conta os dados observados como um dos arranjos possíveis, o que impede que o valor-p seja exatamente zero (Phipson e Smyth 2010).

```
fig = Figure(size = (700, 380))
ax = Axis(fig[1, 1], xlabel = "Diferença no tamanho corporal médio
    ↳ (mm), permanente - temporária",
    ylabel = "Embaralhamentos")
hist!(ax, difs_nulas, bins = 50, color = (:grey60, 0.8))
vlines!(ax, [observado, -observado], color = :darkorange, linestyle =
    ↳ :dash, linewidth = 2)
fig
```

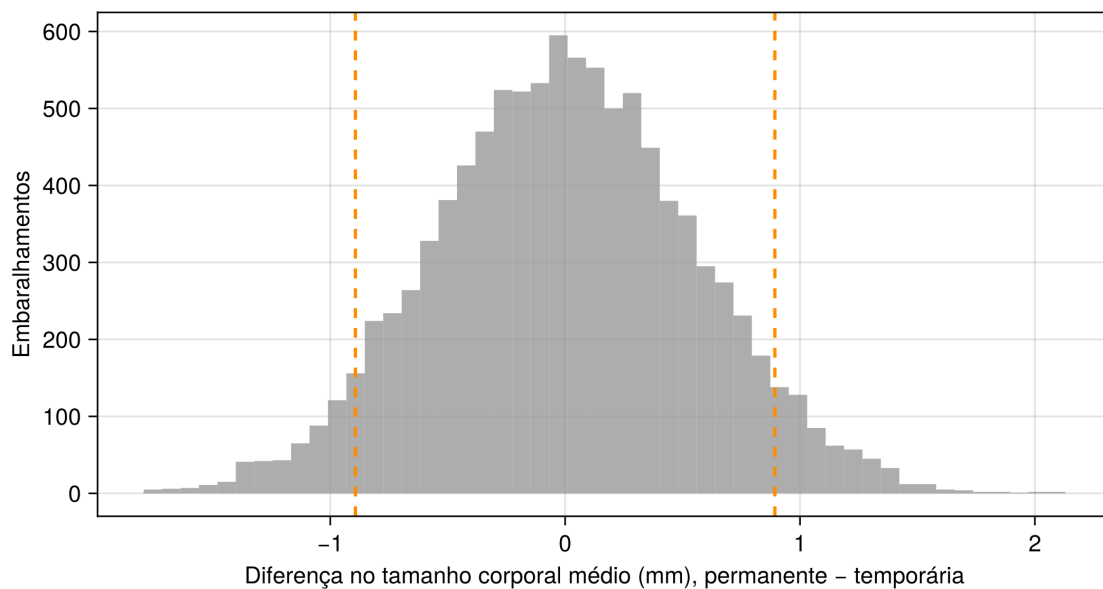


Figura 11.1: Distribuição nula da diferença no tamanho corporal médio, a partir de 9 999 embaralhamentos dos rótulos de hidroperíodo. As linhas tracejadas marcam a diferença observada e o seu espelho.

O valor-p, cerca de 0,10, fica perto do 0,103 do teste de Welch: com 173 lagoas e dados aproximadamente simétricos, a teoria e o embaralhamento concordam. Isso nem sempre vale com amostras pequenas ou dados assimétricos, e é aí que a permutação mostra seu valor. O `HypothesisTests.jl` empacota a mesma ideia:

```
lagoas_perm = ponds.body_size[eh_perm]
lagoas_temp = ponds.body_size[.!eh_perm]
pvalue(ApproximatePermutationTest(lagoas_perm, lagoas_temp, mean,
    ↳ 9_999))
```

0.10461046104610461

i Sua vez 11.1 — Dossel e a espécie-foco

A cobertura de dossel é diferente nas lagoas onde a espécie-foco ocorre? Faça um teste de permutação bilateral para a diferença na média de canopy entre lagoas com `focal == 1` e `focal == 0`, com 9 999 embaralhamentos, e passe o valor-p para `checar("11.1", ...)`.

Dica

Reaproveite `dif_medias` e o loop acima, com `ponds.canopy` como resposta e `ponds.focal == 1` como grupos.

Solução

```
tem_focal = ponds.focal == 1
obs_dossel = dif_medias(ponds.canopy, tem_focal)

grupos = copy(tem_focal)
nulo_dossel = [dif_medias(ponds.canopy, shuffle!(grupos))
  ↪ for _ in 1:9_999]
p_dossel = (count(abs.(nulo_dossel) .>= abs(obs_dossel))
  ↪ + 1) / (9_999 + 1)
```

0.0353

```
checar("11.1", p_dossel)
```

✓ Correto!

Cerca de 0,04: lagoas com a espécie-foco têm cerca de 7 pontos a mais de dossel. A simulação pôs exatamente isso: a ocorrência aumenta com o dossel (`scripts/make_data.jl`). A compreensão faz o mesmo que o loop, em uma linha.

11.3 Permutando uma regressão

A mesma lógica testa uma inclinação. Se o tamanho corporal não depende do dossel, qualquer pareamento de tamanhos corporais com valores de dossel é tão provável quanto o observado; então embaralhar a resposta quebra a relação e mantém intactas as duas distribuições:

```
inclinacao(x, y) = cov(x, y) / var(x)
```

```

obs_inclinacao = inclinacao(ponds.canopy, ponds.body_size)

y = copy(ponds.body_size)
nulo_inclinacao = [inclinacao(ponds.canopy, shuffle!(y)) for _ in
    ↪ 1:9_999]
extremos = count(abs.(nulo_inclinacao) .>= abs(obs_inclinacao))

(inclinacao = obs_inclinacao, extremos = extremos, p = (extremos + 1)
    ↪ / (9_999 + 1))

```

```
(inclinacao = -0.04966562886720383, extremos = 0, p = 0.0001)
```

$\text{cov}(x, y) / \text{var}(x)$ é a inclinação de mínimos quadrados, o mesmo número que o `lm` estima na Seção 8.2. Quase nenhum embaralhamento produz uma inclinação tão acentuada quanto a observada — a contagem está em extremos —, então o valor-p fica no menor valor que 9 999 embaralhamentos conseguem mostrar, 1/10 000, ou perto dele. Para dizer mais, seriam necessários mais embaralhamentos, mas “cerca de 1 em 10 000 ou menos” costuma ser tudo o que um resultado precisa.

i Sua vez 11.2 — Tamanho corporal e área da lagoa

Teste por permutação se o tamanho corporal está relacionado à área da lagoa: use a inclinação de `body_size` em `log_area`, bilateral, com 9 999 embaralhamentos. Passe o valor-p para checar("11.2", ...).

Dica

O bloco acima com `ponds.log_area` no lugar de `ponds.canopy`. Embaralhe uma cópia de `body_size`, não a própria coluna, para manter os dados intactos.

Solução

```

obs_area = inclinacao(ponds.log_area, ponds.body_size)
y = copy(ponds.body_size)
nulo_area = [inclinacao(ponds.log_area, shuffle!(y)) for
    ↪ _ in 1:9_999]
p_area = (count(abs.(nulo_area) .>= abs(obs_area)) + 1)
    ↪ / (9_999 + 1)

```

```
0.2583
```

```
checar("11.2", p_area)
```

✓ Correto!

Cerca de 0,26. A simulação não tem efeito da área sobre o tamanho corporal, e o teste não encontra nenhum — corretamente, desta vez. Um teste em cada vinte encontraria um mesmo assim.

11.4 O bootstrap

Um teste de permutação pergunta como os dados seriam se não houvesse efeito. O **bootstrap** (Efron 1979) faz outra pergunta: quanto uma estimativa mudaria se tivéssemos coletado outra amostra? Não podemos coletá-la, mas podemos imitá-la sorteando uma amostra nova do mesmo tamanho *a partir dos próprios dados, com reposição*. Algumas lagoas aparecem duas vezes, outras nenhuma, e a dispersão da estimativa ao longo de muitas dessas amostras mede a sua incerteza.

```
n = nrow(ponds)
medianas_boot = [median(sample(ponds.body_size, n)) for _ in 1:9_999]

(mediana = median(ponds.body_size),
 ic_95   = quantile(medianas_boot, [0.025, 0.975]))
```

```
(mediana = 35.56, ic_95 = [35.08, 36.54])
```

Os quantis de 2,5% e 97,5% das medianas bootstrap dão um **intervalo percentil** de 95% — para uma estatística, a mediana, que não tem uma fórmula simples para o seu erro-padrão.

O bootstrap também confere a incerteza que o próprio modelo reporta. Reamostre lagoas inteiras (linhas), reajuste o modelo Poisson g1 da Seção 9.2 a cada vez e compare a dispersão do coeficiente de dossel com o erro-padrão que o modelo informa:

```
g1 = glm(@formula(richness ~ log_area + canopy + temp_c), ponds,
  ↪ Poisson(), LogLink())

dossel_boot = map(1:2_000) do _
  linhas = sample(1:n, n)
  coef(glm(@formula(richness ~ log_area + canopy + temp_c),
    ponds[linhas, :], Poisson(), LogLink()))[3]
end

(ep_bootstrap = std(dossel_boot),
 ep_modelo    = stderror(g1)[3],
 razao        = std(dossel_boot) / stderror(g1)[3])
```

```
(ep_bootstrap = 0.0018378945669366084, ep_modelo = 0.0013361976263705473, razao = 1.375466)
```

O erro-padrão bootstrap é cerca de 1,4 vez maior que o que o modelo Poisson informa — perto de $\sqrt{1,95}$, a raiz da estatística de dispersão de g1 na Seção 9.3. Os erros-padrão do modelo supõem que a variância é igual à média; o bootstrap não supõe nada sobre ela, e enxerga a variação extra que o modelo deixa de fora.

Blocos do

`map(1:2_000) do _ ... end` é `map(_ -> ..., 1:2_000)` escrito como bloco, para funções anônimas com mais de uma linha. `_` nomeia um argumento que você não

usa.

i Sua vez 11.3 — Uma função de bootstrap

Escreva `boot_means(x, B)`, que retorna um vetor com as médias de `B` amostras bootstrap de `x`. Verifique com `checar("11.3", boot_means)`.

Dica

Uma compreensão: `[mean(sample(x, length(x))) for _ in 1:B]`. Cada amostra bootstrap tem o mesmo tamanho de `x` e é sorteada com reposição.

Solução

```
boot_means(x, B) = [mean(sample(x, length(x))) for _ in
    ↪ 1:B]

checar("11.3", boot_means)
```

✓ Correto!

i Sua vez 11.4 — Quão ricas são as lagoas amazônicas?

Usando `boot_means` com 9 999 amostras, calcule um intervalo percentil de 95% para a riqueza média das lagoas da Amazônia. Passe-o como `[inferior, superior]` para `checar("11.4", ...)`. Compare-o com o intervalo t usual.

Dica

`ponds.richness[ponds.region == "Amazon"]` seleciona as lagoas da Amazônia. `quantile(v, [0.025, 0.975])` retorna os dois extremos de uma vez.

Solução

```
amazonia = ponds.richness[ponds.region == "Amazon"]
ic_boot = quantile(boot_means(amazonia, 9_999), [0.025,
    ↪ 0.975])
```

```
2-element Vector{Float64}:
 9.478260869565217
12.956521739130435
```

```
checar("11.4", ic_boot)
confint(OneSampleTTest(amazonia))
```

✓ Correto!

(9.384117221613428, 12.963708865343095)

Os dois intervalos quase coincidem, de cerca de 9,4–9,5 a 13. O bootstrap começa um pouco mais alto: a riqueza é uma contagem assimétrica, e o bootstrap acompanha a assimetria, enquanto o intervalo t é simétrico por construção.

11.5 Quantas lagoas eu preciso?

A reamostragem reaproveita os dados que você tem. A simulação cria dados que você ainda não tem, a partir de um processo que você especifica — e, como foi você que escolheu o processo, sabe a resposta certa. Essa é a lógica de todo este livro (Seção 5.1), e ela responde a uma pergunta que todo projeto de campo enfrenta antes de começar: **quantas unidades amostrais bastam?**

Suponha que lagoas permanentes tenham cerca de 25% mais espécies que as temporárias, como na simulação por trás de `ponds.csv` ($e^{0,22} \approx 1,25$). Esta função gera um levantamento de n lagoas a partir desse processo:

```
function simulate_ponds(n; p_temporary = 0.5, rng =
  ↪ Random.default_rng())
  log_area = rand(rng, Normal(5.0, 0.9), n)
  hydro    = ifelse.(rand(rng, n) .< p_temporary, "temporary",
  ↪ "permanent")
  mu       = @. exp(0.9 + 0.28 * log_area + 0.22 * (hydro ==
  ↪ "permanent"))
  richness = rand.(rng, Poisson.(mu))
  return DataFrame(; log_area, hydro, richness)
end

first(simulate_ponds(5), 5)
```

| | log_area | hydro | richness |
|---|----------|-----------|----------|
| | Float64 | String | Int64 |
| 1 | 4.34568 | temporary | 13 |
| 2 | 6.30406 | permanent | 18 |
| 3 | 5.49257 | temporary | 12 |
| 4 | 4.17274 | permanent | 7 |
| 5 | 4.56875 | permanent | 9 |

Para um levantamento de n lagoas, ajuste o modelo e registre se ele detecta o efeito do hidroperíodo ao nível de 5%. A proporção de levantamentos simulados que detectam é o **poder** desse desenho:

```
function detects_hydro(n; kwargs...)
  d = simulate_ponds(n; kwargs...)
  m = glm(@formula(richness ~ log_area + hydro), d, Poisson(),
  ↪ LogLink())
```

```

    return coefstable(m).cols[4][3] < 0.05      # valor-p do
    ↪ coeficiente de hydro
end

tamanhos = [20, 40, 60, 80, 100, 140, 180]
poder = [mean(detects_hydro(n) for _ in 1:500) for n in tamanhos]

DataFrame(n = tamanhos, poder = poder)

```

| | n | poder |
|---|-------|---------|
| | Int64 | Float64 |
| 1 | 20 | 0.35 |
| 2 | 40 | 0.616 |
| 3 | 60 | 0.796 |
| 4 | 80 | 0.886 |
| 5 | 100 | 0.954 |
| 6 | 140 | 0.992 |
| 7 | 180 | 1.0 |

```

fig = Figure(size = (700, 380))
ax = Axis(fig[1, 1], xlabel = "Número de lagoas", ylabel = "Poder")
lines!(ax, tamanhos, poder, color = :steelblue, linewidth = 2.5)
scatter!(ax, tamanhos, poder, color = :steelblue)
hlines!(ax, 0.8, color = :grey40, linestyle = :dash)
ylims!(ax, 0, 1)
fig

```

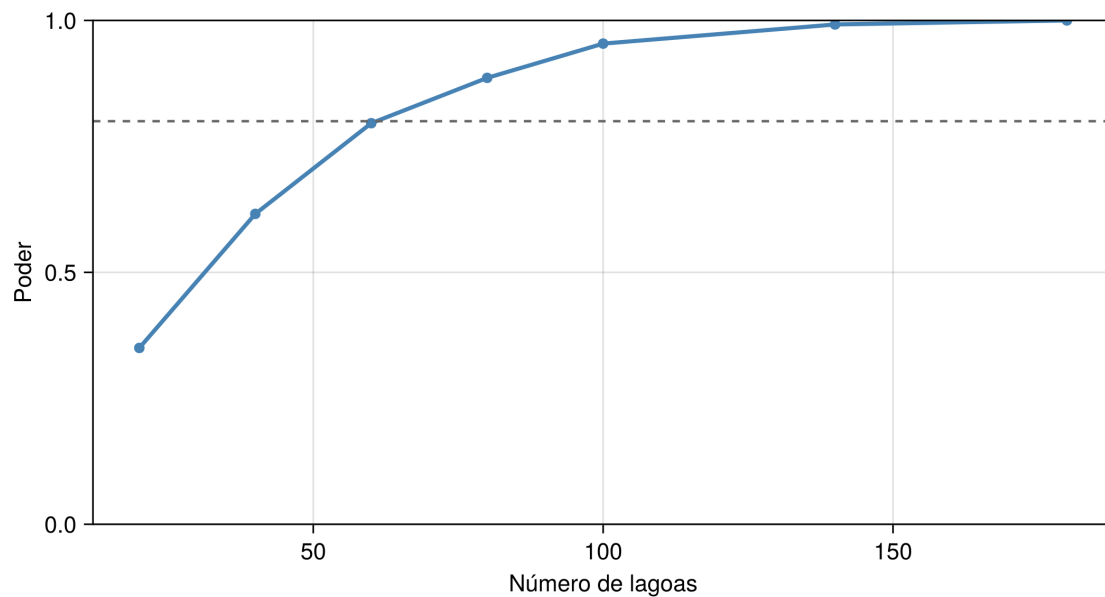


Figura 11.2: Poder para detectar uma diferença de 25% na riqueza entre lagoas permanentes e temporárias, a partir de 500 levantamentos simulados por tamanho amostral. A linha tracejada marca a meta convencional de 80%.

Cerca de 60 lagoas atingem os 80% de poder usuais; com 20, dois levantamentos em cada três deixariam de ver um efeito que existe de verdade. E um resultado não significativo de um levantamento pequeno diz pouco: a maioria deles é não significativa exista ou não o efeito.

i Sua vez 11.5 — Quando o desenho é desbalanceado

No campo, raramente se consegue o mesmo número de cada tipo de lagoa. Estime o poder com 60 lagoas quando só 20% são temporárias (`p_temporary = 0.2`), a partir de pelo menos 1 000 levantamentos simulados, e passe-o para `checar("11.5", ...)`.

Dica

`detects_hydro` repassa seus argumentos nomeados para `simulate_ponds` (Seção 10.2), então `detects_hydro(60; p_temporary = 0.2)` já faz o que você precisa.

Solução

```
poder_desbalanceado = mean(detects_hydro(60; p_temporary
  ↪ = 0.2) for _ in 1:2_000)
```

0.5885

```
checar("11.5", poder_desbalanceado)
```

✓ Correto!

Cerca de 0,60, abaixo dos 0,80 do desenho balanceado: as mesmas 60 lagoas dizem menos, porque a comparação se apoia em só umas 12 lagoas temporárias.

11.6 Um modelo nulo para coocorrência de espécies

A permutação também é a base dos **modelos nulos** em ecologia de comunidades. A pergunta aqui é se as espécies coocorrem menos do que o esperado ao acaso, o que pode indicar competição ou exigências de hábitat diferentes. Os dados são uma matriz de presença e ausência, com uma linha por espécie e uma coluna por local.

O `ponds.csv` não tem listas de espécies; por isso esta seção usa uma pequena matriz simulada para o propósito: seis espécies em doze lagoas.

```
M = [1 1 1 0 1 1 0 0 0 0 0 1;  
      1 0 1 1 1 1 0 1 0 0 0 0;  
      0 1 1 1 1 1 0 0 0 1 0 0;  
      1 0 0 0 0 0 1 1 1 0 1 1;  
      0 0 1 0 0 0 1 1 0 1 1 1;  
      0 1 0 0 0 0 1 0 1 1 1 1]  
  
(ocorrencias_especie = vec(sum(M, dims = 2)),  
 riqueza_lagoa       = vec(sum(M, dims = 1)))
```

```
(ocorrencias_especie = [6, 6, 6, 6, 6, 6], riqueza_lagoa = [3, 3, 4, 2, 3, 3, 3, 3, 2, 3, 3, 3])
```

Uma estatística comum é o **C-score** (Stone e Roberts 1990). Para cada par de espécies, conte as “unidades em tabuleiro de xadrez”: com r_i e r_j o número de locais que cada uma ocupa e S o número de locais que compartilham, o par contribui com $(r_i - S)(r_j - S)$. O C-score é a média sobre todos os pares. Quanto menos as espécies compartilham locais, maior ele é.

i Sua vez 11.6 — O C-score

Escreva `cscore(M)` para uma matriz com espécies nas linhas e locais nas colunas, e verifique com `checar("11.6", cscore)`. O restante desta seção usa essa função.

Dica

`r = vec(sum(M, dims = 2))` dá as ocorrências de cada espécie. Percorra os pares com `for i in 1:s-1, j in i+1:s, onde s = size(M, 1);` os locais compartilhados são `sum(M[i, :] & M[j, :])`.

Solução

```
function cscore(M)
    r = vec(sum(M, dims = 2))
    s = size(M, 1)
    total, pares = 0.0, 0
    for i in 1:s-1, j in i+1:s
        compartilhados = sum(M[i, :] .& M[j, :])
        total += (r[i] - compartilhados) * (r[j] -
↪   compartilhados)
        pares += 1
    end
    return total / pares
end

cscore(M)
```

13.866666666666667

```
checar("11.6", cscore)
```

✓ Correto!

Um C-score de cerca de 13,9 é alto? Depende do que assembleias aleatórias *destas espécies nestas lagoas* dariam. Espécies comuns compartilham mais locais que as raras só por acaso, e lagoas ricas têm mais espécies que as pobres; então um modelo nulo justo mantém fixos tanto o número de ocorrências de cada espécie quanto a riqueza de cada lagoa, e aleatoriza apenas quais espécies vão para onde (Gotelli 2000).

O algoritmo de **troca** (*swap*) faz exatamente isso. Ele sorteia duas espécies e duas lagoas; se formarem um tabuleiro — [1 0; 0 1] ou [0 1; 1 0] —, ele o inverte. Os totais de linhas e colunas nunca mudam:

```
function swap!(M; attempts = 1_000)
    nsp, nloais = size(M)
    for _ in 1:attempts
        i, j = sample(1:nsp, 2; replace = false)
        k, l = sample(1:nloais, 2; replace = false)
        if M[i, k] == M[j, l] && M[i, l] == M[j, k] && M[i, k] != M[i,
↪   l]
            M[i, k], M[i, l] = M[i, l], M[i, k]
            M[j, k], M[j, l] = M[j, l], M[j, k]
        end
    end
    return M
end

c_nulos = [cscore(swap!(copy(M))) for _ in 1:999]
```

```

c_obs = cscore(M)
(observado = c_obs,
 media_nula = mean(c_nulos),
 p = (count(c_nulos .>= c_obs) + 1) / (999 + 1),
 ses = (c_obs - mean(c_nulos)) / std(c_nulos))

```

```
(observado = 13.866666666666667, media_nula = 12.801067734401064, p = 0.011, ses = 3.35475)
```

```

fig = Figure(size = (700, 380))
ax = Axis(fig[1, 1], xlabel = "C-score", ylabel = "Assembleias nulas")
hist!(ax, c_nulos, bins = 25, color = (:grey60, 0.8))
vlines!(ax, c_obs, color = :darkorange, linestyle = :dash, linewidth =
↪ 2)
fig

```

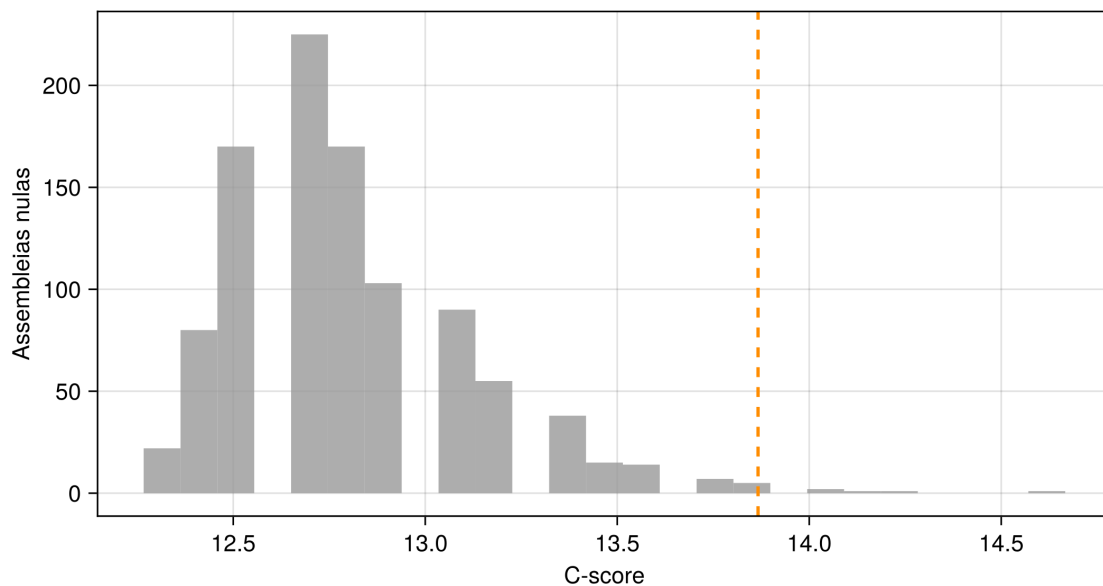


Figura 11.3: C-scores de 999 assembleias nulas com as mesmas frequências de espécies e riquezas de lagoas que a observada. A linha tracejada marca o C-score observado.

O C-score observado fica na cauda superior: cerca de 1% das assembleias nulas são tão segregadas quanto ela. O teste é unilateral, porque a pergunta era sobre *menos* coocorrência que o esperado. O **tamanho de efeito padronizado** (ses), a distância até a média nula em desvios-padrão nulos, é a forma usual de comparar resultados como esse entre matrizes de tamanhos diferentes.

Fazendo isso de verdade

Esta é uma versão didática. Para uma análise real, o pacote `vegan` do R (`nullmodel`, `oecosimu`) implementa muitos algoritmos nulos com o cuidado que eles exigem — inclusive quantas trocas bastam para que as matrizes nulas sejam independentes, algo que esta versão trata por alto. A Seção 9.7.1 mostra como chamá-lo a partir de Julia.

Desafios

1. Repita o teste de permutação da Seção 11.2 com `median` em vez de `mean` como estatística. Qual você reportaria para o tamanho corporal, e por quê?
2. Refaça a análise de poder para um efeito de 10% ($e^\beta = 1,10$). Quantas lagoas seriam necessárias? O que você diria a um colega que planeja um levantamento de 40 lagoas?
3. Monte uma matriz em que só *um* par de espécies é perfeitamente segregado e as demais estão espalhadas ao acaso. O modelo nulo fixo-fixo detecta isso? O que isso diz sobre o C-score como medida da interação de um único par?

Referências

- Anscombe, F. J. 1973. «Graphs in statistical analysis». *The American Statistician* 27 (1): 17–21. <https://doi.org/10.1080/00031305.1973.10478966>.
- Bates, Douglas, Phillip Alday, Dave Kleinschmidt, et al. 2026. *MixedModels.jl: A Julia package for fitting (statistical) mixed-effects models*. Versão 5.8.2. Released. <https://github.com/JuliaStats/MixedModels.jl>.
- Bates, Douglas, Martin Mächler, Ben Bolker, e Steve Walker. 2015. «Fitting Linear Mixed-Effects Models Using lme4». *Journal of Statistical Software* 67 (1): 1–48. <https://doi.org/10.18637/jss.v067.i01>.
- Bezanson, Jeff, Alan Edelman, Stefan Karpinski, e Viral B. Shah. 2017. «Julia: A Fresh Approach to Numerical Computing». *SIAM Review* 59 (1): 65–98. <https://doi.org/10.1137/141000671>.
- Chamberlin, T. C. 1890. «The Method of Multiple Working Hypotheses». *Science* ns-15 (366): 92–96. <https://doi.org/10.1126/science.ns-15.366.92>.
- Da Silva, Fernando Rodrigues, Thiago Gonçalves-Souza, Gustavo Brant Paterno, Diogo Borges Provete, e Maurício Humberto Vancine. 2022. *Análises Ecológicas no R*. Nupeea; Canal 6. <https://analises-ecologicas.com/>.
- Danisch, Simon, e Julius Krumbiegel. 2021. «Makie.jl: Flexible high-performance data visualization for Julia». *Journal of Open Source Software* 6 (65): 3349. <https://doi.org/10.21105/joss.03349>.
- Efron, Bradley. 1979. «Bootstrap methods: another look at the jackknife». *The Annals of Statistics* 7 (1). <https://doi.org/10.1214/aos/1176344552>.
- Gotelli, Nicholas J. 2000. «Null model analysis of species co-occurrence patterns». *Ecology* 81 (9): 2606–21. [https://doi.org/10.1890/0012-9658\(2000\)081%5B2606:NMAOSC%5D2.0.CO;2](https://doi.org/10.1890/0012-9658(2000)081%5B2606:NMAOSC%5D2.0.CO;2).
- Hurlbert, Stuart H. 1984. «Pseudoreplication and the Design of Ecological Field Experiments». *Ecological Monographs* 54 (2): 187–211. <https://doi.org/10.2307/1942661>.
- Oliveira, Alexandre Adalardo de. 2026. *ecoR: curso de introdução à linguagem R*. Wiki, Instituto de Biociências, Universidade de São Paulo. <https://ecor.ib.usp.br/>.
- Phipson, Belinda, e Gordon K. Smyth. 2010. «Permutation P-values should never be zero: calculating exact P-values when permutations are randomly drawn». *Statistical Applications in Genetics and Molecular Biology* 9 (1). <https://doi.org/10.2202/1544-6115.1585>.

- Rodrigues, Rodolpho Credo, Érica Hasui, Julia Camara Assis, João Carlos Castro Pena, Renata L. Muylaert, et al. 2019. «ATLANTIC BIRD TRAITS: a data set of bird morphological traits from the Atlantic forests of South America». *Ecology* 100 (6): e02647. <https://doi.org/10.1002/ecy.2647>.
- Stone, Lewi, e Alan Roberts. 1990. «The checkerboard score and species distributions». *Oecologia* 85 (1): 74–79. <https://doi.org/10.1007/BF00317345>.
- Zuur, Alain F., Elena N. Ieno, e Chris S. Elphick. 2010. «A protocol for data exploration to avoid common statistical problems». *Methods in Ecology and Evolution* 1 (1): 3–14. <https://doi.org/10.1111/j.2041-210X.2009.00001.x>.
- Zuur, Alain F., Elena N. Ieno, Neil J. Walker, Anatoly A. Saveliev, e Graham M. Smith. 2009. *Mixed Effects Models and Extensions in Ecology with R*. Statistics for Biology e Health. Springer. <https://doi.org/10.1007/978-0-387-87458-6>.

Histórico de versões

Cada versão do livro é arquivada no Zenodo com um DOI próprio (veja “Como citar” no [README](#)). Os números seguem o versionamento semântico: a mudança no primeiro número indica uma reorganização grande, no segundo, conteúdo novo, e no terceiro, correções.

v1.0.0 (2026-10-03)

Primeira versão citável.

- Capítulos 1 a 11, em português e inglês: introdução, da pergunta ao modelo (só o plano), pré-requisitos, noções básicas de Julia, manipulação de dados, visualização, análise exploratória, modelos lineares, GLMs, funções, e reamostragem e simulação.
- Exercícios com correção automática (checar).
- PDFs das duas edições.